

MySQL Essentials 9



MySQL 9 Essentials

MySQL 9 Essentials

ISBN-13: 978-1-965764-10-7

© 2025 Neil Smyth / Payload Media, Inc. All Rights Reserved.

This book is provided for personal use only. Unauthorized use, reproduction and/or distribution strictly prohibited. All rights reserved.

The content of this book is provided for informational purposes only. Neither the publisher nor the author offers any warranties or representation, express or implied, with regard to the accuracy of information contained in this book, nor do they accept any liability for any loss or damage arising from any errors or omissions.

This book contains trademarked terms that are used solely for editorial purposes and to the benefit of the respective trademark owner. The terms used within this book are not intended as infringement of any trademarks.

Rev: 1.0a



<https://www.payloadbooks.com>

Table of Contents

1. Start Here.....	1
1.1 About MySQL	1
1.2 Downloading the database snapshots.....	2
1.3 Importing the database snapshots.....	2
1.4 Feedback.....	2
1.5 Errata.....	2
1.6 Knowledge tests	2
2. The Basics of Databases.....	3
2.1 Database vs. DBMS	3
2.2 Client-server databases.....	3
2.3 What is a database server?.....	4
2.4 Understanding database tables	4
2.5 Introducing database schema	4
2.6 Columns and data types	4
2.7 Database rows	5
2.8 Primary keys	5
2.9 What is SQL?.....	5
2.10 Knowledge test.....	6
2.11 Reference points	6
3. Installing and Launching MySQL on Windows.....	7
3.1 Downloading MySQL for Windows	7
3.2 Running the software installer.....	7
3.3 Configuring and starting MySQL Server	9
3.3.1 Data directory	9
3.3.2 Type and network settings	10
3.3.3 Accounts and roles	11
3.3.4 Windows service	11
3.3.5 Server file permissions.....	12
3.3.6 Sample databases	13
3.3.7 Applying the configuration	13
3.4 Setting the PATH environment variable	13
3.5 Securing MySQL Server on Windows.....	15
3.6 Launching the mysql client	16
3.7 Knowledge test.....	17
3.8 Reference points	17
4. Installing and Launching MySQL on macOS	19
4.1 Downloading MySQL for macOS	19
4.2 Running the package installer	19
4.3 Using the Preferences Pane	22
4.3.1 The Instances pane	23
4.3.2 The Configuration pane.....	23

Table of Contents

4.4 Adding MySQL to your PATH	23
4.5 Starting MySQL Server using launch control	24
4.6 Starting MySQL Server using mysql.server	24
4.7 Securing MySQL Server on macOS	25
4.8 Launching the mysql client	26
4.9 Knowledge test	27
4.10 Reference points	27
5. Installing and Launching MySQL on Linux.....	29
5.1 Downloading MySQL for Linux	29
5.2 Installing MySQL on RPM-based distributions	30
5.3 Securing MySQL Server on RPM-based systems	30
5.4 Installing MySQL on Debian and Ubuntu	31
5.5 Securing MySQL Server on Debian and Ubuntu	32
5.6 Launching the mysql client	33
5.7 Knowledge test	34
5.8 Reference points	34
6. The MySQL Client	37
6.1 The mysql command-line tool	37
6.2 Using the mysql client	38
6.3 Typing and editing SQL statements	39
6.4 Exiting from the mysql client	40
6.5 Knowledge test	40
6.6 Reference points	40
7. MySQL Users, Privileges, and Security	43
7.1 MySQL security	43
7.2 Getting information about users	43
7.3 Adding a MySQL user account	44
7.4 Deleting a MySQL user	45
7.5 Renaming a MySQL user	45
7.6 Changing the password for a MySQL user	45
7.7 Granting privileges	46
7.8 Privileges and grantable levels	46
7.9 Privilege examples	48
7.10 Removing remote root access	49
7.11 Managing password verification	49
7.12 Knowledge test	50
7.13 Reference points	50
8. Creating Databases and Tables	53
8.1 Creating a new MySQL database	53
8.2 Enabling table encryption	54
8.3 Creating tables with SQL	54
8.4 Understanding NULL and NOT NULL values	55
8.5 Primary keys	55
8.6 Using AUTO_INCREMENT	55
8.7 Defining default values during table creation	56
8.8 Displaying table schema	56

8.9 Deleting databases and tables.....	56
8.10 MySQL storage engine types.....	57
8.11 Knowledge test.....	58
8.12 Reference points	58
9. Inserting Data into Database Tables.....	61
9.1 Creating the sample table.....	61
9.2 The basics of data insertion.....	61
9.3 Inserting a row.....	61
9.4 Adding multiple rows to a table	63
9.5 Inserting results from a SELECT statement.....	64
9.6 Reducing the INSERT performance load.....	65
9.7 Knowledge test.....	65
9.8 Reference points	65
10. Updating and Managing MySQL Tables	67
10.1 Opening the sample database.....	67
10.2 Altering a database table	67
10.3 Adding table columns.....	67
10.4 Moving table columns	69
10.5 Renaming tables	69
10.6 Renaming table columns.....	69
10.7 Deleting table columns.....	70
10.8 Changing the data type of a column.....	70
10.9 Deleting tables	71
10.10 Duplicating tables	71
10.11 Knowledge test	72
10.12 Reference points.....	72
11. Updating and Deleting Table Data.....	75
11.1 Opening the sample database.....	75
11.2 Updating database data	75
11.3 Ignoring update errors.....	76
11.4 Understanding auto-commit	76
11.5 Transactions and rollbacks.....	78
11.6 Deleting table rows.....	78
11.7 Knowledge test.....	79
11.8 Reference points	79
12. Retrieving Data from MySQL Databases.....	81
12.1 Opening the sample database.....	81
12.2 Retrieving a single column	81
12.3 Using SELECT to retrieve multiple columns.....	82
12.4 Restricting the number of results.....	83
12.5 Eliminating duplicate values.....	83
12.6 Sorting result sets	84
12.7 Sorting on multiple columns	85
12.8 Sorting data in descending order.....	85
12.9 Expressions and aggregate functions.....	86
12.10 Knowledge test	88

Table of Contents

12.11 Reference points	88
13. Filtering Result Sets with the WHERE Clause.....	91
13.1 Opening the sample database.....	91
13.2 The basics of the WHERE clause	91
13.3 Comparison Operators.....	92
13.4 Checking for NULL values	93
13.5 Searching within value ranges	93
13.6 Knowledge test.....	94
13.7 Reference points	94
14. Filtering Results with Logical Operators.....	95
14.1 Opening the sample database.....	95
14.2 Filtering results with the OR operator.....	95
14.3 Filtering results with the AND operator	96
14.4 Combining AND and OR operators.....	96
14.5 Understanding operator precedence	96
14.6 Specifying condition ranges using the IN operator	97
14.7 Using the NOT Operator	97
14.8 Using subqueries with WHERE IN clauses	98
14.9 Knowledge test.....	99
14.10 Reference points	99
15. Wildcard and Regular Expression Matching	101
15.1 What are wildcards?.....	101
15.2 Opening the sample database.....	101
15.3 Single character wildcards.....	102
15.4 Multiple character wildcards	102
15.5 What are regular expressions?	103
15.6 Regular expression character matching	104
15.7 Matching with a group of characters	104
15.8 Matching character ranges	104
15.9 Handling special characters.....	105
15.10 Whitespace metacharacters	105
15.11 Matching by character type	106
15.12 Repetition metacharacters	108
15.13 Matching by text position	109
15.14 Replacing text with regular expressions.....	110
15.15 Knowledge test	111
15.16 Reference points	111
16. Understanding Joins and Unions	113
16.1 How joins work.....	113
16.2 Opening the sample database.....	114
16.3 Creating the supplier table	114
16.4 Adding the foreign key to the product table.....	115
16.5 Adding key values to supplier_id.....	115
16.6 Performing a cross join.....	116
16.7 Performing an inner join.....	117
16.8 Performing left joins	118

16.9 Performing right joins	119
16.10 Understanding WHERE in join statements	120
16.11 Working with unions	120
16.12 Full outer joins in MySQL	122
16.13 Knowledge test	124
16.14 Reference points	124
17. An Introduction to MySQL Views	127
17.1 An introduction to MySQL views	127
17.2 Opening the sample database	128
17.3 Creating a basic view	128
17.4 Joins and views	130
17.5 Updating view-based tables	131
17.6 Updating join-based views	133
17.7 Altering existing views	134
17.8 Deleting and replacing views	134
17.9 Knowledge test	135
17.10 Reference points	135
18. Understanding MySQL Indexes	137
18.1 Understanding MySQL indexes	137
18.2 Primary index	137
18.3 Unique index	137
18.4 Regular index	138
18.5 Full-text indexes	138
18.6 Composite index	139
18.7 Spatial index	140
18.8 Getting index details	140
18.9 Deleting indexes	141
18.10 Limitations of indexes	141
18.11 Knowledge test	142
18.12 Reference points	142
19. MySQL Stored Routines	145
19.1 An overview of stored routines	145
19.2 The benefits of stored routines	145
19.3 Creating stored procedures	145
19.4 Opening the sample database	146
19.5 Working with MySQL variables	146
19.6 Stored procedure examples	147
19.7 Creating stored functions	148
19.7.1 RETURNS	149
19.7.2 RETURN	149
19.7.3 DETERMINISTIC	149
19.7.4 NON DETERMINISTIC	149
19.7.5 NO SQL	149
19.7.6 READS SQL DATA	149
19.7.7 MODIFIES SQL DATA	149
19.7.8 CONTAINS SQL	149

Table of Contents

19.8 Stored function examples.....	150
19.9 Using stored functions as subqueries	152
19.10 Introducing Common Table Expressions	153
19.11 Viewing procedures and functions	153
19.12 Deleting stored procedures and functions.....	154
19.13 Knowledge test	155
19.14 Reference points	155
20. Working with Control Flow	157
20.1 Opening the sample database.....	157
20.2 Looping in MySQL.....	157
20.2.1 The WHILE loop.....	157
20.2.2 The REPEAT loop	159
20.3 The LOOP construct.....	160
20.4 Conditional Statements in MySQL	165
20.4.1 The IF statement.....	165
20.4.2 The CASE statement	166
20.5 Knowledge test.....	169
20.6 Reference points	169
21. Error Handling in MySQL.....	171
21.1 Overview of MySQL error handling.....	171
21.2 Deciphering SQLSTATE codes.....	171
21.3 Opening the sample database.....	171
21.4 Declaring a handler.....	172
21.5 Error handling with SQLEXCEPTION	172
21.6 Error handling with SQLWARNING and NOT FOUND	175
21.7 Handling specific errors with SQLSTATE	175
21.8 Custom error handling with SIGNAL	177
21.9 Knowledge test.....	178
21.10 Reference points	178
22. Automation with MySQL Triggers	181
22.1 An overview of MySQL triggers.....	181
22.2 The syntax for creating triggers	181
22.3 Accessing trigger event data	181
22.4 Triggers in action.....	182
22.4.1 An AFTER INSERT example	182
22.4.2 A BEFORE INSERT example	183
22.4.3 A BEFORE DELETE example	184
22.5 Displaying trigger information.....	184
22.6 Deleting triggers.....	185
22.7 Limitations of triggers	185
22.8 Knowledge test.....	185
22.9 Reference points	185
23. Scheduling Tasks with MySQL Events	187
23.1 An introduction to MySQL events.....	187
23.2 Enabling the MySQL event scheduler	187
23.3 Creating Events.....	188

23.4 Opening the sample database.....	188
23.5 Adding the event_tests table.....	188
23.6 Creating a one-time event.....	189
23.7 Creating recurring events.....	189
23.8 Viewing event details.....	190
23.9 Preserving events.....	191
23.10 Altering event properties.....	192
23.11 Deleting events.....	193
23.12 Knowledge test.....	194
23.13 Reference points.....	194
24. An Introduction to mysqladmin.....	197
24.1 Setting up mysqladmin.....	197
24.2 Basic mysqladmin syntax.....	197
24.3 Checking server status.....	198
24.4 Shutting down the server.....	198
24.5 Resetting user passwords.....	198
24.6 Flushing privileges and logs.....	199
24.7 Creating and dropping databases.....	199
24.8 Viewing server variables.....	199
24.9 Connecting to remote servers.....	199
24.10 Knowledge test.....	199
24.11 Reference points.....	199
25. Configuring MySQL using the my.cnf File.....	203
25.1 Introducing the my.cnf File.....	203
25.2 The my.cnf File Structure.....	203
25.3 Key my.cnf parameters.....	204
25.3.1 General server settings.....	204
25.3.2 Performance tuning.....	204
25.3.3 Storage engine configuration.....	205
25.3.4 Logging and debugging.....	205
25.3.5 Security settings.....	205
25.3.6 Applying my.cnf file changes.....	205
25.4 Knowledge test.....	206
25.5 Reference points.....	206
26. Installing MySQL Workbench.....	209
26.1 Downloading MySQL Workbench.....	209
26.2 Installation on Windows.....	209
26.3 Installation on macOS.....	211
26.4 Installation on RPM-based Linux distributions.....	211
26.5 Installation on Ubuntu.....	212
26.6 Knowledge test.....	213
26.7 Reference points.....	213
27. MySQL Workbench Administration.....	215
27.1 Creating a server connection.....	215
27.2 The MySQL Workbench user interface.....	217
27.3 Managing servers with MySQL Workbench.....	218

Table of Contents

27.3.1 Server status	218
27.3.2 Client connections	219
27.3.3 Users and privileges	220
27.3.4 Status and system variables	220
27.4 Importing and exporting data	221
27.5 Performance monitoring	221
27.6 Instance management	222
27.7 Knowledge test.....	224
27.8 Reference points	224
28. MySQL Workbench Queries and Models.....	227
28.1 Introducing the SQL Editor	227
28.2 Creating a database model	230
28.3 Adding a user to the model.....	232
28.4 Generating the database creation script.....	232
28.5 Generating EER diagrams.....	234
28.6 Knowledge test.....	236
28.7 Reference points	236
29. Database Administration with phpMyAdmin.....	239
29.1 Introducing phpMyAdmin	239
29.2 Setting up a web server on Red Hat-based Linux distributions.....	239
29.3 Testing the web server	240
29.4 Installing phpMyAdmin	240
29.5 Securing phpMyAdmin access	241
29.6 Opening phpMyAdmin	242
29.7 Knowledge test.....	242
29.8 Reference points	242
30. A Guided Tour of phpMyAdmin.....	245
30.1 The phpMyAdmin home screen.....	245
30.2 Using the console	246
30.3 Managing databases.....	247
30.4 Running SQL queries.....	248
30.5 Monitoring server status	248
30.6 User accounts.....	249
30.7 Importing and exporting data	249
30.8 Working with database diagrams.....	249
30.9 Knowledge test.....	250
30.10 Reference points	250
Index	253

1. Start Here

Databases are the foundation of applications like websites, mobile apps, and large-scale enterprise systems. MySQL is one of the most widely deployed and highly regarded database management systems in use today. MySQL 9 Essentials provides a concise guide to MySQL, covering fundamental concepts, techniques, and best practices.

The book begins by installing and configuring MySQL on Windows, macOS, and Linux before outlining the fundamentals of relational database management systems.

MySQL 9 Essentials also covers key MySQL features such as views, stored routines, indexing for performance optimization, automation with triggers and events, and database modeling with MySQL Workbench.

In addition to covering the command-line tools provided with MySQL, several chapters introduce the phpMyAdmin and MySQL Workbench tools, which offer user-friendly graphical interfaces for database management.

By following this book, you will learn how to:

- Install and configure MySQL on Windows, macOS, and Linux.
- Use MySQL client tools and graphical interfaces like MySQL Workbench and phpMyAdmin.
- Design efficient database schemas and relationships.
- Manage databases using the Structured Query Language (SQL).
- Optimize database performance using indexing and query optimization techniques.
- Automate repetitive database tasks using triggers and scheduled events.
- Secure MySQL databases by managing user privileges and access controls.

This book explains each topic in detail and includes practical examples that provide hands-on experience. The chapters also contain quick-reference summaries highlighting key points for easy review and access to online knowledge quizzes to assess and reinforce your understanding.

By the end of this book, you will have the confidence to build and manage MySQL databases.

1.1 About MySQL

Before the arrival of MySQL, implementing a database was typically a complex and expensive task involving the purchase, installation, and maintenance of a proprietary database management system from a vendor such as Oracle or IBM. In contrast, MySQL provides a free, open-source database management system that is easy to install, implement, and maintain. In addition, MySQL is fast, extremely reliable, and widely deployed by many companies and organizations worldwide.

It is not an exaggeration to say that MySQL has brought the power of a fully featured, scalable relational database management system into the reach of anyone with a computer and the desire to build a data-driven application or website. MySQL was originally developed by MySQL AB, a company founded in Sweden in 1995, and remained independent until it was acquired by Sun Microsystems in 2008. Oracle purchased Sun Microsystems in 2008

and found itself owning MySQL. Oracle provides a free community and a subscription-based enterprise edition of MySQL. Though the core elements of the two editions are identical, the enterprise edition includes additional scaling, performance, security, backup features, and 24x7 support. To address the broadest possible audience, this book is based on MySQL 9 Community Edition.

The name MySQL comes from a combination of “My” and “SQL.” In this context, “My” is a proper noun rather than a possessive adjective and is named after My, the daughter of Monty Widenius, one of the original developers of MySQL. SQL stands for Structured Query Language, the language used for managing databases.

1.2 Downloading the database snapshots

Many chapters in this book assume that you have completed the steps from previous chapters. If you would rather not read the chapters in sequence, you can import database snapshots at the beginning of each chapter using the snapshot files available for download at the following link:

<https://www.payloadbooks.com/product/mysql9-prag/>

1.3 Importing the database snapshots

To import a snapshot, open a terminal or command prompt, navigate to the directory that contains the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampledb;"  
mysql -u root -p sampledb < snapshot_file_name.sql
```

1.4 Feedback

We want you to be satisfied with your purchase of this book. Therefore, if you find any errors in the book or have any comments, questions, or concerns, please contact us at info@payloadbooks.com.

1.5 Errata

While we make every effort to ensure the accuracy of the content of this book, inevitably, a book covering a subject area of this size and complexity may include some errors and oversights. Any known issues with the book will be outlined, together with solutions, at the following URL:

https://www.payloadbooks.com/mysql9_errata

If you find an error not listed in the errata, email our technical support team at info@payloadbooks.com.

1.6 Knowledge tests



Look for this section at the end of most chapters and use the link or scan the QR code to take a knowledge quiz to test and reinforce your understanding of the covered topic. Use the following link to review the full list of tests available for this book:

<https://www.answertopia.com/ce9o>



2. The Basics of Databases

The chances are that if you have ever logged into a website or purchased an item on the internet you have interacted with a database in some way. Anything that involves the retrieval or storage of information on a computer system is most likely to involve a database. In fact, databases are the core of almost every application that relies on data of some form to complete a task. In this chapter, we will introduce the basic concepts of databases.

2.1 Database vs. DBMS

The first step in learning MySQL is understanding the difference between a database and a database management system (DBMS). The term database refers to the entity that stores the actual data (such as ID numbers, names, and addresses, for example) in a structured way. A database management system (DBMS), on the other hand, refers to the software used to store, access, and manipulate the data stored in the database. All interactions with the database are performed via the DBMS.

Modern databases and database management systems are not restricted to storing just text. Today, databases store such items as images, videos, and software objects.

2.2 Client-server databases

MySQL is classified as a client-server database management system (DBMS). This type of DBMS consists of two main components. The server, which usually resides on the same physical computer as the database files, is responsible for all interactions with the database. The second component is the client, which sends database requests to the server. The server processes these requests and returns the results to the client.

There are several key advantages to using a client-server architecture for a database management system (DBMS). First, the client does not need to run on the same computer as the server. Instead, clients can send requests over a network or internet connection to a server located on a remote host. This setup makes the database accessible to a large number of clients. In large-scale enterprise environments, it also allows for fault tolerance, high performance, and load balancing to be implemented effectively.

Second, separating the client from the server allows a wider range of client types to be used to access the database. Typical clients include MySQL tools, desktop and mobile apps, web-based applications, web servers, and even other database servers:

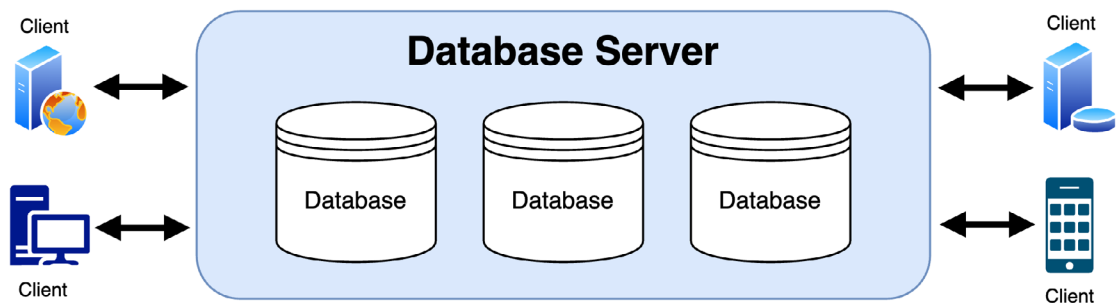


Figure 2-1

2.3 What is a database server?

The term “database server” can be somewhat misleading as it can refer to different concepts. One definition relates to the computer system hosting a Database Management System (DBMS) and other applications and services. However, in this book, we will specifically refer to the software component of a DBMS responsible for executing database operations on behalf of clients and returning the results. In the context of MySQL, this role is fulfilled by MySQL Server.

A database server can contain multiple databases, each containing one or more tables.

2.4 Understanding database tables

Database tables provide the most basic level of data structure in a database. Each database can contain multiple tables, each designed to hold information of a specific type. For example, a database may contain a customer table containing the name, address, and telephone number for all the customers of a particular business. The same database may also include a product table that stores the product descriptions with associated product codes and pricing for the items the business sells.

Each table in a database is assigned a name that must be unique to that particular database. A table name, once assigned to a table in one database, may only be re-used within the context of a different database:

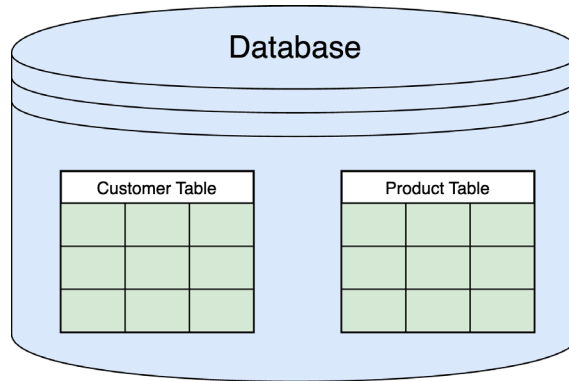


Figure 2-2

2.5 Introducing database schema

Database schemas define the characteristics of the data stored in a database table. For example, the schema for a customer database table might define that the customer name is a string of no more than 20 characters in length and that the customer phone number is a numerical data field of a specific format.

Schemas are also used to define the structure of entire databases and the relationship between the various tables in each database.

2.6 Columns and data types

At this stage, it is helpful to begin viewing a database table similar to a spreadsheet, where data is stored in rows and columns.

Each column represents a data field in the corresponding table. For example, a table’s name, address, and telephone data fields are all columns.

Each column, in turn, is defined to contain a specific data type, which dictates the type of data it can store. Therefore, a column designed to store numbers would be defined as a numerical data type.

2.7 Database rows

Each new record saved to a table is stored in a row, which consists of the columns of data associated with the saved record.

Once again, consider the spreadsheet analogy described earlier. Each entry in a customer table is equivalent to a row in a spreadsheet, and each column contains the data for each customer (name, address, telephone number, etc.). The individual columns within a specific row are referred to as *fields*.

When a new customer is added to the table, a new row is created, and the data for that customer is stored in the corresponding columns of the new row.

Rows are also sometimes referred to as *records*, and these terms can generally be used interchangeably:

Table				
Rows	Columns			
	customer_id	customer_name	customer_address	customer_phone
	1001	John Smith	London	123-54355
	1002	David West	Miami	424-09238
	1003	Mark Williams	Paris	234-84509
	1004	Sarah Parker	New York	768-30895

Figure 2-3

2.8 Primary keys

Each database table must contain one or more columns that uniquely identify each row. This is known in database terminology as the *primary key*. For example, a table may use a bank account number column as the primary key. Alternatively, a customer table may assign unique identifiers to each customer as the primary key.

Primary keys allow the database management system to uniquely identify a specific row in a table. Without a primary key, retrieving or deleting a specific row in a table would be impossible because there is no certainty that the correct row has been selected. For example, suppose a table existed where the customer's last name had been defined as the primary key. Imagine the problem if more than one customer called "Smith" was recorded in the database. Without some guaranteed way to uniquely identify a specific row, it would be impossible to ensure the correct data was being accessed at any given time.

Primary keys can comprise a single column or multiple columns in a table. To qualify as a single column primary key, no two rows can contain matching primary key values. When using multiple columns to construct a primary key, individual column values do not need to be unique, but all the columns combined must be unique.

Finally, while primary keys are not mandatory in database tables, their use is strongly recommended.

2.9 What is SQL?

As discussed previously, a database management system (DBMS) provides the means to access the data stored in a database. One key method for achieving this is via a language called Structured Query Language (SQL), which is abbreviated to SQL and pronounced "sequel".

SQL is a straightforward and easy-to-use language developed at IBM in the 1970s specifically to enable the reading and writing of database data. Because SQL contains a small set of keywords, it can be learned quickly. In addition, SQL syntax is identical in most DBMS implementations, so having learned SQL for one system, your

skills will likely transfer to other database management systems.

Throughout this book, particular attention will be paid to explaining the key SQL commands so that you will be proficient in using SQL to read, write, and manage database data.

2.10 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of database architecture:

<https://www.answertopia.com/vwh2>



2.11 Reference points

The main points covered in this chapter are as follows:

- **Databases vs. DBMS**
 - A database stores structured data (e.g., names, addresses).
 - A DBMS (Database Management System) is software to access, store, and manipulate the database (e.g., MySQL).
- **Database Client-Server Architecture**
 - Databases operate on a server, and clients (apps, web servers) access them locally or remotely via a network.
 - A database server may host multiple databases, each with one or more tables.
- **Database Tables**
 - Tables organize data into rows (records) and columns (fields).
 - Each column has a specific data type (e.g., numerical, text).
- **Database Schema**
 - Defines the structure of a database and its tables, including data types and relationships.
- **Primary Keys**
 - Unique identifier for rows in a table (e.g., customer ID).
 - Can be a single column or a combination of columns.
 - Strongly recommended to ensure reliable row identification.
 - Proper table structure and primary keys ensure efficient data management and retrieval.
- **SQL (Structured Query Language)**
 - The primary language for interacting with databases.
 - Used for querying, updating, and managing database data.
 - SQL is simple, widely adopted, and transferable across DBMS platforms.

3. Installing and Launching MySQL on Windows

This chapter will explain how to install and run MySQL on Microsoft Windows. This will include instructions for starting and stopping the MySQL server. Additionally, we will cover how to use the MySQL client and explore various configuration options available for MySQL on Windows.

3.1 Downloading MySQL for Windows

MySQL for Windows is available as a Microsoft Software Installer (.MSI) file. To download the latest Community Edition of MySQL 9, open a web browser and visit the following page:

<https://dev.mysql.com/downloads/mysql/>

On the download page, use the menu at the top to select the latest version of MySQL 9 (marked as A in Figure 3-1). Additionally, set the operating system menu (B) to Microsoft Windows.

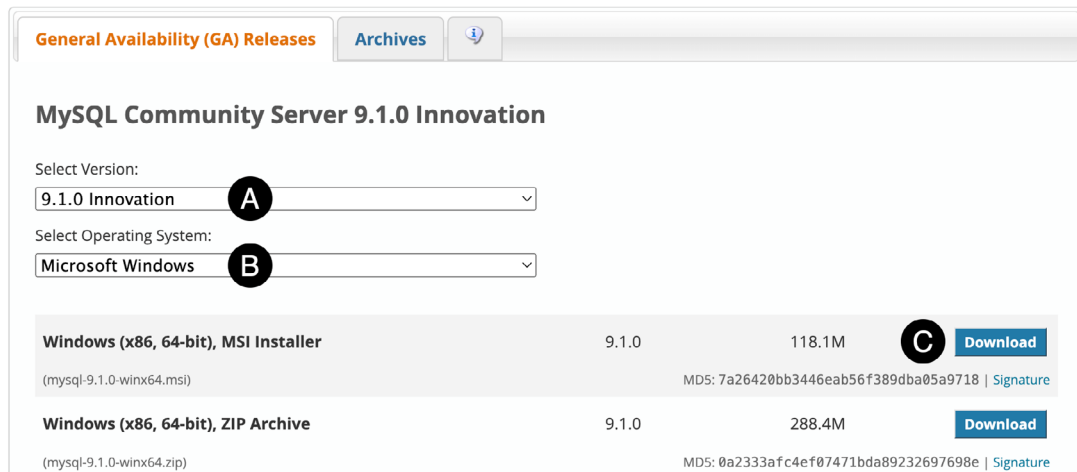


Figure 3-1

To download the MSI Installer, click the corresponding Download button (C). When prompted, you can either sign in with your Oracle web account or choose “No thanks, just start my download” to proceed without signing in.

3.2 Running the software installer

Using Windows Explorer, navigate to where you downloaded the MySQL MSI file and double-click it to begin the installation. Once the installer has loaded, the screen shown in Figure 3-2 will appear:

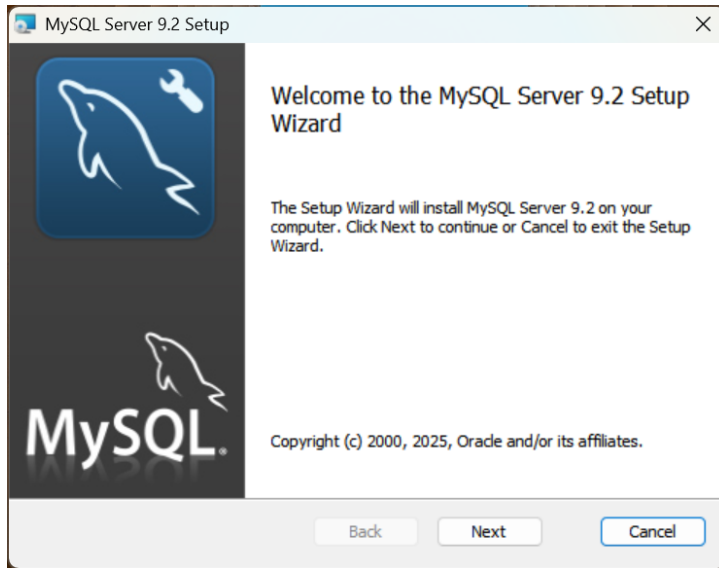


Figure 3-2

Click the Next button to review and accept the licensing terms before moving on to the setup type screen shown below:

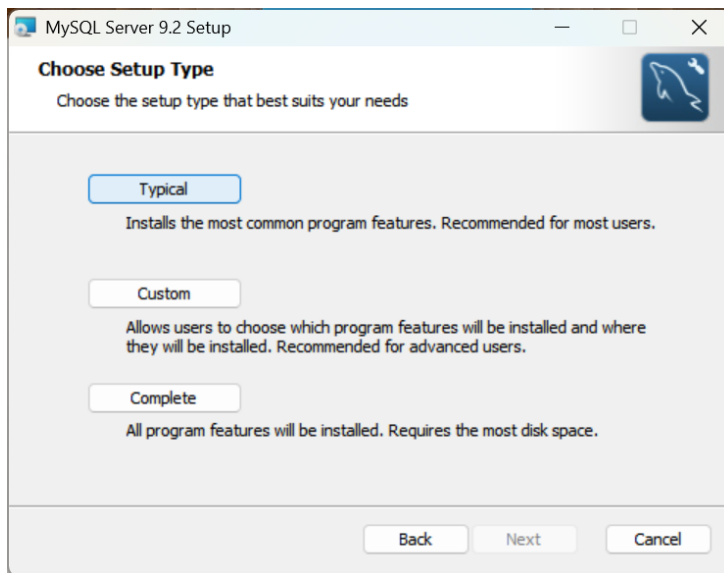


Figure 3-3

For the purposes of this book, the “typical” option will provide everything you need to learn MySQL, so select this option to start the installation process.

After completing the MySQL installation, the screen displayed in Figure 3-4 will appear. The next step is to configure and start the MySQL Server using the MySQL Configurator tool. Ensure the “Run MySQL Configurator” option is checked, then click the Finish button:

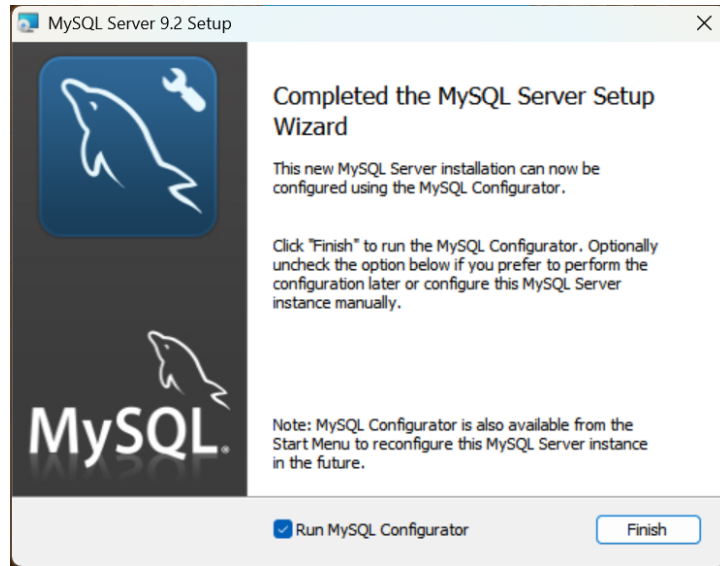


Figure 3-4

3.3 Configuring and starting MySQL Server

The MySQL Configurator tool configures the installation, initializes the database, and starts the server. It can be launched from the Windows start menu anytime to modify configuration settings. When you open the MySQL Configurator for the first time, it will display a welcome screen, as shown in Figure 3-5:

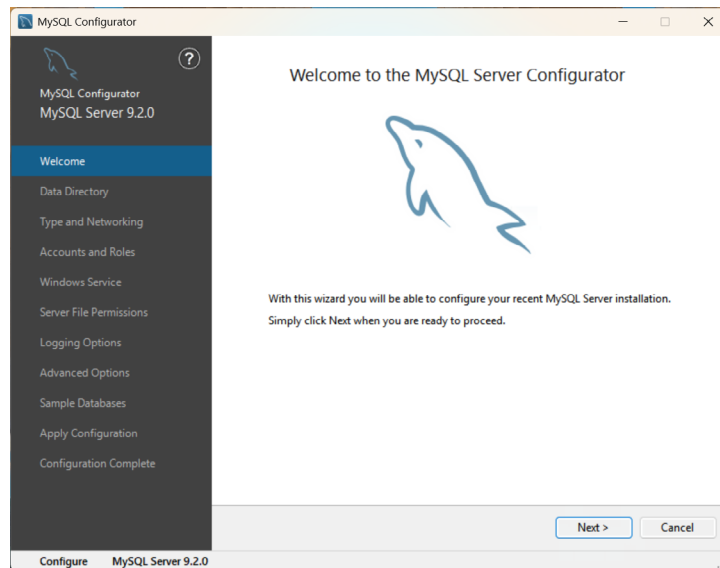


Figure 3-5

3.3.1 Data directory

Click Next to proceed to the Data Directory screen (Figure 3-6). The default location for the database files is as follows, where <version> is replaced by the version of MySQL that was installed:

```
C:\ProgramData\MySQL Server <version>\
```

Installing and Launching MySQL on Windows

For example, if you installed MySQL 9.2, the data directory will default to the following path:

```
C:\ProgramData\MySQL Server 9.2\
```

If you prefer to use a different directory for the database data, click the button labeled ‘..’ to select another location:

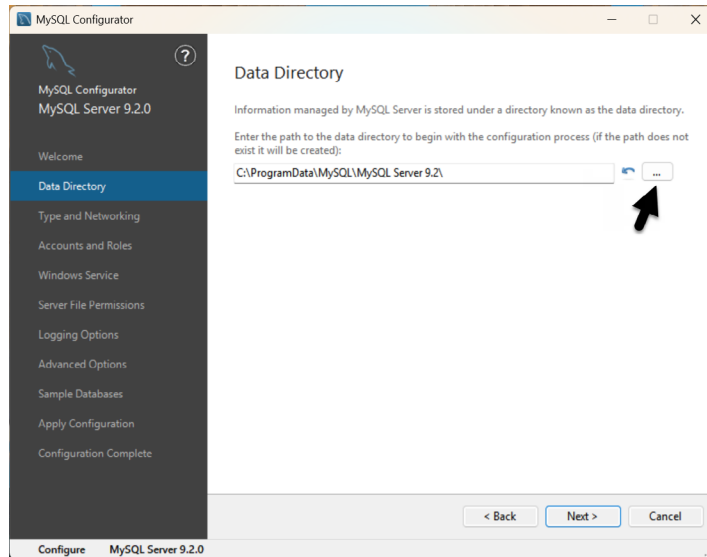


Figure 3-6

3.3.2 Type and network settings

The next screen contains settings related to the configuration type and network connectivity:

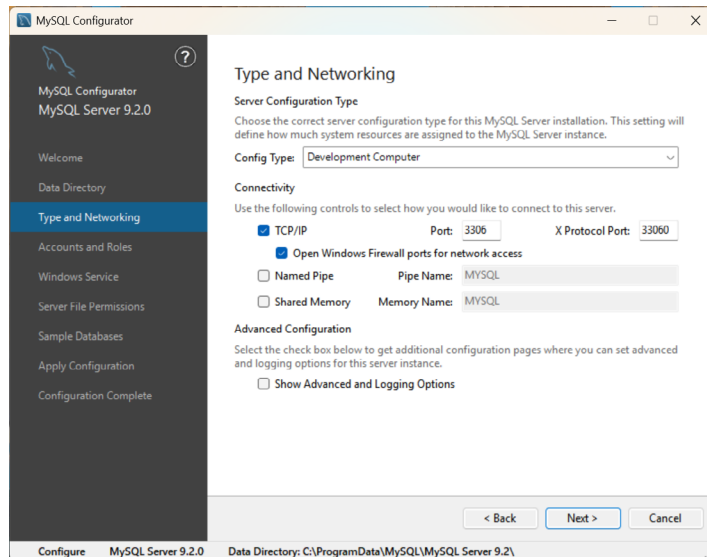


Figure 3-7

The Config Type menu allows you to configure the memory allocated to the MySQL Server during operation. The available types are as follows:

- **Development Computer** - This setting is intended for situations where MySQL is run on a personal computer during development, and the system is also used for other tasks not related to MySQL. To prevent any negative impact on the performance of other applications on the system, this option configures MySQL to use a minimal amount of memory.
- **Server Computer** - Choose this option for production servers that will be running additional services alongside MySQL Server, such as web and application servers. This option allocates more memory to MySQL compared to the Development type while maintaining the performance of the other services.
- **Dedicated Computer** - This configuration is intended for systems that run only MySQL Server, allowing the server to utilize all available system memory.
- **Manual** - This option prevents the MySQL Configurator from altering the memory allocation. As a result, MySQL will revert to the settings specified in the *my.ini* file located in the MySQL data directory, which was configured on the previous screen of the configurator.

The network settings manage access to MySQL Server from external sources. For most development installations, these settings typically do not require changes.

3.3.3 Accounts and roles

Use the Accounts and Roles screen to set a password for the MySQL root user account. The root account functions as a “superuser,” providing complete access and control over the databases managed by MySQL. For this reason, it is crucial to select a sufficiently complex password. You can also add additional user accounts in this section, though we will discuss this topic further later in the book:

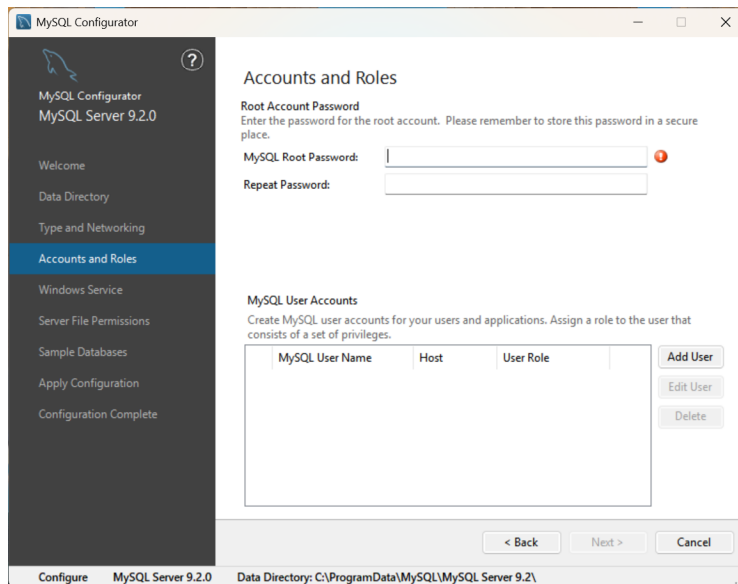


Figure 3-8

3.3.4 Windows service

The Windows Services screen determines how MySQL Server operates on the system. MySQL can run as a managed Windows service or be launched manually by executing the *mysqld.exe* file. However, launching the server manually involves additional steps and requires passing several command-line configuration options to *mysqld.exe*. Using the Windows service option is recommended unless you have a specific requirement that necessitates manual execution.

Installing and Launching MySQL on Windows

When the Windows service option is enabled, you can configure MySQL Server to start automatically when the system boots up. By default, the MySQL Windows service runs under the system's NetworkService user account. However, you can choose to use a different user account. Keep in mind that this account must have the appropriate permissions and access rights on the Windows system. Setting this up requires advanced knowledge of Windows system administration:

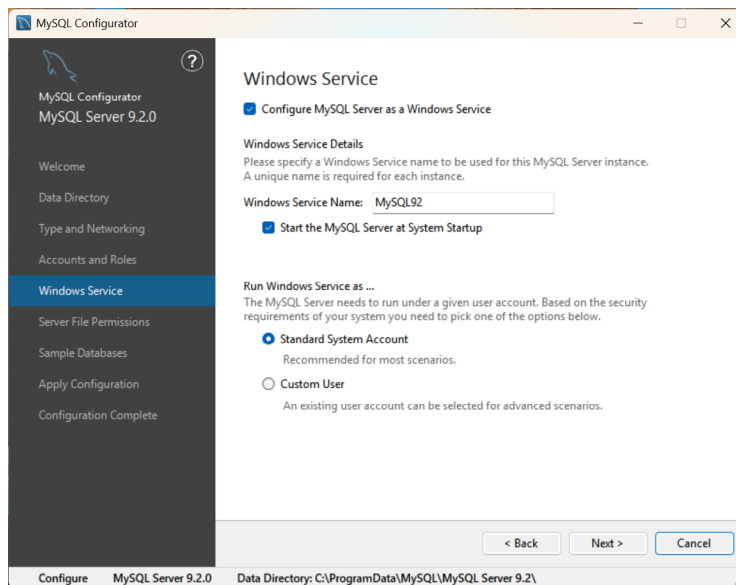


Figure 3-9

3.3.5 Server file permissions

The Server File Permissions screen lets you configure access controls for files in the data directory. Generally, it is advisable to grant access to both the current user (likely yourself) and system administrators:

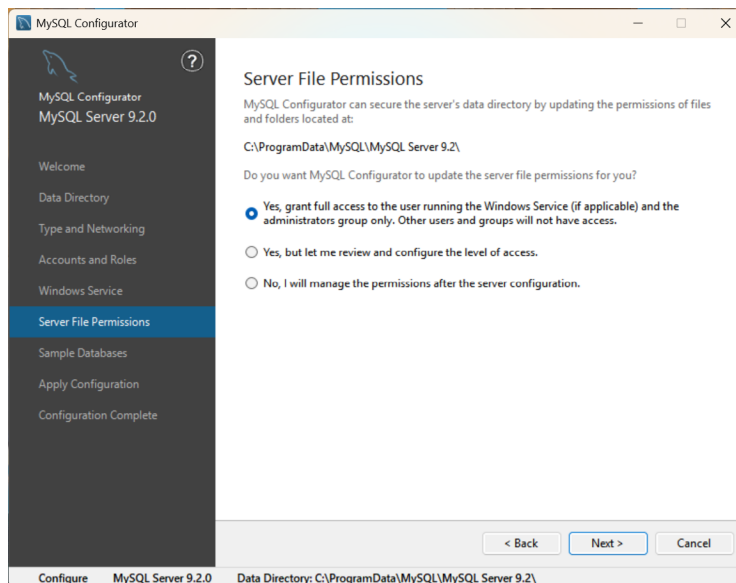


Figure 3-10

3.3.6 Sample databases

MySQL for Windows includes a collection of sample databases that can be installed for educational purposes. Although these databases will not be utilized in this book, you are welcome to install them on your system.

3.3.7 Applying the configuration

Before using MySQL, the final step is to apply the configuration settings and start the server. The screen in Figure 3-11 outlines the steps to be performed and will update as each step is completed. To begin the configuration, click the “Execute” button and wait for the process to finish:

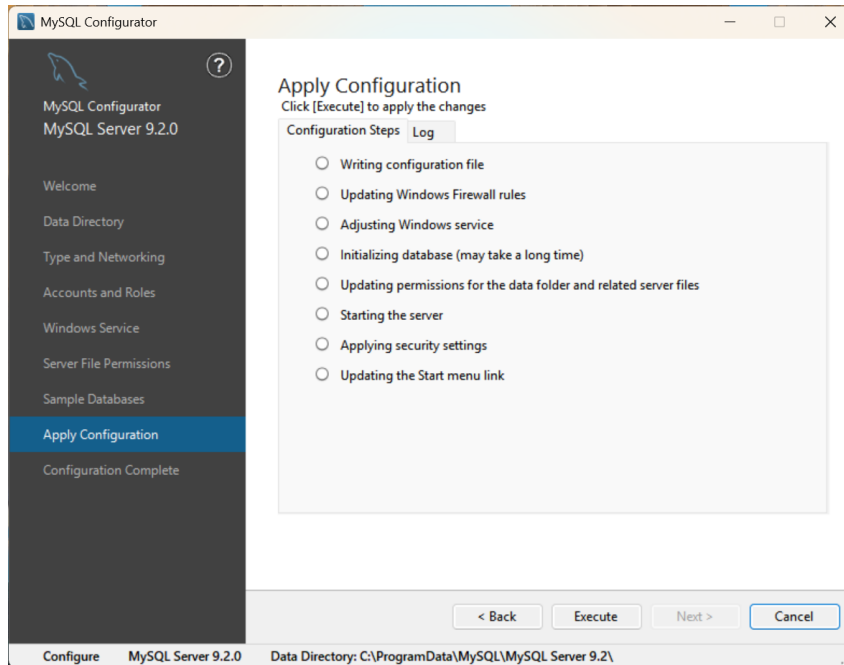


Figure 3-11

If the configuration fails, select the Log tab and review the log information for error details.

3.4 Setting the PATH environment variable

In addition to the Configurator, MySQL includes several command-line tools for managing and interacting with the MySQL server. To avoid typing a long pathname each time you want to execute a command, it helps to add the path to these tools to your PATH environment variable. Assuming that you installed MySQL into the default location, the following path will need to be added to your PATH environment variable (where *<version>* is replaced by the version of MySQL that was installed):

```
C:\Program Files\MySQL\MySQL Server <version>\bin
```

To ensure this is included in the path whenever a Command Prompt or PowerShell is opened, right-click on the Windows Start menu, select Settings from the resulting menu, and enter “Edit the system environment variables” into the “Find a setting” text field. In the System Properties dialog (Figure 3-12), click the *Environment Variables...* button:

Installing and Launching MySQL on Windows

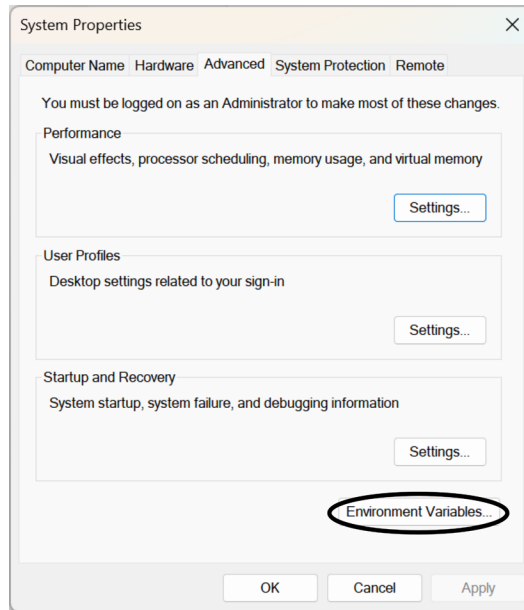


Figure 3-12

In the Environment Variables dialog, locate the Path variable in the User variables list, select it, and click the *Edit...* button:

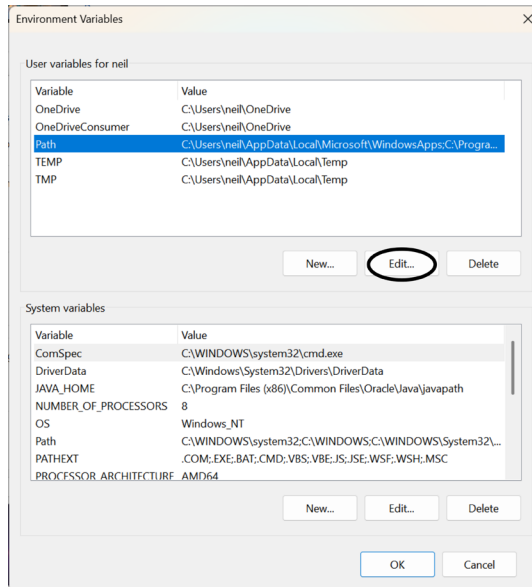


Figure 3-13

Using the *New* button in the edit dialog (Figure 3-14), add a new entry to the path. For example, assuming MySQL was installed into *C:\Program Files\MySQL\MySQL Server 9.1\bin*, the following entry would need to be added:

```
C:\Program Files\MySQL\MySQL Server 9.2\bin\
```

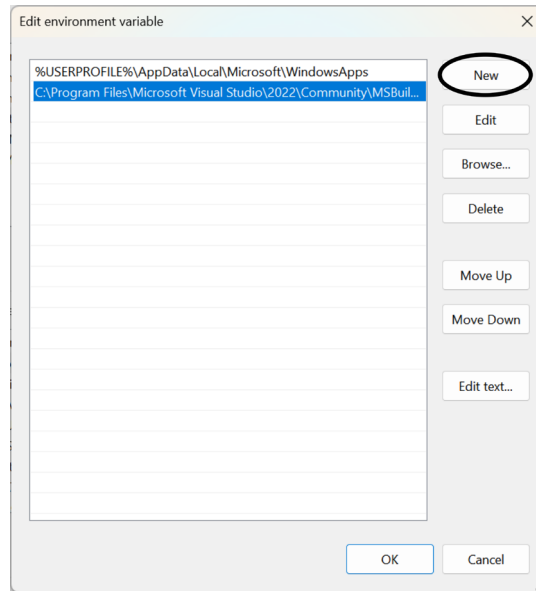


Figure 3-14

Once the new path entry has been added, click OK in each dialog box and close the system properties control panel.

Open a Command Prompt window by pressing Windows + R on the keyboard and entering *cmd* into the Run dialog. Within the Command Prompt window, enter:

```
echo %Path%
```

The returned path variable value should include the MySQL path. Verify that the path is correct by attempting to run the *mysql* client as follows:

```
mysql --version
```

If the Path is correctly configured, you should see output similar to the following:

```
mysql Ver 9.2.0 for Win64 on x86_64 (MySQL Community Server - GPL)
```

3.5 Securing MySQL Server on Windows

Once the installation is complete, an additional step is recommended to increase MySQL's security. MySQL provides a tool called *mysql_secure_installation*, which allows us to eliminate potential vulnerabilities. When executed, this command will prompt for the current root password:

```
C:\Demo> mysql_secure_installation

Securing the MySQL server deployment.

Enter password for user root:
```

The next option is to install the password validation component. This component ensures that all user passwords are strong and secure. It is recommended to install this component if you are working in a production environment; however, it is not essential if you are simply learning MySQL on a local system.

```
VALIDATE PASSWORD COMPONENT can be used to test passwords
```

Installing and Launching MySQL on Windows

```
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?
```

```
Press y|Y for Yes, any other key for No:
```

The tool will then ask if you wish to change the current root password:

```
Using existing password for root.
Change the password for root ? ((Press y|Y for Yes, any other key for No) :
```

The *mysql_secure_installation* command will prompt you to remove the anonymous user, disallow remote root access, and delete the test database. Generally, it is recommended to agree to these suggestions. However, if you are accessing your Linux server remotely, such as through a secure shell (SSH) connection, you will need to keep remote root access until you have created at least one additional user account:

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.

Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : n

By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.

Remove test database and access to it? (Press y|Y for Yes, any other key for
No) : y
  - Dropping test database...
Success.

  - Removing privileges on test database...
Success.
```

Finally, *mysql_secure_installation* will request permission to reload the privilege tables to apply the changes. Enter 'y' to complete the process of securing the MySQL installation:

```
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
```

3.6 Launching the mysql client

Now that the MySQL server is running, we need a way to interact with it. The simplest option is to use the MySQL client (*mysql*). This tool connects to the server and provides a command-line environment for executing SQL statements on the stored data.

When starting the MySQL client, you must provide a username and password to access the server. Since this is a new installation, the only configured user (unless you added more in the MySQL Configurator) is the root user, and you will use the password that you specified during the configuration process. To start the MySQL client, run the following command in a Command Prompt window and enter the root password when prompted:

```
mysql -u root -p
```

Once *mysql* has connected, the following output will appear:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 9.2.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

At the *mysql*> prompt, enter the following SQL statement:

```
mysql> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
+-----+
4 rows in set (0.02 sec)
```

To exit from *mysql*, enter *exit* or *quit* at the prompt:

```
mysql> quit
Bye
```

3.7 Knowledge test



Click the link below or scan the QR code to test that you know how to install MySQL on Windows:

<https://www.answerstopia.com/hr7u>



3.8 Reference points

The main points covered in this chapter are as follows:

- **Downloading MySQL for Windows**
 - Available as a Microsoft Software Installer (.MSI) file.
 - Download from <https://dev.mysql.com/downloads/mysql/>.
 - Choose the latest version and Windows OS, then download the MSI Installer.
- **Running the Software Installer**

Installing and Launching MySQL on Windows

- Navigate to the downloaded MSI file and double-click to start installation.
- Accept licensing terms and choose the “Typical” setup for basic functionality.
- At the end of the installation, enable “Run MySQL Configurator” and finish.
- **Configuring and Starting MySQL Server**
 - MySQL Configurator sets up the database, initializes it, and starts the server.
 - Data Directory: Default location is C:\ProgramData\MySQL Server <version>\, but can be changed.
- **Configuration Types**
 - **Development Computer:** Uses minimal memory for personal development.
 - **Server Computer:** Balances MySQL with other production services.
 - **Dedicated Computer:** Allocates maximum memory for MySQL.
 - **Manual:** Uses settings from the my.ini file.
- **Accounts and Roles**
 - Set a root password for full database control.
 - Additional users can be added.
- **Windows Service Configuration**
 - MySQL can run as a Windows service or be launched manually.
 - Running as a service allows automatic startup with Windows.
 - Default service runs under NetworkService but can be changed.
- **Applying Configuration**
 - Click “Execute” to apply settings and start the server.
 - Review logs if configuration fails.
- **Securing MySQL with mysql_secure_installation**
 - Removes security vulnerabilities by:
 - Setting password validation rules.
 - Removing anonymous users.
 - Disallowing remote root access.
 - Deleting the test database.

4. Installing and Launching MySQL on macOS

This chapter will explain how to install and run MySQL on macOS, including instructions for starting and stopping the MySQL server. Additionally, we will cover how to use the MySQL client and explore some of the configuration options available in MySQL on macOS.

4.1 Downloading MySQL for macOS

MySQL for macOS is provided as macOS disk image (.dmg) files and is available for Apple Silicon (arm64) and Intel (x86_64) Mac systems. To download the latest MySQL 9 community edition, open a browser window and navigate to the following web page:

<https://dev.mysql.com/downloads/mysql/>

Use the menu at the top of the download page to select the latest MySQL 9 version (marked A in Figure 5-1) and set the operating system (B) and version (C) menus to match your macOS version and processor architecture. Finally, click the Download button (D) next to the DMG Archive list item:

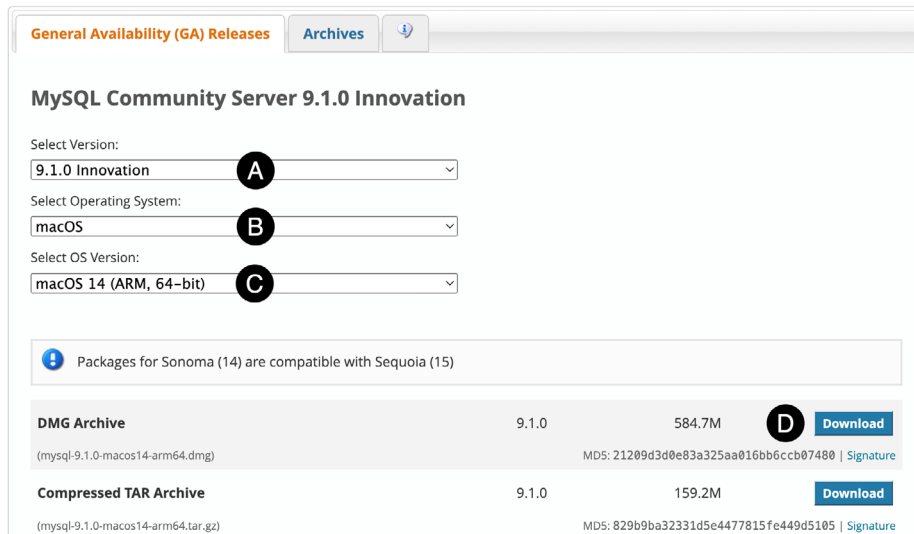


Figure 4-1

When prompted, you can either sign in with an Oracle web account or select the “No thanks, just start my download” option to proceed anonymously. Finally, save the disk image to your local filesystem.

4.2 Running the package installer

To complete the installation, locate the downloaded MySQL disk image file and double-click it to open the image in a Finder window:

Installing and Launching MySQL on macOS

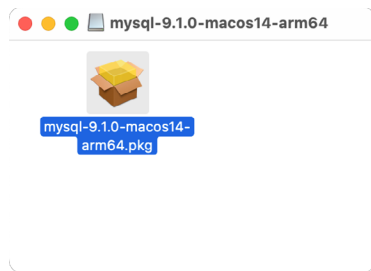


Figure 4-2

Double-click the *.pkg* file located within the disk image to launch the MySQL package installer. The installer interface should appear as shown in Figure 4-3:

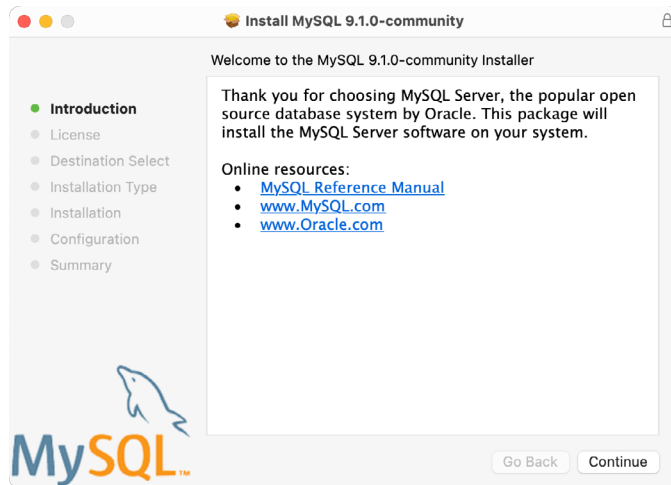


Figure 4-3

Click the Continue button on the welcome screen, review and accept the licensing terms, and on the destination selection screen, choose the option to install MySQL for all system users:

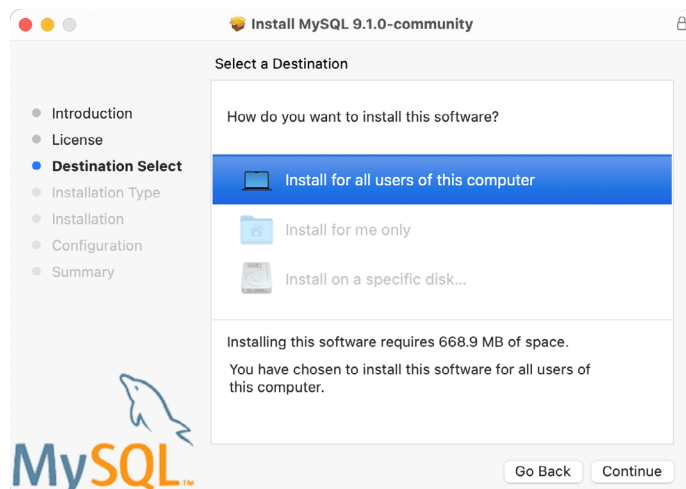


Figure 4-4

Click the Continue button to proceed to the installation type screen. By default, MySQL will be installed in the `/usr/local/mysql` directory, which we will use as the reference point for the remainder of this chapter. If you prefer to install MySQL in a different location, click the “Change Install Location...” button and choose your desired directory.

Remaining in the Installation type screen, click the Customize button, as shown by the arrow in Figure 4-5 below:

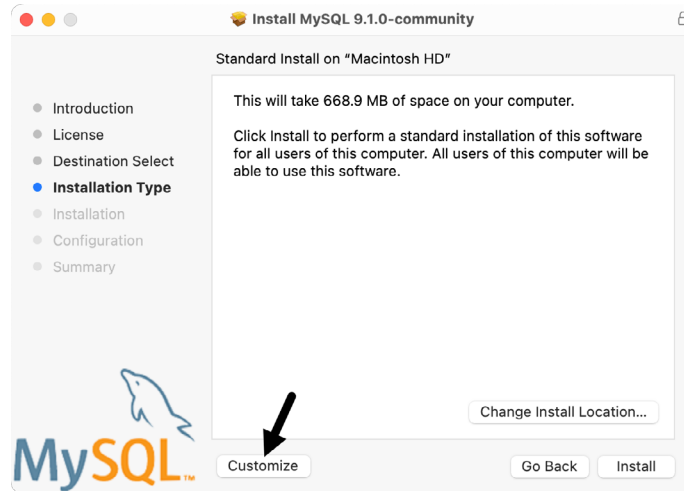


Figure 4-5

The customization screen lets you choose which MySQL packages to install on your Mac. Enable the MySQL Server, Preference Pane, and Launchd Support packages. After that, click the Go Back button to return to the installation type screen:

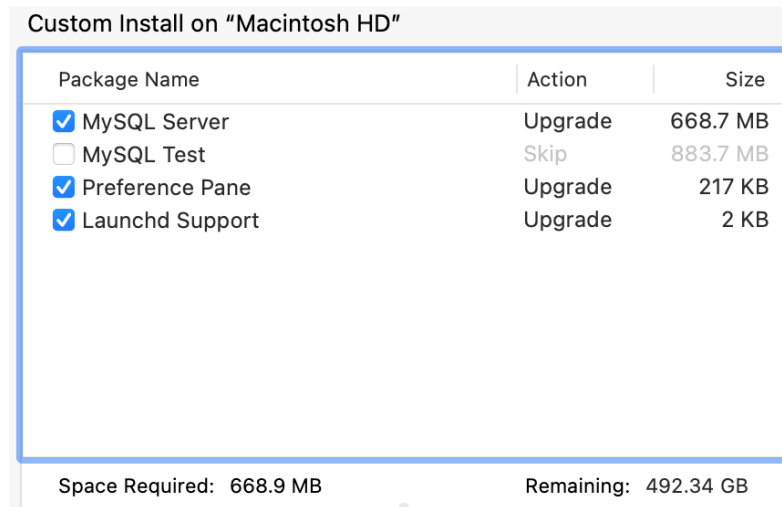


Figure 4-6

After returning to the installation type screen, click the Install button. Once the installation is complete, the package installer will prompt you to enter a password for the root account of your MySQL Server configuration. The root account acts as a “superuser,” granting you unlimited access and control over the databases you create with MySQL. Therefore, it is important to choose a sufficiently complex password.

Installing and Launching MySQL on macOS

We will explain how to start and stop the MySQL Server later, so make sure to turn off the “Start MySQL Server after installation” option before clicking Finish:

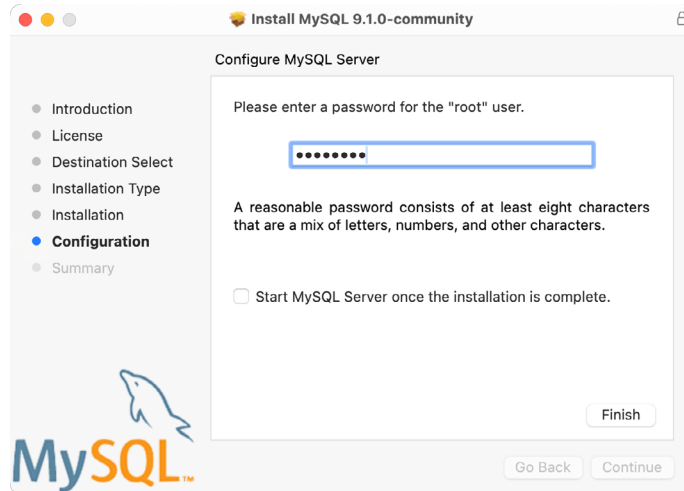


Figure 4-7

If prompted, enter your macOS user password, then wait while the installer performs some final database initialization steps. Once these steps are complete, click the Close button to exit the package installer.

4.3 Using the Preferences Pane

During the installation, we chose to install the Preference Pane. This pane is integrated into the macOS System Settings app and offers a central location for managing and configuring MySQL on your Mac. To access the panel, click on the Apple logo in the top left corner of the desktop toolbar and select “System Settings...” from the menu. In the System Settings app, you can either enter “MySQL” in the search bar or scroll down the left-hand navigation panel until you see the MySQL entry, as illustrated below:

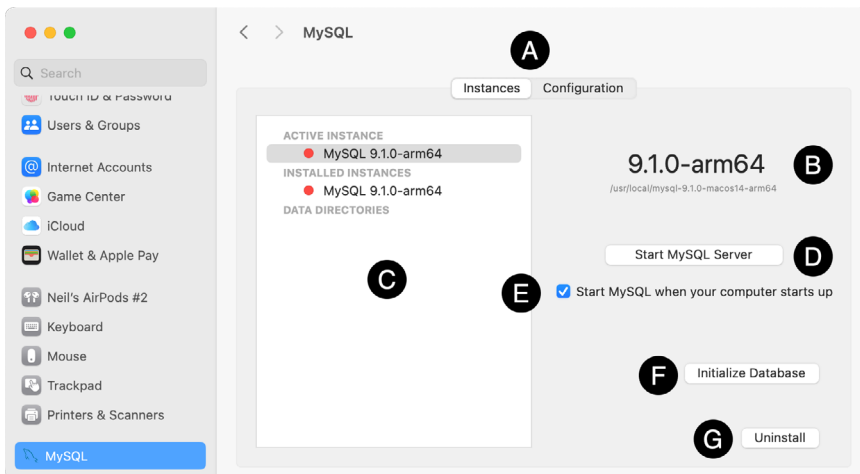


Figure 4-8

The tab bar (marked A above) switches between the Instances and Configuration panels.

4.3.1 The Instances pane

The instances panel provides information about the version of MySQL installed (B) and the currently active instances (C). It also lists any custom data directories you have configured on your system, with the default being `/usr/local/mysql/data`. The button labeled D is used to start and stop MySQL Server instances, while the option to start MySQL automatically when your system boots is indicated by E.

The Initialize Database button (F) creates a new data directory. If the data directory already exists, this action will overwrite any existing data, so it should be used with caution. The Uninstall button (G) will remove MySQL from your system, including data.

To launch the server, click the Start MySQL Server button. Note that the status indicators next to the active and installed instances will change from red to green. You can click the button again to stop the server.

4.3.2 The Configuration pane

The Configuration button in the tab bar, marked as A above, opens the panel shown in Figure 4-9. This panel allows you to modify the directories used by MySQL, including the base, data, and plugin directories. You can also change the name and location of the error log and PID files. The PID file is a text file that contains the process ID of the currently running MySQL instance, which is used to determine whether the MySQL server is running on the system.

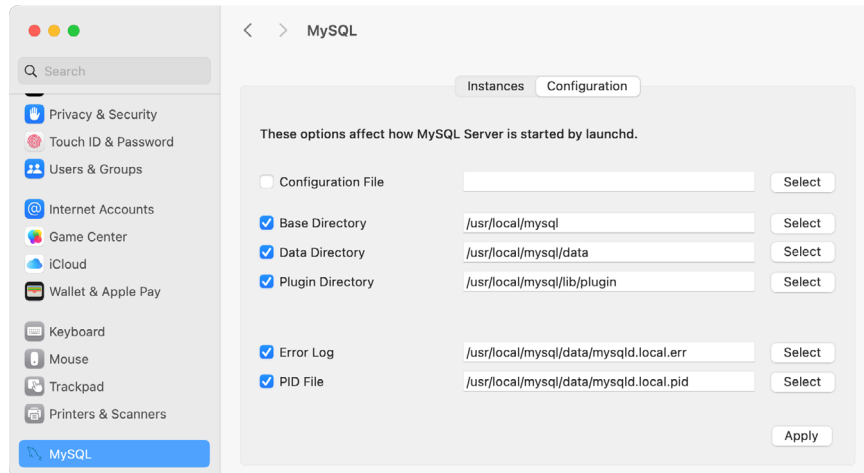


Figure 4-9

The configuration pane allows you to specify a custom path for the MySQL configuration file. This XML property list (.plist) file contains the settings for your MySQL installation and is updated in the background by the Preferences Pane as you make changes. Alternatively, you can edit this file directly instead of using the Preferences Pane. The default path for this configuration file is as follows:

```
/Library/LaunchDaemons/com.oracle.oss.mysql.mysqlld.plist
```

4.4 Adding MySQL to your PATH

In addition to the Preferences Pane, MySQL includes several command-line tools for managing and interacting with the MySQL server. To avoid typing long pathnames each time you want to execute a command, it helps to add the paths to these tools to your PATH environment variable. Assuming that you installed MySQL into the default location, the following paths will need to be added to your PATH environment variable:

```
/usr/local/mysql/bin
```

Installing and Launching MySQL on macOS

```
/usr/local/mysql/support-files
```

To ensure these are included in the path whenever a new terminal window is opened, an export statement needs to be placed in the *.zprofile* file in your home directory. To add the export statement, open a Terminal window and execute the following command, replacing */usr/local/mysql* with your custom path if you specified one during installation:

```
echo "export PATH=\$PATH:/usr/local/mysql/bin:/usr/local/mysql/support-files" >>
$HOME/.zprofile
```

After adding the path to your shell profile, open a new Terminal window and run the following command:

```
mysql --version
```

If your path is configured correctly, you should see output similar to this:

```
mysql Ver 9.2.0 for macos14 on arm64 (MySQL Community Server - GPL)
```

If the shell reports that the *mysql* command was not found, run the following command to check if the paths were added to the end of the PATH environment variable:

```
echo $PATH
```

Next, check the configuration settings in the Preferences pane and verify that the base directory path matches those added to the PATH environment variable.

4.5 Starting MySQL Server using launch control

MySQL Server can be launched from the command line in several ways, but the recommended method is to use the macOS launch control system. This approach is advantageous because it utilizes the same configuration property list file as the Preferences pane and works regardless of where MySQL is installed on the filesystem.

To start and stop the MySQL server, we use the *launchctl* command with the *load* and *unload* arguments, including the path to the property list file. The *launchctl* tool requires superuser privileges, so we must use *sudo* to execute the command.

To start MySQL Server, open a Terminal window and run the following command:

```
sudo launchctl load -F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist
```

When prompted, enter your macOS user password to grant permission to launch the server.

To stop the MySQL Server, use this command:

```
sudo launchctl unload -F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.
plist
```

4.6 Starting MySQL Server using mysql.server

Alternatively, MySQL Server can be started and stopped using the *mysql.server* script included in the MySQL *support-files* directory as follows:

```
sudo mysql.server start
sudo mysql.server stop
```

This command does not utilize the configuration settings in the property list file and requires additional steps if MySQL is installed in a location other than */usr/local/mysql*. If you installed MySQL in a custom directory, you will need to either navigate to that location before executing the command or specify the base directory path

directly on the command line. For example:

```
sudo mysql.server start --basedir=/my/custom/path/mysql
sudo mysql.server start --basedir=/my/custom/path/mysql
```

4.7 Securing MySQL Server on macOS

Once the installation is complete, an additional step is recommended to increase the security of MySQL. MySQL provides a tool called *mysql_secure_installation*, which allows us to eliminate potential vulnerabilities. When executed, this command will prompt for the current root password:

mysql_secure_installation

Securing the MySQL server deployment.

Enter password for user root:

The next option is to install the password validation component. This component ensures that all user passwords are strong and secure. It is recommended to install this component if you are working in a production environment; however, it is not essential if you are simply learning MySQL on a local system:

VALIDATE PASSWORD COMPONENT can be used to test passwords and improve security. It checks the strength of password and allows the users to set only those passwords which are secure enough. Would you like to setup VALIDATE PASSWORD component?

Press y|Y for Yes, any other key for No:

The tool will then ask if you wish to change the current root password:

Using existing password for root.

Change the password for root ? ((Press y|Y for Yes, any other key for No) :

The *mysql_secure_installation* command will prompt you to remove the anonymous user, disallow remote root access, and delete the test database. Generally, it is recommended to agree to these suggestions:

Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.

Normally, root should only be allowed to connect from 'localhost'. This ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : n

By default, MySQL comes with a database named 'test' that anyone can access. This is also intended only for testing, and should be removed before moving into a production environment.

Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y

- Dropping test database...
Success.

- Removing privileges on test database...

Installing and Launching MySQL on macOS

```
Success.
```

Finally, *mysql_secure_installation* will request permission to reload the privilege tables to apply the changes. Enter 'y' to complete the process of securing the MySQL installation:

```
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.
All done!.
```

4.8 Launching the mysql client

Now that the MySQL Server is running, we need a way to interact with it. The simplest option is to use the MySQL client (*mysql*). This tool connects to the server and provides a command-line environment for executing SQL statements on the stored data.

When starting the MySQL client, you must provide a username and password to access the server. Since this is a new installation, the only configured user is the root user, and you will use the password you specified during the installation process. To start the MySQL client, run the following command in a Terminal window and enter the root password when prompted:

```
mysql -u root -p
```

Once mysql has connected, the following output will appear:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 9.2.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

At the *mysql>* prompt, enter the following SQL statement:

```
mysql> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
+-----+
4 rows in set (0.02 sec)
```

To exit from mysql, enter *exit* or *quit* at the prompt:

```
mysql> quit
Bye
```

4.9 Knowledge test



Click the link below or scan the QR code to test that you know how to install MySQL on macOS:

<https://www.answerstopia.com/39pp>



4.10 Reference points

The main points covered in this chapter are as follows:

- **Downloading MySQL for macOS**
 - Available as a .dmg (disk image) file for Apple Silicon (arm64) and Intel (x86_64) Macs.
 - Download from <https://dev.mysql.com/downloads/mysql/>.
 - Choose MySQL 9, select your macOS version and architecture, then download the DMG image.
- **Running the Package Installer**
 - Locate and open the .dmg file in Finder.
 - Double-click the .pkg file to launch the installer.
 - Accept licensing terms and select installation for all system users.
 - By default, MySQL is installed in `/usr/local/mysql`.
 - Customize installation to include MySQL Server, Preference Pane, and Launchd Support.
 - Set a root password for MySQL access.
- **Using the Preferences Pane**
 - Installed in *System Settings* -> *MySQL*.
 - **Instances Panel**
 - Displays MySQL version and active instances.
 - Allows starting/stopping MySQL Server.
 - Option to automatically start MySQL at boot.
 - **Configuration Panel**
 - Modify directories (base, data, plugins, logs).
 - Manage MySQL configuration file (.plist).
- **Starting/Stopping MySQL Server from the command-line**
 - **Starting using launchctl (recommended):**

```
sudo launchctl load \
-F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist
```

Installing and Launching MySQL on macOS

- **Stopping using launchctl:**

```
sudo launchctl unload \  
-F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist
```

- **Using mysql.server script**

```
sudo mysql.server start  
sudo mysql.server stop
```

- **Securing MySQL with mysql_secure_installation**

- Removes security vulnerabilities by:

- Enforcing strong passwords.
- Removing anonymous users.
- Disabling remote root login.
- Deleting the test database.
- Final step reloads privilege tables.

- **Launching the MySQL Client**

- Connect to the server using:

```
mysql -u root -p
```

5. Installing and Launching MySQL on Linux

This chapter explains how to install and run MySQL on Linux, including instructions for starting and stopping the MySQL server and using the MySQL client. The steps to install MySQL will depend on the Linux distribution, of which there are many. Fortunately, most distributions support one of the two main package management systems, which will be covered in this chapter.

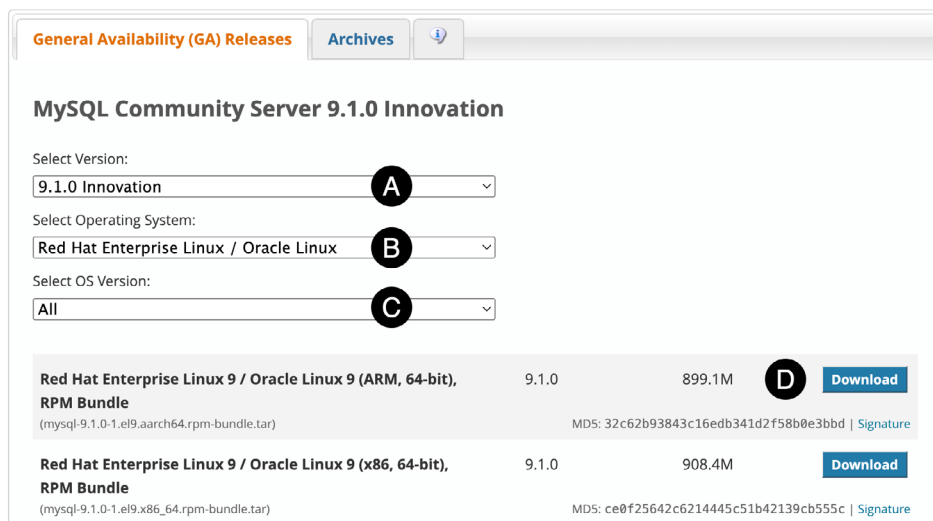
5.1 Downloading MySQL for Linux

Most Linux repositories include MySQL packages that can be downloaded and installed using package management tools such as *dnf* and *apt*. Additionally, you can configure your Linux system to download and install MySQL directly from the Oracle repositories. However, these options often do not provide access to the latest version of MySQL. For example, as of now, most Linux distributions and the Oracle repositories only offer packages for MySQL 8. To ensure that we install the latest MySQL 9 packages, we will download them directly from the MySQL website and install them manually.

MySQL for Linux is provided as a tar file containing a bundle of package files. Depending on the Linux variant, MySQL is available for Intel (x86_64), AMD (amd64), and ARM (aarch64) systems. To download the latest MySQL 9 community edition, open a browser window and navigate to the following web page:

<https://dev.mysql.com/downloads/mysql/>

Use the menu at the top of the download page to select the latest MySQL 9 version (marked A in Figure 5-1) and set the operating system (B) and version (C) menus to match your Linux distribution and processor architecture. Finally, click the Download button (D) next to the “Bundle” list item:



General Availability (GA) Releases Archives

MySQL Community Server 9.1.0 Innovation

Select Version: **A** 9.1.0 Innovation

Select Operating System: **B** Red Hat Enterprise Linux / Oracle Linux

Select OS Version: **C** All

Red Hat Enterprise Linux 9 / Oracle Linux 9 (ARM, 64-bit), RPM Bundle (mysql-9.1.0-1.el9.aarch64.rpm-bundle.tar)	9.1.0	899.1M	D Download
Red Hat Enterprise Linux 9 / Oracle Linux 9 (x86, 64-bit), RPM Bundle (mysql-9.1.0-1.el9.x86_64.rpm-bundle.tar)	9.1.0	908.4M	Download

Figure 5-1

Installing and Launching MySQL on Linux

When prompted, you can either sign in with an Oracle web account or select the “No thanks, just start my download” option to proceed anonymously. Finally, save the image to your local filesystem.

Once the download is complete, open a terminal or shell and extract the packages from the bundle tar file as follows, where *<tar-bundle-file-name>* is replaced by the name of the file you downloaded:

```
tar xvf <tar-bundle-file-name>.tar
```

5.2 Installing MySQL on RPM-based distributions

MySQL for Red Hat Linux-based systems (including RHEL, Fedora, CentOS, AlmaLinux, and Rocky Linux) is provided as a collection of Red Hat Package Manager (RPM) files named using the following convention:

```
mysql-community-<component name>-<version>.<os>.<arch>.rpm
```

For example, the RPM file for the MySQL 9.1.0-1 client for Red Hat Enterprise Linux 9 on ARM-based systems is named as follows:

```
mysql-community-client-9.1.0-1.el9.aarch64.rpm
```

From the packages contained in the bundle, we will install the following:

```
mysql-community-icu-data-files
mysql-community-common
mysql-community-client-plugins
mysql-community-libs
mysql-community-client
mysql-community-server
```

To complete the installation, change directory to the location of the extracted RPM files, run the following command, and enter your user password when prompted:

```
sudo rpm -ihv mysql-{community-icu-data-files,community-common,community-client-plugins,community-libs,community-client,community-server}-9*.rpm
```

After the installation is complete, use the *systemctl* command to start the MySQL server service as follows:

```
sudo systemctl start mysqld.service
```

To check that the service started, run the following command:

```
sudo systemctl status mysqld.service
```

If you would like MySQL Server to load each time the system boots, run the following command to change the default configuration:

```
sudo systemctl enable mysqld.service
```

To stop the server, use the following *systemctl* command:

```
sudo systemctl stop mysqld.service
```

5.3 Securing MySQL Server on RPM-based systems

Once the installation is complete, the next step is to provide a password for the root account of your MySQL Server configuration. The root account acts as a superuser, granting you unlimited access and control over the databases you create with MySQL. In most cases, MySQL may have applied a temporary password to the root account during installation. If this is the case, the password will have been logged in the *mysqld.log* file and can

be identified using the following command:

```
sudo grep "temporary password" /var/log/mysqld.log
```

If a temporary root password exists, you will see output similar to the following:

```
2024-11-19T20:02:49.552754Z 6 [Note] [MY-010454] [Server] A temporary password
is generated for root@localhost: RD17E#uvkywM
```

If no output appears, a password has not yet been applied to the root account. To apply a root password or change the temporary password provided by MySQL, use the *mysql_secure_installation* tool as follows:

mysql_secure_installation

```
Securing the MySQL server deployment.
```

```
Enter password for user root:
```

If MySQL has assigned a temporary password, enter it here and input the new password you want to replace it with. If a temporary password was not assigned, enter a password of your choice for the root account.

Once the password has been configured, *mysql_secure_installation* will ask if you want to remove the anonymous user, disallow remote root access, and delete the test database. In general, you should agree to these suggestions.

Finally, *mysql_secure_installation* will request permission to reload the privilege tables to apply the changes. Enter 'y' to finish securing the MySQL installation.

5.4 Installing MySQL on Debian and Ubuntu

The bundles for Debian and Ubuntu contain a set of Debian package (.deb) files that can be installed using the Debian package manager (*dpkg*). From the packages contained in the bundle, we will install the following:

```
mysql-common
mysql-community-server-core
mysql-community-client-plugins
mysql-community-client-core
mysql-community-client
mysql-client
mysql-community-server
mysql-server
```

The Debian package files for MySQL use the following naming convention:

```
<package-name>_<version><os>_<arch>.deb
```

For example, the package file for the MySQL 9.1.0-1 client for Ubuntu 24.10 on AMD 64-bit systems is named as follows:

```
mysql-client_9.1.0-1ubuntu24.10_amd64.deb
```

In addition to the packages mentioned above, MySQL for Ubuntu and Debian requires the installation of the *libaio1* and *libmecab2* library packages. Therefore, begin by installing these two packages using the following commands and enter your user password when prompted:

On Ubuntu:

```
sudo apt install libaio1t64 libmecab2
```

Installing and Launching MySQL on Linux

On Debian:

```
sudo apt install libaio1 libmecab2
```

To finish the installation, navigate to the directory where the extracted Debian package files are located and execute the following command (note that we are using the wildcard '*' character to avoid typing the version and platform details):

```
sudo dpkg -i mysql-{common,community-client-plugins,community-client-core,community-client,client,community-server-core,community-server,server}*_*.deb
```

Before the package installation process is complete, a dialog (Figure 5-2) will appear requesting the password for the root user account of the MySQL database:

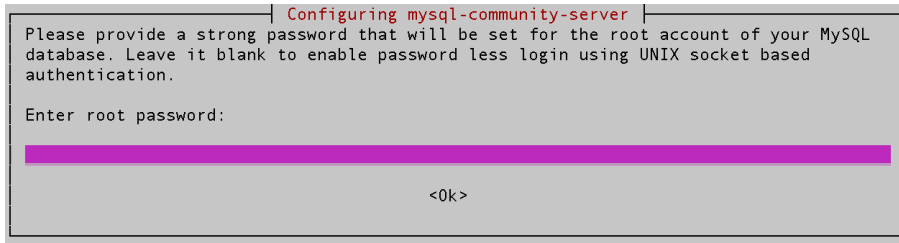


Figure 5-2

The root account serves as a super user, providing unrestricted access and control over the databases you create in MySQL. Enter a strong password, then press Enter to close the dialog box and continue with the installation.

Once the installation is complete, use the *systemctl* command to start the MySQL Server service as follows:

```
sudo systemctl start mysql.service
```

To check that the service started, run the following command:

```
sudo systemctl status mysql.service
```

If you would like MySQL Server to load each time the system boots, run the following command to change the default configuration:

```
sudo systemctl enable mysql.service
```

To stop the server, use the following *systemctl* command:

```
sudo systemctl stop mysql.service
```

5.5 Securing MySQL Server on Debian and Ubuntu

Once the installation is complete, an additional step is recommended to increase the security of MySQL. MySQL provides a tool called *mysql_secure_installation*, which allows us to eliminate potential vulnerabilities. When executed, this command will prompt for the current root password:

```
mysql_secure_installation

Securing the MySQL server deployment.

Enter password for user root:
```

The next option is to install the password validation component. This component ensures that all user passwords are strong and secure. It is recommended to install this component if you are working in a production environment; however, it is not essential if you are simply learning MySQL on a local system.

```
VALIDATE PASSWORD COMPONENT can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?
```

```
Press y|Y for Yes, any other key for No:
```

The tool will then ask if you wish to change the current root password:

```
Using existing password for root.
```

```
Change the password for root ? ((Press y|Y for Yes, any other key for No) :
```

The *mysql_secure_installation* command will prompt you to remove the anonymous user, disallow remote root access, and delete the test database. Generally, it is recommended to agree to these suggestions:

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.
```

```
Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.
```

```
Disallow root login remotely? (Press y|Y for Yes, any other key for No) : n
```

```
By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.
```

```
Remove test database and access to it? (Press y|Y for Yes, any other key for
No) : y
- Dropping test database...
Success.
```

```
- Removing privileges on test database...
Success.
```

Finally, *mysql_secure_installation* will request permission to reload the privilege tables to apply the changes. Enter 'y' to complete the process of securing the MySQL installation:

```
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.
All done!
```

5.6 Launching the mysql client

Now that the MySQL server is secured and running, we need a way to interact with it. The simplest option is to use the MySQL client (*mysql*). This tool connects to the server and provides a command-line environment for executing SQL statements on the stored data.

When starting the MySQL client, you must provide a username and password to access the server. Since this is a new installation, the only configured user is the root user, and you will use the password that you specified in

Installing and Launching MySQL on Linux

the previous section. To start the MySQL client, run the following command, and enter the root password when prompted:

```
mysql -u root -p
```

Once mysql has connected, the following output will appear:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 9.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

At the *mysql>* prompt, test the installation by entering the following SQL statement to generate a list of databases:

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql      |
| performance_schema |
| sys        |
+-----+
4 rows in set (0.02 sec)
```

To exit from mysql, enter *exit* or *quit* at the prompt:

```
mysql> quit
Bye
```

5.7 Knowledge test



Click the link below or scan the QR code to test that you know how to install MySQL on Linux:

<https://www.answertopia.com/zj6i>



5.8 Reference points

The main points covered in this chapter are as follows:

• Downloading MySQL for Linux

- Available as a tar bundle for Intel (x86_64), AMD (amd64), and ARM (aarch64) systems.
- Download from MySQL official site: <https://dev.mysql.com/downloads/mysql/>

- Extract the tar bundle using

```
tar xvf <tar-bundle-file-name>.tar
```

- **Installing MySQL on RPM-Based Distributions**

- Installation using RPM:

```
sudo rpm -ihv mysql-{community-icu-data-files,community-common, \
community-client-plugins,community-libs, \
community-client,community-server}-9*.rpm
```

- **Securing MySQL on RPM-Based Systems**

- Retrieve temporary root password:

```
sudo grep "temporary password" /var/log/mysqld.log
```

- Run *mysql_secure_installation* to configure security settings.

- **Managing MySQL Service on RHEL-based Distributions**

- Start MySQL:

```
sudo systemctl start mysqld.service
```

- Check MySQL status:

```
sudo systemctl status mysqld.service
```

- Enable MySQL on startup:

```
sudo systemctl enable mysqld.service
```

- Stop MySQL:

```
sudo systemctl stop mysqld.service
```

- **Installing MySQL on Debian and Ubuntu**

- Install required dependencies:

```
sudo apt install libaio1t64 libmecab2 # Ubuntu
sudo apt install libaio1 libmecab2 # Debian
```

- Install MySQL packages:

```
sudo dpkg -i mysql-{common,community-client-plugins, \
community-client-core,community-client,client, \
community-server-core,community-server,server}_*.deb
```

- **Managing MySQL Service on Debian and Ubuntu**

- Start MySQL:

```
sudo systemctl start mysql.service
```

- Check MySQL status:

```
sudo systemctl status mysql.service
```

Installing and Launching MySQL on Linux

- Enable MySQL on startup:

```
sudo systemctl enable mysql.service
```

- Stop MySQL:

```
sudo systemctl stop mysql.service
```

- **Securing MySQL on Debian and Ubuntu**

- Run *mysql_secure_installation* to configure security settings.

- **Launching and Using the MySQL Client:**

- Connect to MySQL Server:

```
mysql -u root -p
```

6. The MySQL Client

In the chapter titled “*The Basics of Databases*” we learned that MySQL is a client-server-based database management system (DBMS). Along with the MySQL server, MySQL provides an interactive client that allows administrators to connect to server instances and execute SQL statements. In this chapter, we will explore the features of the MySQL client.

6.1 The `mysql` command-line tool

The MySQL client is a command-line tool installed with the MySQL package bundle. It can also be installed separately on non-server systems. From the MySQL shell prompt, you can execute various commands on both local and remote database servers. This includes tasks such as creating and deleting databases and tables, searching for data, adding new rows, and much more. There are alternative client tools for interacting with MySQL Server, including MySQL Workbench and phpMyAdmin, which will be discussed in later chapters.

Assuming MySQL has been installed, the `mysql` tool may be loaded at the operating system command prompt as follows:

```
mysql -u <username> -p
```

The `-p` command-line flag indicates that the account belonging to `<username>` is password protected, prompting the client to request the password before granting access to the server:

```
mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 9.1.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Another option is to specify the password on the command line, but this method is less secure because the password appears in plain text:

```
mysql -u root --password='mypassword'
```

If the `mysql` client is running remotely from the MySQL server, the `-h` flag is used to specify the server host, either using the hostname or IP address:

```
mysql -h <remote server> -u <username> -p
```

By default, access to MySQL servers from remote clients is blocked for all user accounts, including the root

The MySQL Client

account. Appropriate privileges must be granted to enable remote access for specific user accounts. This topic will be discussed in the “MySQL Users, Privileges, and Security” chapter.

In the standard configuration, the MySQL client and server communicate over TCP port 3306. If you need to connect to a server that is configured to use a different port, you should use the `-P` flag as follows:

```
mysql -h <remote server> -P <port number> -u <username> -p
```

For example:

```
mysql -h MyDBServer -P 3306 -u john -p
```

You can obtain a list of command-line options by running `mysql --help` at the command prompt. Once MySQL is running, you can enter commands at the `mysql>` prompt. Typing ‘help’ at this prompt will display a list of commands supported by the MySQL client.

6.2 Using the mysql client

Once the client is running, you will see the `mysql>` prompt where you can enter commands and statements. SQL statements typed at this prompt are executed interactively, and the results are shown in real-time. For example, the following statement will display a list of databases managed by the server:

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql      |
| performance_schema |
| sys        |
+-----+
4 rows in set (0.05 sec)
```

SQL statements must end with a semicolon (;) or \G. If a command is not terminated, pressing Enter continues the current statement on the next line:

```
mysql> SELECT *
-> FROM myDB.customers
-> WHERE account_status='active';
```

The output from SQL statements ending with a semicolon is formatted horizontally. Long output lines will wrap to the next line, making the content difficult to read. Consider the following output as an example:

```
mysql> SELECT * FROM mysql.user;
+-----+-----+-----+-----+-----+-----+-----+
| Host | User | Select_priv | Insert_priv | Update_priv | Delete_priv | Create_priv | Drop_priv | Reload_priv |
+-----+-----+-----+-----+-----+-----+-----+
|      |      |             |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |
+-----+-----+-----+-----+-----+-----+-----+
| Host | User | Select_priv | Insert_priv | Update_priv | Delete_priv | Create_priv | Drop_priv | Reload_priv |
+-----+-----+-----+-----+-----+-----+-----+
|      |      |             |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |             |
|      |      |             |             |             |             |             |             |             |
+-----+-----+-----+-----+-----+-----+-----+
```

If we replace the semicolon with \G, the output is formatted vertically, making it easier to read:

```
mysql> SELECT * FROM mysql.user\G
***** 1. row *****
      Host: %
      User: mrlocal
      Select_priv: N
      Insert_priv: N
      Update_priv: N
      Delete_priv: N
      Create_priv: N
      Drop_priv: N
      Reload_priv: N
      Shutdown_priv: N
      Process_priv: N
      File_priv: N
      Grant_priv: N
      References_priv: N
      Index_priv: N
      Alter_priv: N
.
.
```

6.3 Typing and editing SQL statements

In the previous examples, we have entered SQL statements in uppercase. However, these statements are not case-sensitive when used in the MySQL client. The following statements, for example, are interpreted identically:

```
SELECT * FROM MYSQL.USER;
select * from mysql.user;
Select * From Mysql.User;
```

Quoted parameters are case-sensitive. For example, if an account name was created with a capital letter (e.g., Ashley), it must be referenced with the same capitalization in SQL statements:

```
SELECT * FROM mysql.user WHERE user='Ashley';
```

The MySQL client offers command-line editing and history features. Figure 6-1 outlines several keyboard editing options, with support varying by platform:

Key Sequence	Function
Up Arrow	Navigate up through the command history.
Down Arrow	Navigate down through the command history.
Left Arrow	Move left one character in the current line.
Right Arrow	Move right one character in the current line.
Backspace	Delete the character to the left of the cursor position.
Delete	Delete the character to the right of the cursor position.
Ctrl-A	Move to the start of the current line.
Ctrl-U	Delete entire line.
Ctrl-W	Delete the word before the cursor position.

Figure 6-1

6.4 Exiting from the mysql client

To exit from the MySQL client, type *quit* or *exit* at the `mysql>` command-prompt:

```
mysql> exit
Bye
```

6.5 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of the MySQL client:

<https://www.answertopia.com/2p4e>



6.6 Reference points

The main points covered in this chapter are as follows:

• Overview

- The MySQL client is a command-line tool for managing MySQL servers and executing SQL statements.
- Connects to local and remote MySQL Server instances.
- Alternative tools include MySQL Workbench and phpMyAdmin.

• Connecting to MySQL

- Use: `mysql -u <username> -p` (prompted for password).
- For remote servers: `mysql -h <hostname> -u <username> -p`
- Default communication port is 3306; specify a different port with `-P <port>`.

• Executing Commands

- Enter commands interactively at the `mysql>` prompt.
- Commands must end with a semicolon (;) or \G for vertical output formatting.
- Example: `SHOW DATABASES;` or `SELECT * FROM table\G`

• SQL Statements

- SQL is case-insensitive for commands but case-sensitive for quoted parameters.
- Example: 'User' and 'user' are treated differently.

• Editing Commands

- Supports command-line history and editing.
- Use up/down arrow keys to navigate through history.
- Ctrl-A moves to the start of a line, Ctrl-U deletes the current line.

• Viewing Help

- Use `help` or `\h` in the client to view available commands.
- Command-line options can be listed with `mysql --help`.
- **Exiting the Client**
 - Exit by typing `quit` or `exit` at the `mysql>` prompt.

7. MySQL Users, Privileges, and Security

Security is crucial in any IT environment, particularly concerning databases. When hackers gain access to a system, their primary objective is often to steal sensitive data that can be exploited for financial gain. This sensitive information may include customers' credit card details, personally identifiable information (PII), or intellectual property. Therefore, taking steps to secure your MySQL databases is essential.

Typically, a database server is protected by multiple layers of defense, including firewalls, encrypted access, and intrusion detection systems. However, frequent news reports about data breaches remind us that these security measures can be circumvented by external threats or employees within the organization.

In the event of a system breach or data exploitation, the database management system acts as the final line of defense against data theft. This chapter provides an overview of securing user access to MySQL databases and strengthening this critical layer of security.

7.1 MySQL security

MySQL security is focused on limiting access to databases and controlling users' actions once they have access. This involves careful consideration of various factors, such as determining which users are allowed to read from or write to specific database tables, and identifying those who have the authority to delete tables or use other MySQL features.

7.2 Getting information about users

The first step in securing a database is to identify which users already have access. This information is stored in a MySQL database called *mysql*. Within this database is a table named *user*, which includes several columns such as the user login name, user privileges, password settings, and connection rights.

To obtain a list of user accounts, you must log into the MySQL client as the root user. You can do this by entering the following command and then providing the root password when prompted:

```
mysql -u root -p
```

Once logged in, use the following SELECT statement to list the user records from the *user* table:

```
SELECT user FROM mysql.user;
+-----+
| user                |
+-----+
| mysql.infoschema    |
| mysql.session       |
| mysql.sys           |
| root                |
+-----+
4 rows in set (0.00 sec)
```

Unless users were added during the installation, a newly installed MySQL database will typically display only

one user, the root user, along with three reserved user accounts used internally by MySQL.

```
+-----+
| user          |
+-----+
| mysql.infoschema |
| mysql.session  |
| mysql.sys      |
| root           |
+-----+
4 rows in set (0.00 sec)
```

In a more established database, you are likely to find additional users. Some of these users will have been created manually, while others may have been generated automatically as a result of installing third-party software that uses MySQL, for example:

```
SELECT user FROM mysql.user;
+-----+
| user          |
+-----+
| mysql.infoschema |
| mysql.session  |
| mysql.sys      |
| root           |
| wikiuser       |
| wordpress      |
| wordpressuser  |
+-----+
7 rows in set (0.00 sec)
```

7.3 Adding a MySQL user account

When setting up a new user account, you must provide a user login name. While a password is optional, it is not advisable to create an account without one.

The syntax for creating a user account is as follows:

```
CREATE USER 'username'@'hostname' IDENTIFIED BY 'password';
```

For example, to create a new account for a user named “john” with password protection, you would use the following statement:

```
CREATE USER 'john'@'localhost' IDENTIFIED BY 'yrthujoi';
```

An interesting point to note is that when you press the up arrow to navigate through the command history, the CREATE USER command does not appear. This happens because the command contained the user’s password in plain text, and for security reasons, it is not stored in the command history.

You can verify that the new user has been added by querying the user table:

```
SELECT host, user FROM mysql.user WHERE user='john';
+-----+-----+
| host      | user    |
+-----+-----+
| localhost | john    |
+-----+-----+
```

```
1 row in set (0.00 sec)
```

You may have noticed that we specified John could only connect from localhost, meaning he can only connect from the same system on which the MySQL server is running. If John attempts to connect to the MySQL server from a remote client, the connection will fail. To create an account that can connect from a specific host, simply replace localhost with the desired hostname or IP address in the example above:

```
CREATE USER 'john'@'remoteserver' IDENTIFIED BY 'yrthujoi';
CREATE USER 'john'@'192.168.87.23' IDENTIFIED BY 'yrthujoi';
```

To enable a user to connect to the MySQL server from any remote host, use the '%' wildcard character instead of a specific host name:

```
CREATE USER 'john'@'%' IDENTIFIED BY 'yrthujoi';
```

If a host is not specified when creating a user, MySQL grants access from all remote hosts. To modify the host for an existing user, you can use the following UPDATE syntax:

```
UPDATE mysql.user SET Host='newhost' WHERE Host='oldhost' AND User='username';
FLUSH PRIVILEGES;
```

For example, this statement changes the host for the user “john” from “localhost” to allow access from any host:

```
UPDATE mysql.user SET Host='%' WHERE Host='localhost' AND User='john';
FLUSH PRIVILEGES;
```

7.4 Deleting a MySQL user

An existing user account can be deleted using the DROP USER statement, which has the following syntax:

```
DROP USER 'username'@'hostname';
```

For example:

```
DROP USER 'john'@'%';
```

7.5 Renaming a MySQL user

The account name of a MySQL user can be changed using the RENAME USER statement, using the following syntax:

```
RENAME USER 'username'@'hostname' TO 'new_user_name'@'hostname';
```

For example:

```
RENAME USER 'john'@'localhost' TO 'johnrown'@'localhost';
```

7.6 Changing the password for a MySQL user

To change the password for a user account, you can use the SET PASSWORD statement. If you want to change your own password, use the following syntax:

```
SET PASSWORD = 'newpassword';
```

To change the password for another user, use the ALTER USER statement. For example, to change the password for the user account john on localhost, you would use:

```
ALTER USER john IDENTIFIED BY 'newpassword';
```

7.7 Granting privileges

When a new user is created in MySQL, they can log into the server, but by default, they have no privileges to perform any actions once connected. Therefore, the next step after creating a user account is to grant privileges to that account using the GRANT statement.

Before modifying a user's privileges, check what privileges are currently assigned using the SHOW GRANTS statement and the user's account name. For example:

```
SHOW GRANTS FOR 'john'@'localhost';
+-----+
| Grants for john@localhost |
+-----+
| GRANT USAGE ON *.* TO 'john'@'localhost' |
+-----+
1 row in set (0.01 sec)
```

The statement “USAGE ON *.*” indicates that a user has no privileges on any database or table. In simple terms, this means the user cannot perform any actions after logging into the database server.

To grant a privilege, such as the ability to query any table in a database named MySampleDB, you would issue the following command:

```
GRANT SELECT on MySampleDB.* TO 'john'@'localhost';
```

Once this command is executed, the user john will be able to perform SELECT statements on any table within the MySampleDB database.

If you want to enable john to insert rows into a specific table called *product* in the MySampleDB database, you can use the following command:

```
GRANT INSERT on MySampleDB.product TO 'john'@'localhost';
```

It is also possible to specify multiple privileges in a single GRANT statement. For example:

```
GRANT INSERT, UPDATE on MySampleDB.product TO 'john'@'localhost';
```

7.8 Privileges and grantable levels

MySQL supports a wide range of privileges that apply to one or more categories or *grantable levels*. These levels define the extent to which the privilege can be applied. The following privilege levels are available in MySQL:

- **Global** - Global privileges apply to all databases on a server and are typically granted to administrator accounts.
- **Database** - Database-level privileges are specific to individual databases on the server. For instance, you might grant a user permission to insert records into all tables within a particular database while denying access to other databases on the server.
- **Table** - Table-level privileges apply to specific tables within a database. For example, if a database contains two tables, you could use a table-level privilege to allow a user access to only one.
- **Column** - Column privileges are specific to certain columns in a table. For instance, you might allow a user to modify the description column in a product table, but restrict access to the price column.
- **Routine** - In MySQL, a routine refers to a sequence of SQL statements stored on a server that can be executed without re-entering the individual statements. Routine-level privileges control which users have access to specific routines.

- **Proxy User** - MySQL allows users to act as proxies for one another, which enables a user to perform tasks using another user's identity and privileges. The proxy user privilege determines whether one user can act as a proxy for another.

The privileges supported by MySQL are listed in Figure 7-1. It's important to note that many privileges are available at multiple grantable levels:

Privilege	Meaning and Grantable Levels	Grantable Levels
ALL [PRIVILEGES]	Grant all privileges at specified access level except GRANT OPTION and PROXY.	global, database, table, column, routine
ALTER	Allow use of ALTER TABLE.	global, database, table
ALTER ROUTINE	Allow stored routines to be altered or dropped.	global, database, routine
CREATE	Allow database and table creation.	global, database, table
CREATE ROLE	Allow role creation.	global
CREATE ROUTINE	Allow stored routine creation..	global, database
CREATE TABLESPACE	Allow tablespaces and log file groups to be created, altered, or dropped.	global
CREATE TEMPORARY TABLES	Allow use of CREATE TEMPORARY TABLE.	global, database
CREATE USER	Allow use of CREATE USER, DROP USER, RENAME USER, and REVOKE ALL PRIVILEGES.	global
CREATE VIEW	Allow views to be created or altered.	global, database, table
DELETE	Allow use of DELETE.	global, database, table
DROP	Allow databases, tables, and views to be dropped.	global, database, table
DROP ROLE	Allow roles to be dropped.	global
EVENT	Allow use of events for the Event Scheduler.	global, database
EXECUTE	Allow the user to execute stored routines.	global, database, routine
FILE	Allow the user to cause the server to read or write files.	global
FLUSH_PRIVILEGES	Allow the user to issue FLUSH PRIVILEGES statements.	global
GRANT OPTION	Allow privileges to be granted to or removed from other accounts. (global, database, table, routine, proxy)	global, database, table, routine, proxy
INDEX	Allow indexes to be created or dropped.	global, database, table
INSERT	Allow use of INSERT.	global, database, table, column
LOCK TABLES	Allow use of LOCK TABLES on tables for which you have the SELECT privilege.	global, database
PROCESS	Allow the user to see all processes with SHOW PROCESSLIST.	global

PROXY	Allow user proxies.	global
REFERENCES	Allow foreign key creation.	global, database, table, column
RELOAD	Allow use of FLUSH operations.	global
REPLICATION CLIENT	Allow the user to ask where source or replica servers are.	global
REPLICATION SLAVE	Allow replicas to read binary log events from the source.	global
SELECT	Allow use of SELECT.	global, database, table, column
SHOW DATABASES	Allow SHOW DATABASES to show all databases.	global
SHOW VIEW	Allow use of SHOW CREATE VIEW.	global, database, table
SHUTDOWN	Allow use of mysqladmin shutdown.	global
SUPER	Allow use of other administrative operations such as CHANGE REPLICATION SOURCE TO, KILL, PURGE BINARY LOGS, SET GLOBAL, and mysqladmin debug command.	global
TRIGGER	Allow trigger operations.	global, database, table
UPDATE	Allow use of UPDATE.	global, database, table, column
USAGE	No privileges	global, database, table, column, routine

Figure 7-1

7.9 Privilege examples

Now that we have covered the MySQL privileges, the table below lists some examples of SQL GRANT statements:

Description	SQL Statement
Grant global INSERT privileges on all databases and tables.	GRANT INSERT ON *.* TO 'john'@'localhost';
Grant global ALL privileges on all databases and tables.	GRANT ALL ON *.* TO 'john'@'localhost';
Grant database level INSERT privileges on all tables of a specific database.	GRANT INSERT ON MySampleDB.* TO 'john'@'localhost';
Grant table level DELETE privileges on a specific database table.	GRANT DELETE ON MySampleDB.product TO 'john'@'localhost';
Grant column level UPDATE privileges on a specific column in a database table.	GRANT UPDATE (price) ON MySampleDB.product TO 'john'@'localhost';

Grant SELECT privilege on the <i>productname</i> and <i>description</i> columns and UPDATE privilege on the <i>price</i> column.	GRANT SELECT (productname, description), UPDATE (price) ON MySampleDB.product TO 'john'@'localhost';
Allow a user to CREATE stored routines for a specific database.	GRANT CREATE ROUTINE ON MySampleDB.* TO 'john'@'localhost';
Allowing a user to EXECUTE a specific routine.	GRANT EXECUTE ON ON MySampleDB.listsales TO 'john'@'localhost';
Allowing a user to act as a proxy for another.	GRANT PROXY ON 'john'@'localhost' TO 'janeT'@'otherhost';

Figure 7-2

7.10 Removing remote root access

During the installation of MySQL, we used the *mysql_secure_installation* script, which provided the option to disable remote access to the root account. Considering the extensive privileges associated with the root user, it is important to consider this decision carefully. If you haven't disabled remote root access yet, you can do so by either re-running *mysql_secure_installation* or by modifying the settings in the MySQL user database with the following command:

```
DELETE FROM mysql.user WHERE User='root' AND Host NOT IN ('localhost',
'127.0.0.1', '::1');
FLUSH PRIVILEGES;
```

7.11 Managing password verification

MySQL offers an optional component that enforces the creation of strong passwords for user accounts. You may remember that the *mysql_secure_installation* tool prompted you to install the Validate Password component during the initial setup of your MySQL environment. When this component is installed, MySQL will reject passwords that do not meet specific complexity requirements. You will receive the following error message if your password fails to meet these requirements:

```
ERROR 1819 (HY000): Your password does not satisfy the current policy
requirements
```

To install the Validate Password component, you can use the following statement:

```
INSTALL COMPONENT 'file://component_validate_password';
```

If you decide to uninstall the component later, you can do so with this command:

```
UNINSTALL COMPONENT 'file://component_validate_password';
```

The Validate Password component follows a defined set of rules that pertain to various characteristics, such as password length, combinations of mixed-case letters, and requirements for special characters. Additionally, options are available to compare passwords against a dictionary, exclude the username from the password, and require a specified number of character changes between the new and old passwords.

You can use the following statement to list the current password requirements:

```
SHOW VARIABLES LIKE 'validate_password%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
```

```
| validate_password.changed_characters_percentage | 0      |
| validate_password.check_user_name              | ON     |
| validate_password.dictionary_file              |        |
| validate_password.length                      | 8      |
| validate_password.mixed_case_count            | 1      |
| validate_password.number_count                | 1      |
| validate_password.policy                      | MEDIUM |
| validate_password.special_char_count          | 1      |
+-----+-----+
8 rows in set (0.00 sec)
```

The *validate_password.policy* variable determines which password rules are enforced, as described below:

- **LOW (0)** - Password length only.
- **MEDIUM (1)** - Password length, mixed case, number, and special characters.
- **STRONG (2)** - Password length, mixed case, numeric and special characters, and dictionary comparison.

The values assigned to these variables can be updated using SET GLOBAL. For example:

```
SET GLOBAL validate_password.length = 8;
SET GLOBAL validate_password.policy = LOW;
SET GLOBAL validate_password.mixed_case_count= 3;
```

7.12 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL user management:

<https://www.answertopia.com/5mv1>



7.13 Reference points

The main points covered in this chapter are as follows:

- **Security Importance**
 - Databases are prime hacker targets for sensitive data.
 - Layers of defense can be bypassed; database security is crucial.
- **User Management**
 - Query users: `SELECT user FROM mysql.user;`
 - Create users: `CREATE USER 'username'@'hostname' IDENTIFIED BY 'password';`
 - Modify host access: `UPDATE mysql.user SET Host='newhost' WHERE User='username';`
 - Delete users: `DROP USER 'username'@'hostname';`
 - Rename users: `RENAME USER 'username'@'hostname' TO 'newname'@'hostname';`
- **Privileges**
 - Assign privileges: `GRANT [privileges] ON [scope] TO 'user'@'host';`

- Levels: Global, Database, Table, Column, Routine, Proxy.
- Revoke privileges: `REVOKE [privileges] ON [scope] FROM 'user'@'host';`
- **Password Security**
 - Change passwords: `SET PASSWORD FOR 'user'@'host' = PASSWORD('newpassword');`
 - Enforce strong passwords with the Validate Password component.
- **Best Practices**
 - Restrict root access to local connections.
 - Regularly audit users and privileges.
 - Remove remote root access.

8. Creating Databases and Tables

Before you can do much with MySQL, you must create a database containing at least one table. This chapter will explain how to create new databases and tables using SQL statements in the MySQL client. We will also explore some of the storage engines that MySQL supports.

This chapter assumes that the `mysql` client is running and connected to the MySQL database server. If this is not the case, and you are unsure how to achieve this, refer to the “*The MySQL Client*” chapter.

8.1 Creating a new MySQL database

A new database can be created using the `CREATE DATABASE` statement, followed by the desired name for the database. Alternatively, the `CREATE SCHEMA` statement can also be used for this purpose.

Begin by entering the following statement at the `mysql>` prompt to create a new database named `sampledb`:

```
CREATE DATABASE sampledb;
```

If successful, the command will generate output similar to the following:

```
Query OK, 1 row affected (0.00 sec)
```

If a specified database name conflicts with an existing one, MySQL will display an error message reporting the issue:

```
ERROR 1007 (HY000): Can't create database 'sampledb'; database exists
```

In this case, a different database name must be chosen, or the `IF NOT EXISTS` option should be utilized. This option will only create the database if it does not already exist:

```
CREATE DATABASE IF NOT EXISTS sampledb;  
Query OK, 1 row affected, 1 warning (0.04 sec)
```

In the example above, MySQL indicates that the statement generated a warning. To view the warning description, execute the `SHOW WARNINGS` statement:

```
SHOW WARNINGS;  
+-----+-----+-----+  
| Level | Code | Message                                     |  
+-----+-----+-----+  
| Note  | 1007 | Can't create database 'sampledb'; database exists |  
+-----+-----+-----+  
1 row in set (0.00 sec)
```

To view a list of databases on the server, use the following command:

```
SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| sampledb |  
| information_schema |
```

```
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.02 sec)
```

8.2 Enabling table encryption

The default behavior of MySQL is not to encrypt database tables unless explicitly instructed to do so. This behavior is controlled by the `default_table_encryption` global variable. You can check the current setting as follows:

```
SHOW GLOBAL VARIABLES LIKE 'default_table_encryption';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| default_table_encryption | OFF |
+-----+-----+
1 row in set (0.01 sec)
```

You can opt to encrypt all tables in the database by using the `ENCRYPTION` option during the creation of the database:

```
CREATE DATABASE sampledbs ENCRYPTION = 'Y';
```

Alternatively, use this statement to enable encryption by default in MySQL:

```
SET GLOBAL default_table_encryption=ON;
```

8.3 Creating tables with SQL

New tables are added to an existing database using the `CREATE TABLE` statement. The `CREATE TABLE` statement is followed by the name of the table to be created, followed by a comma-separated list of the names and definitions of each table column:

```
CREATE TABLE table_name ( column_name definitions, table_name definitions ...,
PRIMARY KEY=(column_name));
```

The definitions for each column specify important information, such as the data type, whether the column can store `NULL` values, and if the column will automatically increment every time a new row is added to the table. The `CREATE TABLE` statement also allows you to designate a column (or a group of columns) as the primary key. For more information on primary keys, refer to the chapter “*The Basics of Databases*”.

Before creating a table, you must select a database so MySQL knows where to create it. This is done using the `USE` command (note that `USE` is a command rather than a SQL statement; therefore, it does not need to be terminated with a semicolon).

```
USE sampledbs
```

After selecting a database, the following example creates a table with three columns: `customer_id`, `customer_name`, and `customer_address`. The `customer_id` and `customer_name` columns must have values (i.e., they cannot be `NULL`). The `customer_id` column will hold an integer that automatically increments as new rows are added, while the `customer_name` and `customer_address` columns will contain character strings limited to a maximum length of 20 characters. The primary key for the table is defined as the `customer_id` column:

```
CREATE TABLE customer
```

```
(
  customer_id int NOT NULL AUTO_INCREMENT,
  customer_name char(20) NOT NULL,
  customer_address char(20) NULL,
  PRIMARY KEY (customer_id)
);
```

8.4 Understanding NULL and NOT NULL values

When a column is defined as NULL, it means that a row can be added to a database without a value assigned to that column. On the other hand, if a column is defined as NOT NULL, it must have a value assigned to it before the row can be added to the table.

8.5 Primary keys

A primary key is a column in a table that uniquely identifies individual records. The values in a primary key column must be unique within that table. If a primary key consists of multiple columns, the combination of values across those columns must also be unique for each row.

The primary key is defined using the PRIMARY KEY statement when creating a table. If multiple columns are used as part of the primary key, they should be separated by commas:

```
PRIMARY KEY (column_name, column_name ... )
```

In the following example, a table is created using two columns as the primary key:

```
CREATE TABLE product
(
  prod_code INT NOT NULL AUTO_INCREMENT,
  prod_name char(30) NOT NULL,
  prod_desc char(60) NULL,
  PRIMARY KEY (prod_code, prod_name)
);
```

8.6 Using AUTO_INCREMENT

AUTO_INCREMENT is one of the simplest and most useful column definitions in SQL. When a column is defined with AUTO_INCREMENT, its value automatically increases each time a new row is added to the table. This feature is particularly helpful for columns used as primary keys, as it eliminates the need to write SQL statements to generate a new unique ID for each row. The MySQL server manages this process when a new row is inserted.

There are two important rules to follow when using AUTO_INCREMENT. First, only one column per table can be assigned AUTO_INCREMENT status. Second, the AUTO_INCREMENT column must be indexed, typically by declaring it as the primary key.

You can override the AUTO_INCREMENT value by specifying your own value in an INSERT statement. As long as the specified value is unique, it will be used for the new row, and subsequent increments will adjust based on the newly inserted value.

To retrieve the most recent AUTO_INCREMENT value, you can use the LAST_INSERT_ID() function as follows:

```
SELECT LAST_INSERT_ID();
+-----+
| LAST_INSERT_ID() |
+-----+
```

```
|          192 |
+-----+
1 row in set (0.00 sec)
```

8.7 Defining default values during table creation

Default values during table creation allow values to be specified for each column to be used when a value is not explicitly defined when rows are inserted into a database.

Default values are specified using the `DEFAULT` keyword in the `CREATE TABLE` statement. For example, the following SQL statement specifies a default value for the `sales_quantity` column:

```
CREATE TABLE sales
(
    sales_number int NOT NULL,
    sales_quantity int NOT NULL DEFAULT 1,
    sales_desc char(20) NOT NULL,
    PRIMARY KEY (sales_number)
);
```

If no quantity is defined when a new row is inserted, MySQL will insert the default value of 1 for the `sales_quantity` column.

8.8 Displaying table schema

The `DESCRIBE` statement displays the schema of a database table, including column names, data types, primary key details, null and default settings, and auto-increment information:

```
DESCRIBE customer;
+-----+-----+-----+-----+-----+-----+
| Field          | Type   | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| customer_id    | int    | NO   | PRI | NULL    | auto_increment |
| customer_name  | char(20) | NO   |     | NULL    |                 |
| customer_address | char(20) | YES  |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

8.9 Deleting databases and tables

Databases and tables can be deleted from a MySQL server using the `DROP` statement. To delete the `sales` table from the `sampledb` database, use the following command:

```
DROP TABLE sales;
```

Alternatively, use the `TRUNCATE TABLE` statement to permanently remove all rows from a table without deleting the table itself:

```
TRUNCATE TABLE sales;
```

Similarly, you can delete the entire `sampledb` database using the following command:

```
DROP DATABASE sampled;
```

8.10 MySQL storage engine types

MySQL comes with several storage engines, each offering specific advantages. Using the `ENGINE=` directive, you can choose the storage engine for each table individually. Some of the storage engines currently available in MySQL include:

- **InnoDB** - The InnoDB storage engine was introduced with MySQL version 4.0 and is known for being transaction-safe. A transaction-safe storage engine guarantees that all database transactions are fully completed and will roll back any partially completed transactions (for example, as a result of a server or power failure). This ensures that a database is never left with incomplete data updates.
- **MEMORY** - The MEMORY storage engine stores data in memory rather than on disk. This makes the engine extremely fast. However, the transient nature of data in memory means this engine is more suitable for temporary table storage.
- **CSV** - The CSV storage engine saves data in plain text files using a comma-separated values (CSV) format. This engine is particularly useful when the stored data needs to be exchanged with other platforms, such as spreadsheets or accounting software.
- **ARCHIVE** - The ARCHIVE engine utilizes zlib compression to minimize the storage space required for large data volumes.

To generate a list of supported engines, use the following `SHOW ENGINES` statement:

```
SHOW ENGINES\G
***** 1. row *****
    Engine: ndbcluster
    Support: NO
    Comment: Clustered, fault-tolerant tables
    Transactions: NULL
        XA: NULL
    Savepoints: NULL
***** 2. row *****
    Engine: MEMORY
    Support: YES
    Comment: Hash based, stored in memory, useful for temporary tables
    Transactions: NO
        XA: NO
    Savepoints: NO
***** 3. row *****
    Engine: InnoDB
    Support: DEFAULT
    Comment: Supports transactions, row-level locking, and foreign keys
    Transactions: YES
        XA: YES
    Savepoints: YES
***** 4. row *****
    Engine: PERFORMANCE_SCHEMA
    Support: YES
    Comment: Performance Schema
.
.
```

Different engine types can be used within a database; for example, some tables may utilize the InnoDB engine, while others might use the CSV engine. If an engine type is not specified when creating a table, MySQL will

Creating Databases and Tables

default to using the InnoDB engine for that table. To identify the default engine, you can use the following statement:

```
SELECT @@default_storage_engine;
+-----+
| @@default_storage_engine |
+-----+
| InnoDB                   |
+-----+
1 row in set (0.01 sec)
```

To specify a particular engine type for a table, add the appropriate `ENGINE=` definition after defining the table columns. For instance, the following example specifies the `MEMORY` engine:

```
CREATE TABLE tmp_orders (
    tmp_number INT NOT NULL,
    tmp_quantity INT NOT NULL,
    tmp_desc CHAR(20) NOT NULL,
    PRIMARY KEY (tmp_number)
) ENGINE=MEMORY;
```

8.11 Knowledge test



Click the link below or scan the QR code to test that you understand how to create MySQL databases and tables:

<https://www.answertopia.com/2dvd>



8.12 Reference points

The main points covered in this chapter are as follows:

• Creating Databases

- Use `CREATE DATABASE dbname;` or `CREATE SCHEMA dbname;`
- Avoid conflicts with existing databases using `IF NOT EXISTS`.
- List databases with `SHOW DATABASES;`

• Enabling Table Encryption

- Default encryption is off (`default_table_encryption = OFF`).
- Enable encryption during database creation: `CREATE DATABASE dbname ENCRYPTION = 'Y';`
- Set encryption globally: `SET GLOBAL default_table_encryption=ON;`

• Creating Tables

- Use `CREATE TABLE` to define columns, data types, constraints, and primary keys.
- Specify a database with `USE dbname` before creating tables.

• Key Table Features

- Primary Keys: Ensure unique identifiers for rows.
- AUTO_INCREMENT: Automatically generate unique values for a column.
- NULL/NOT NULL: Define whether columns can accept null values.
- DEFAULT Values: Provide default values for columns when no input is given.
- **Viewing Table Schema**
 - Use `DESCRIBE tablename;` to display column definitions and constraints.
- **Deleting Databases and Tables**
 - Remove tables: `DROP TABLE tablename;`
 - Delete databases: `DROP DATABASE dbname;`
 - Remove table data: `TRUNCATE TABLE tablename;`
- **Storage engine Types**
 - InnoDB: Transaction-safe, prevents partial updates, lacks full-text search.
 - MyISAM: High performance with full-text search, not transaction-safe.
 - MEMORY: Stores data in memory for fast access, suitable for temporary tables.
 - Specify engine during table creation: `ENGINE=<engine name>`

9. Inserting Data into Database Tables

Up to this point, we have explored the process of installing MySQL and covered the essential steps for creating and managing databases and tables. This chapter will take the next step by focusing on populating your database tables with data. Specifically, you will learn how to use the SQL INSERT statement to add rows of data to your tables.

9.1 Creating the sample table

Start the MySQL client, select the *sampledb* database, and delete the tables created in the previous chapter:

```
USE sampled  
DROP TABLE customer, product, tmp_orders;
```

Next, recreate the *product* table using the SQL statement provided below:

```
CREATE TABLE product  
(  
  id INT NOT NULL AUTO_INCREMENT,  
  name VARCHAR(25) NOT NULL,  
  description VARCHAR(20) NULL,  
  price DECIMAL(6,2),  
  quantity INT,  
  PRIMARY KEY (id)  
);
```

9.2 The basics of data insertion

The SQL INSERT statement is used to add new rows of data to a specified database table. With the INSERT statement, you can insert entire, partial, or multiple rows or rows generated from a database query. In this chapter, we will explore each of these techniques.

9.3 Inserting a row

To add a single row to a table, the INSERT statement requires the name of the table where the row will be added, along with the corresponding column names and their associated values. The INSERT statement syntax consists of a comma-separated list of column names enclosed in parentheses, followed by the VALUES keyword and then a comma-separated list of values matching the order of the column names, as outlined below:

```
INSERT INTO <table_name>(  
  <column_name_1>,  
  <column_name_2>,  
  <column_name_3>,  
  .  
  .  
)  
VALUES (  
  <value_1>,  
  <value_2>,  
  <value_3>,  
  .  
  .  
  .  
);
```

```
};
```

Using the following SQL statement, add a row to the product table:

```
INSERT INTO product (
    id,
    name,
    description,
    price,
    quantity)
VALUES (
    NULL,
    'MacBook Pro M4',
    'Apple laptop',
    2499.99,
    10
);
```

Note that the id value is specified as NULL because the id column in the product table is set to AUTO_INCREMENT. This means we do not need to assign a value manually because the MySQL database engine will automatically generate a new value each time a row is added.

To verify that the row was successfully inserted into the table, we need to execute a query using the SELECT statement. We will discuss database queries in detail in the next chapter, but for now, run the following statement to list the rows in the product table:

```
SELECT * FROM product;
```

id	name	description	price	quantity
1	MacBook Pro M4	Apple laptop	2499.99	10

```
1 row in set (0.00 sec)
```

A more concise way to add records to a table is by providing only the values. While this approach can be effective, the values must be specified in the exact order corresponding to the columns, as defined when the table was created. If the order is incorrect, you may encounter an error message (especially if the data types do not align with the columns), or worse, you could insert data into the wrong columns. Additionally, this method may work with the current table structure, but it could lead to issues if the schema is modified in the future. The syntax for this insertion technique is as follows:

```
INSERT INTO <table_name> VALUES (
    <value_1>,
    <value_2>,
    <value_3>,
    .
    .
);
```

With the above warnings in mind, use the following statement to insert a record without the referencing column names:

```
INSERT INTO product VALUES (
    NULL,
```

```
'MacBook Air M3',
'Apple laptop',
1499.99,
5
);
```

Re-run the SELECT statement to confirm the new row has been added:

```
SELECT * FROM product;
+----+-----+-----+-----+-----+
| id | name          | description      | price  | quantity |
+----+-----+-----+-----+-----+
| 1  | MacBook Pro M4 | Apple laptop     | 2499.99 | 10        |
| 2  | MacBook Air M3 | Apple laptop     | 1499.99 | 5         |
+----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

9.4 Adding multiple rows to a table

To add multiple rows to a table, you can use several INSERT statements or include all the rows in a single statement. When adding multiple rows to a MySQL database table using one statement, separate each set of values with a comma, as shown in the example below:

```
INSERT INTO product (
    id,
    name,
    description,
    price,
    quantity)
VALUES (
    NULL,
    'Mac Mini M3',
    'Apple desktop',
    999.99,
    100
),
(
    NULL,
    'Mac Studio',
    'Apple desktop',
    1999.99,
    15
);
```

Use the SELECT statement once again to verify the insertion worked:

```
SELECT * FROM product;
+----+-----+-----+-----+-----+
| id | name          | description      | price  | quantity |
+----+-----+-----+-----+-----+
| 1  | MacBook Pro M4 | Apple laptop     | 2499.99 | 10        |
| 2  | MacBook Air M3 | Apple laptop     | 1499.99 | 5         |
| 3  | Mac Mini M3    | Apple desktop    | 999.99  | 100       |
| 4  | Mac Studio     | Apple desktop    | 1999.99 | 15        |
+----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

9.5 Inserting results from a SELECT statement

A useful technique involves combining the INSERT and SELECT statements. In this approach, the SELECT statement retrieves data from a table, and the INSERT statement adds this data to a different table. This method is known as INSERT SELECT, the syntax for which is as follows:

```
INSERT INTO <table_name>(
    <column_name_1>,
    <column_name_2>,
    <column_name_3>,
)
SELECT <column_name_1>, <column_name_2>, <column_name_3> FROM <table_name>;
```

The syntax outlined above assumes that all rows and columns from the source table are inserted into the destination table. In practice, however, you can select which rows and columns to include in the insertion.

As an example, create a new table named `cheaper_product` that includes only `id`, `name`, and `price` columns:

```
CREATE TABLE cheaper_product
(
    id INT NOT NULL AUTO_INCREMENT,
    name VARCHAR(25) NOT NULL,
    price DECIMAL(6,2),
    PRIMARY KEY (id)
);
```

The goal is to select rows from the `product` table where the price is less than \$1,500.00 and then insert the values of the `id`, `name`, and `price` columns from those rows into the `cheaper_product` table. This can be accomplished using the following SQL statement:

```
INSERT INTO cheaper_product
(
    id, name, price
)
SELECT id, name, price FROM product WHERE price < 1500;
```

After the insertion executes, the `cheaper_product` table should contain only rows matching the price criteria:

```
SELECT * FROM cheaper_product;
+----+-----+-----+
| id | name          | price |
+----+-----+-----+
|  2 | MacBook Air M3 | 1499.99 |
|  3 | Mac Mini M3    |  999.99 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

The examples above assume that the `cheaper_product` table already exists and that its schema partly matches the source table. If you want to create a new table and populate it with data from another table, you can combine the CREATE TABLE and SELECT statements. You can copy all the rows and columns from the source table or select specific columns and rows that meet certain criteria.

For instance, in the following example, a new table named `expensive_product` is created and populated with items from the `product` table that cost more than \$1,500.00:

```
CREATE TABLE expensive_product
```

```
SELECT id, name, price FROM product
WHERE price > 1500;
Query OK, 2 rows affected (0.06 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
SELECT * FROM expensive_product;
+----+-----+-----+
| id | name       | price |
+----+-----+-----+
|  1 | MacBook Pro M4 | 2499.99 |
|  4 | Mac Studio    | 1999.99 |
+----+-----+-----+
2 rows in set (0.01 sec)
```

To avoid cluttering the sample database, drop the two new tables before continuing:

```
DROP TABLE cheaper_product, expensive_product;
```

9.6 Reducing the INSERT performance load

The INSERT statement can place a considerable load on the database server, especially during high-volume data insertions. This increased load may adversely affect other transactions, such as data reads, by slowing down their performance.

To minimize the impact of insert operations on the server, you can use the `LOW_PRIORITY` keyword for non-urgent tasks. This approach ensures that critical operations, such as reads, are prioritized over insertions. For example:

```
INSERT LOW_PRIORITY INTO products(
.
.
VALUES (
.
.
.
);
```

The `LOW_PRIORITY` option instructs the database engine to delay the execution of the operation until all other clients have finished accessing the table. This ensures that the operation does not interfere with ongoing transactions, allowing higher-priority tasks to complete without disruption.

9.7 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of the MySQL INSERT statement:

<https://www.answerstopia.com/ztw8>



9.8 Reference points

The main points covered in this chapter are as follows:

- **Data Insertion Basics**
 - Use the `INSERT` statement to add rows to a database table.

Inserting Data into Database Tables

- Specify column names for clarity and to prevent errors when table structures change.
- **Single Row Insertion**
 - Add individual rows by providing values for each column or by relying on `AUTO_INCREMENT` for unique identifiers.
- **Multiple Row Insertion**
 - Insert multiple rows in one statement for efficiency.
 - Separate each row's values with commas.
- **Using SELECT for Insertion**
 - Combine `INSERT` and `SELECT` to populate one table with data retrieved from another.
 - Useful for transferring or filtering data between tables.
- **Table Creation with SELECT**
 - Create and populate a new table directly from the results of a `SELECT` query.
 - Customize columns and rows in the new table based on criteria.
- **Improving Insert Performance**
 - Use the `LOW_PRIORITY` keyword to minimize server load during high-volume insert operations, prioritizing other transactions.

10. Updating and Managing MySQL Tables

Modifying a table after it has been created and populated with data is generally not advised. However, changes are necessary in some circumstances, and these can be made using specific SQL statements. Additionally, it is possible to delete an existing table from a database. This chapter will cover both of these topics in detail.

10.1 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapter. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql managing tables.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled_b
```

10.2 Altering a database table

A preexisting table in a database can be modified using the SQL `ALTER TABLE` statement, which allows you to change the structure of a table. Supported modifications include adding or removing columns, changing a column's data type, and renaming the table or its columns.

Behind the scenes, MySQL's alteration process typically involves creating a temporary table, applying the desired changes to this temporary table, deleting the original table, and then renaming the temporary table to take its place.

The syntax for ALTER TABLE requires the name of the table to be modified, followed by the list of changes to be made:

```
ALTER TABLE <table name> <alteration spec1>, <alteration spec2>, ....
```

10.3 Adding table columns

To add a column to an existing database table, you need to construct an ALTER TABLE statement. This statement includes the table name, the ADD keyword, and the name along with the data type of the new column. For example, the following SQL statement adds a column named `on_sale` to the `product` table:

```
ALTER TABLE product ADD on_sale BOOL;
```

After adding the new column, you can use the `SHOW COLUMNS FROM` statement to view the updated list of columns:

```
SHOW COLUMNS FROM product;
```

Updating and Managing MySQL Tables

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
price	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	
on_sale	tinyint(1)	YES		NULL	

6 rows in set (0.00 sec)

By default, new columns are added to the end of the current list of columns. To add a column at the beginning of the list, use the ALTER TABLE command with the FIRST keyword. Use the following statement to add a date field as the first column in the table:

```
ALTER TABLE product ADD release_date DATE FIRST;
```

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
release_date	date	YES		NULL	
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
price	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	
on_sale	tinyint(1)	YES		NULL	

7 rows in set (0.00 sec)

To insert a column after an existing one, use the AFTER option. The following statement adds a discount field after the price column:

```
ALTER TABLE product ADD discount DECIMAL(6,2) AFTER price;
```

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
release_date	date	YES		NULL	
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
price	decimal(6,2)	YES		NULL	
discount	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	
on_sale	tinyint(1)	YES		NULL	

8 rows in set (0.00 sec)

The decimal declaration for the price column needs clarification. The syntax for the DECIMAL data type is as follows:

```
DECIMAL (M, D)
```

In the syntax above, M indicates the total number of digits the column can store, and D represents the number of decimal places. Therefore, the (6,2) specification for the price column defines a range from -9999.99 to 9999.99.

10.4 Moving table columns

In addition to adding new columns, the ALTER TABLE statement can be used to change the position of an existing column in a table. This is done using the MODIFY COLUMN clause along with either the AFTER or FIRST options. The syntax is as follows:

```
ALTER TABLE <table_name> MODIFY COLUMN <column> <definition> ... AFTER|FIRST
<column>;
```

For example, to place the *release_date* field immediately after the *price* column, you would use the following statement:

```
ALTER TABLE product MODIFY COLUMN release_date DATE AFTER price;
```

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
price	decimal(6,2)	YES		NULL	
release_date	date	YES		NULL	
discount	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	
on_sale	tinyint(1)	YES		NULL	

8 rows in set (0.01 sec)

10.5 Renaming tables

Tables are renamed using variations of the RENAME TABLE or ALTER TABLE statements. The syntax to change a database table name using RENAME TABLE is as follows:

```
RENAME TABLE <old_name> TO <new_name>;
```

The following statement, for example, renames the product table to new_product:

```
RENAME TABLE product TO new_product;
```

The same result can be achieved using an ALTER TABLE statement. The following example renames new_product back to product:

```
ALTER TABLE new_product RENAME product;
```

10.6 Renaming table columns

Columns can be renamed using the ALTER TABLE statement and CHANGE keyword, which requires the old and new column names followed by the column's data type. For example, to change the column name from discount to discount_price, you would use the following statement:

```
ALTER TABLE product CHANGE discount discount_price DECIMAL(6,2);
```

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
price	decimal(6,2)	YES		NULL	
release_date	date	YES		NULL	
discount_price	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	
on_sale	tinyint(1)	YES		NULL	

8 rows in set (0.00 sec)

10.7 Deleting table columns

In addition to adding and renaming columns, you can also delete columns from tables using the DROP COLUMN option of the ALTER TABLE statement. Using the following statement, delete the release_date, discount_price, and on_sale columns:

```
ALTER TABLE product DROP COLUMN release_date;
ALTER TABLE product DROP COLUMN discount_price;
ALTER TABLE product DROP COLUMN on_sale;
```

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
retail_price	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	

5 rows in set (0.02 sec)

10.8 Changing the data type of a column

You can change the data type of a column in a table by using the ALTER TABLE statement along with the MODIFY option. For example, you can change the data type of the quantity column from INT to SMALLINT, and increase the character length of the “description” column to 50 characters as follows:

```
ALTER TABLE product MODIFY quantity SMALLINT;
ALTER TABLE product MODIFY description VARCHAR(50);
```

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(50)	YES		NULL	
price	decimal(6,2)	YES		NULL	
quantity	smallint	YES		NULL	

5 rows in set (0.00 sec)

There are no restrictions on changing column data types in empty tables. However, once a table contains rows, attempts to change a column's data type may fail or result in data loss. In the example above, we successfully increased the character length of the *description* column without any issues. If the table contained data, though, trying to make the column shorter than the longest stored description would lead to an error:

```
ALTER TABLE product MODIFY description VARCHAR(5);
ERROR 1265 (01000): Data truncated for column 'description' at row 1
```

10.9 Deleting tables

A table may be deleted from a database using the DROP TABLE statement. All that is required with this statement is the name of the table to be deleted. The deletion is permanent and cannot be undone:

```
DROP TABLE <table_name>;
```

10.10 Duplicating tables

In MySQL, tables can be duplicated in two ways: either by duplicating just the table's data or by replicating its entire structure, which includes keys, indexes, and auto increment settings along with the data.

To create a new table based solely on the data from another, we combine the CREATE TABLE and SELECT statements as follows:

```
CREATE TABLE <new_table> AS SELECT * FROM <old_table>;
```

For example, use the following statement to copy the product table as a new table called new_product:

```
CREATE TABLE new_product AS SELECT * FROM product;
```

If we examine the rows in the new_product table, they are identical to those in the original product table:

```
SELECT * FROM new_product;
```

id	name	description	price	quantity
1	MacBook Pro M4	Apple laptop	2499.99	10
2	MacBook Air M3	Apple laptop	1499.99	5
3	Mac Mini M3	Apple desktop	999.99	100
4	Mac Studio	Apple desktop	1999.99	15

4 rows in set (0.01 sec)

If we examine the column structure, however, we discover that the auto increment and primary key definitions were lost during the duplication:

```
SHOW COLUMNS FROM product;
```

Field	Type	Null	Key	Default	Extra
id	int	NO		NULL	
name	varchar(25)	NO		NULL	
description	varchar(20)	YES		NULL	
retail_price	decimal(6,2)	YES		NULL	
quantity	int	YES		NULL	

5 rows in set (0.02 sec)

Updating and Managing MySQL Tables

To maintain the original table's structure in the new table, we must first create a duplicate table using the CREATE TABLE statement along with the LIKE keyword:

```
CREATE TABLE <new_table> LIKE <old_table>;
```

After duplicating the table structure, we can use INSERT and SELECT statements to replicate the data:

```
INSERT INTO <new_table> SELECT * FROM <original_table>;
```

Use the following statements to delete the new_product table and create a duplicate that maintains the table structure:

```
DROP TABLE new_product;
```

```
CREATE TABLE new_product LIKE product;
```

Before duplicating the data, check that the table structure has been preserved in the new table:

```
SHOW COLUMNS FROM new_product;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	NO		NULL	
description	varchar(50)	YES		NULL	
price	decimal(6,2)	YES		NULL	
quantity	smallint	YES		NULL	

5 rows in set (0.01 sec)

With the table structure duplicated, we can now copy the data to the new table:

```
INSERT INTO new_product SELECT * FROM product;
```

Re-run the earlier SELECT * FROM statement to confirm the data duplication before removing the new_product table from the database:

```
DROP TABLE new_product;
```

When duplicating tables, keep in mind that triggers, constraints, and custom properties are not retained, even if the table structure is preserved. Additionally, duplication is a resource-intensive process, especially for large tables.

10.11 Knowledge test



Click the link below or scan the QR code to test that you know how to update and manage MySQL tables:

<https://www.answerstopia.com/ywyh>



10.12 Reference points

The main points covered in this chapter are as follows:

- **Altering Tables**

- Existing tables can be modified to add, remove, or reposition columns.
- Use `ALTER TABLE` to modify an existing table's structure.
- **Repositioning Columns**
 - Columns can be reordered using specific keywords to adjust their placement in the table.
 - Use `FIRST` or `AFTER` keywords to adjust column order.
- **Renaming**
 - Tables and columns can be renamed.
 - Use `RENAME TABLE` or `ALTER TABLE` to rename tables and columns.
- **Deleting Columns**
 - Unnecessary columns can be permanently removed from tables.
 - Remove unwanted columns with `ALTER TABLE ... DROP COLUMN`
- **Changing Data Types**
 - Column data types can be modified, though changes to populated tables may risk data truncation or loss.
 - Use `ALTER TABLE ... MODIFY` to alter column data types.
- **Deleting Tables**
 - Tables can be permanently removed.
 - Permanently delete tables using `DROP TABLE`
- **Table Duplication**
 - MySQL allows duplication of a table's data, structure, or both.
 - Data-only duplication does not preserve keys, indexes, or other structural properties.
 - Structure duplication replicates the original table's schema, including primary keys and `auto_increment` settings.
- **Data-Only Duplication**
 - Use the `CREATE TABLE ... AS SELECT` statement to duplicate table data into a new table.
 - This approach copies all rows but excludes structural definitions like primary keys, indexes, and `auto_increment` properties.
- **Structural Duplication**
 - Use the `CREATE TABLE ... LIKE` statement to duplicate the schema of an existing table.
 - This preserves all column definitions, keys, indexes, and table properties.
- **Combining Structure and Data Duplication**
 - First, duplicate the structure using `CREATE TABLE ... LIKE`.

- Then, populate the new table with data using an `INSERT INTO . . . SELECT` statement.

11. Updating and Deleting Table Data

After data has been added to a MySQL database table, there will eventually come a time when some or all of that data needs to be modified or removed. This could involve correcting errors, updating information to reflect changes in the real world, or cleaning up outdated records. Managing existing data is a crucial aspect of working with a database. This chapter will explore the fundamental operations for updating and deleting data in table rows.

11.1 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"
mysql -u root -p sampled < mysql_updating_tables.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

11.2 Updating database data

When updating data in a table, you can update specific rows or all rows at once. This is done using the SQL UPDATE statement, along with the SET and WHERE keywords.

The UPDATE statement requires several key pieces of information. First, you need to specify the name of the table you want to update. Next, you must indicate which columns will be updated and provide the new values for those columns. It is also essential to specify which specific rows should be updated using the WHERE keyword. The WHERE condition is crucial to remember; if you omit it, the update will affect every row in the table. The syntax for the UPDATE statement is as follows:

```
UPDATE <table_name>
SET <column_name1> = <new_value1>,
    <column_name2> = <new_value2>,
    .
    .
WHERE <select_criteria>;
```

To illustrate this, we will modify the description of a specific item in the product table. Start by listing the current rows using the following SELECT statement:

```
SELECT * FROM product;
```

id	name	description	price	quantity
1	MacBook Pro M4	Apple laptop	2499.99	10
2	MacBook Air M3	Apple laptop	1499.99	5
3	Mac Mini M3	Apple desktop	999.99	100

Updating and Deleting Table Data

4	Mac Studio	Apple desktop	1999.99	15
4 rows in set (0.00 sec)				

The objective is to update the MacBook Pro M4 record by adding the screen size to both the name and description columns. To perform this modification, enter the following UPDATE statement at the mysql> prompt:

```
UPDATE product
SET name = 'MacBook Pro M4 14-in',
description = 'Apple 14-in laptop'
WHERE name = 'MacBook Pro M4';
```

After updating the row, use the following statement to verify that the change was successful:

```
SELECT * from product WHERE name= 'MacBook Pro M4 14-in';
```

id	name	description	price	quantity
1	MacBook Pro M4 14-in	Apple 14-in laptop	2499.99	10
1 row in set (0.00 sec)				

Let’s assume we have received a delivery of 50 units for each of the MacBook models at our hypothetical warehouse. To update the table with this information, we can use the LIKE keyword and the ‘%’ wildcard to select all MacBook products. We will then add 50 to the existing quantity for each matching item. Use the following statement to perform this update operation:

```
UPDATE product
SET quantity = quantity + 50
WHERE name LIKE 'MacBook%';
```

The statement selects all rows where the product name starts with “MacBook”, increments the current quantity by 50, and updates each row. Run the following SELECT statement to verify the results:

```
SELECT * FROM product WHERE name LIKE 'Macbook%';
```

id	name	description	price	quantity
1	MacBook Pro M4 14-in	Apple 14-in laptop	2499.99	60
2	MacBook Air M3	Apple laptop	1499.99	55
2 rows in set (0.04 sec)				

11.3 Ignoring update errors

If an UPDATE statement is intended to update multiple rows, but an error occurs while updating some of those rows, the entire update is canceled. Consequently, any rows that were changed will revert to their original values. However, if you use the IGNORE keyword, the update will proceed, skipping over any rows that cause problems:

```
UPDATE IGNORE <table_name>
SET ...
WHERE ...;
```

11.4 Understanding auto-commit

Before we discuss deleting table rows and the associated risks of accidentally removing data, it’s essential to understand the concept of commits in MySQL. By default, a MySQL Server installation is set to commit changes

automatically. This means that once an SQL statement is executed, the change is immediately saved to the database, and no “undo” option is available.

The *autocommit* variable controls auto-commit behavior. You can check the current state of this variable using the following statement:

```
SHOW VARIABLES WHERE Variable_name='autocommit';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| autocommit    | ON    |
+-----+-----+
1 row in set (0.01 sec)
```

The benefit of auto-commit is that you don’t have to remember to execute a COMMIT after making changes. To understand the significance of committing changes, first turn off auto-commit as follows:

```
SET autocommit = 0;
```

Changes to the autocommit variable only take effect in the current session. The next time you start the client, auto-commit will be enabled again.

Next, add a new row to the product table:

```
INSERT INTO product(
    id,
    name,
    description,
    price,
    quantity)
VALUES(
    NULL,
    'iPhone 17 Pro',
    'Apple iPhone',
    999.99,
    30
);
```

After adding the row, verify that it appears in the table before exiting the MySQL client:

```
SELECT * FROM product WHERE name LIKE 'iPhone%';
+----+-----+-----+-----+-----+
| id | name          | description          | price | quantity |
+----+-----+-----+-----+-----+
|  5 | iPhone 17 Pro | Apple iPhone         | 999.99 |        30 |
+----+-----+-----+-----+-----+
1 row in set (0.00 sec)

exit
Bye
```

Restart the MySQL client and check for the new row again:

```
USE sampledb
SELECT * FROM product WHERE name LIKE 'iPhone%';
Empty set (0.00 sec)
```

Updating and Deleting Table Data

The iPhone product entry is no longer listed because the INSERT operation was not committed before the client session ended. If auto-commit is disabled, always enter the COMMIT statement before ending the session to avoid losing changes:

```
INSERT ....  
COMMIT;
```

Re-insert the iPhone record and execute the COMMIT statement. Then, exit and restart the client to verify that the iPhone record has been preserved.

Since it prevents data loss from forgotten commits, auto-commit is ideal for adding rows and columns to a table. However, it does have some drawbacks. A particular risk is that destructive operations are also committed automatically, such as executing a DELETE operation that removes all records from a table. To avoid this risk, consider using transactions and rollbacks when performing operations that could lead to data loss.

Re-enable autocommit before proceeding:

```
SET autocommit = 1;
```

11.5 Transactions and rollbacks

A MySQL *transaction* is a series of SQL statements executed sequentially. You can use the START TRANSACTION or BEGIN statements to initiate a transaction. It's important to note that even if the autocommit setting is enabled, the operations performed within a transaction must be committed manually. For example:

```
BEGIN;  
INSERT ....;  
DELETE ....;  
ALTER ...;  
COMMIT;
```

Transactions are advantageous when used in conjunction with the ROLLBACK statement. You can think of ROLLBACK as the opposite of the COMMIT statement. While a COMMIT operation finalizes the changes made since the transaction began, a ROLLBACK discards those changes and restores the database to its state before the transaction started. Instead of turning off auto-commit before executing potentially risky operations, it is better to perform those operations within a transaction. After reviewing the changes, you can commit or roll back the transaction.

11.6 Deleting table rows

You can delete all rows or specific rows from a table using the SQL DELETE statement. Using this statement in combination with the WHERE clause is highly recommended. If you omit the WHERE clause, all rows in the table will be deleted. To ensure that your criteria work as intended, it's advisable to test them first using a SELECT statement. Additionally, consider performing the deletion within a transaction for added safety.

To remove rows from our product table where the name field is equal to 'Mac Studio', you can run the following transaction:

```
BEGIN;  
DELETE FROM product WHERE name = 'Mac Studio';
```

The Mac Studio row should no longer appear in the table:

```
SELECT * FROM product;  
+----+-----+-----+-----+-----+  
| id | name | description | price | quantity |
```

```
+-----+-----+-----+-----+
| 1 | MacBook Pro M4 14-in | Apple 14-in laptop | 2499.99 | 60 |
| 2 | MacBook Air M3       | Apple laptop       | 1499.99 | 55 |
| 3 | Mac Mini M3          | Apple desktop      | 999.99  | 100 |
| 6 | iPhone 17 Pro        | Apple iPhone       | 999.99  | 30 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Next, use **ROLLBACK** to reverse the deletion:

```
ROLLBACK;
```

```
SELECT * FROM product;
```

```
+-----+-----+-----+-----+
| id | name                | description          | price  | quantity |
+-----+-----+-----+-----+
| 1 | MacBook Pro M4 14-in | Apple 14-in laptop  | 2499.99 | 60 |
| 2 | MacBook Air M3       | Apple laptop        | 1499.99 | 55 |
| 3 | Mac Mini M3          | Apple desktop       | 999.99  | 100 |
| 4 | Mac Studio           | Apple desktop       | 1999.99 | 15 |
| 6 | iPhone 17 Pro        | Apple iPhone        | 999.99  | 30 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Note that the row has been restored to its original position in the table.

When using the **DELETE** statement to remove all rows from a table, be aware that it only deletes the rows themselves while leaving the table intact. To permanently delete the table and all of its contents, use the **DROP TABLE** statement:

```
DROP TABLE <table_name>;
```

11.7 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL table updates:

<https://www.answerstopia.com/n4jm>



11.8 Reference points

The main points covered in this chapter are as follows:

• Updating Data

- Use the **UPDATE** statement with **SET** to modify specific columns in a table.
- Include a **WHERE** clause to target specific rows; omitting it updates all rows in the table.
- Use wildcards (e.g., %) with **LIKE** to update rows matching a pattern.

• Ignoring Update Errors

- Use the **IGNORE** keyword to skip problematic rows during updates while processing others.

• Understanding Auto-Commit

Updating and Deleting Table Data

- MySQL commits changes immediately by default.
- Disable auto-commit temporarily to review changes before committing them.
- Use `COMMIT` to save changes or `ROLLBACK` to discard them when auto-commit is off.
- **Transactions and Rollbacks**
 - Transactions group multiple operations, providing the ability to commit or roll back as a unit.
 - Use `BEGIN` or `START TRANSACTION` to initiate, and `COMMIT` or `ROLLBACK` to finalize.
- **Deleting Data**
 - Use the `DELETE` statement to remove specific rows, and always include a `WHERE` clause to avoid deleting all rows.
 - Test criteria with a `SELECT` statement before performing deletions.
- **Reversing Deletions**
 - Use transactions to allow rollbacks if a deletion operation needs to be undone.
- **Removing All Rows vs. Dropping Tables**
 - `DELETE` removes rows but retains the table structure.
 - `DROP TABLE` deletes the table and its contents permanently.

12. Retrieving Data from MySQL Databases

A database system is only as valuable as its ability to retrieve the data it stores. Among the most widely used SQL commands is the `SELECT` statement, which is specifically designed to extract data from database tables according to defined criteria. This chapter will explore using the `SELECT` statement to perform basic database queries.

12.1 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"
mysql -u root -p sampled < mysql_retrieving_data.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

Next, insert two more rows into the product table:

```
INSERT INTO product
VALUES (
    NULL,
    'iPhone 17',
    'Apple iPhone',
    799.99,
    30
),
(
    NULL,
    'iMac',
    'Apple desktop',
    1299.99,
    30
);
```

12.2 Retrieving a single column

The most straightforward `SELECT` statement retrieves a single column from a table's rows. The following SQL statement, for example, extracts all entries from the description column in the product table:

```
SELECT description FROM product;
```

Once executed, this command will display a list of every product description contained in the product table:

```

+-----+
| description |
+-----+
| Apple 14-in laptop |
| Apple laptop |
| Apple desktop |
| Apple desktop |
| Apple iPhone |
| Apple iPhone |
| Apple desktop |
+-----+
7 rows in set (0.00 sec)

```

12.3 Using SELECT to retrieve multiple columns

So far, we've explored how simple it is to extract a single column from each table row. In reality, it's more common to need information from multiple columns. Fortunately, the SELECT statement allows us to do this easily. To retrieve data from multiple columns, you can list the column names after the SELECT statement, separating each name with a comma. For example, use the following statement to extract data from three columns in our database table:

```

SELECT name, description, quantity FROM product;
+-----+-----+-----+
| name | description | quantity |
+-----+-----+-----+
| MacBook Pro M4 14-in | Apple 14-in laptop | 60 |
| MacBook Air M3 | Apple laptop | 55 |
| Mac Mini M3 | Apple desktop | 100 |
| Mac Studio | Apple desktop | 15 |
| iPhone 17 Pro | Apple iPhone | 30 |
| iPhone 17 | Apple iPhone | 30 |
| iMac | Apple desktop | 30 |
+-----+-----+-----+
7 rows in set (0.00 sec)

```

While this approach works well if you want to display only some columns in a table, it can become cumbersome if the table has many columns and you want to retrieve all of them. A simpler solution is to use the wildcard symbol (*) instead of listing each column name individually. For example:

```

SELECT * FROM product;
+---+-----+-----+-----+-----+
| id | name | description | price | quantity |
+---+-----+-----+-----+-----+
| 1 | MacBook Pro M4 14-in | Apple 14-in laptop | 2499.99 | 60 |
| 2 | MacBook Air M3 | Apple laptop | 1499.99 | 55 |
| 3 | Mac Mini M3 | Apple desktop | 999.99 | 100 |
| 4 | Mac Studio | Apple desktop | 1999.99 | 15 |
| 6 | iPhone 17 Pro | Apple iPhone | 999.99 | 30 |
| 7 | iPhone 17 | Apple iPhone | 799.99 | 30 |
| 8 | iMac | Apple desktop | 1299.99 | 30 |
+---+-----+-----+-----+-----+
7 rows in set (0.00 sec)

```

12.4 Restricting the number of results

To limit the number of rows returned by a `SELECT` statement when retrieving data from a table, you can use the `LIMIT` keyword, followed by the number of rows to be retrieved. The following example retrieves the first two rows from the `product` table:

```
SELECT * FROM product LIMIT 2;
```

id	name	description	price	quantity
1	MacBook Pro M4 14-in	Apple 14-in laptop	2499.99	60
2	MacBook Air M3	Apple laptop	1499.99	55

2 rows in set (0.00 sec)

Using the `LIMIT` clause alongside the `OFFSET` keyword allows you to set a specific starting point for data retrieval within the table. The following example retrieves three rows, starting after the second row:

```
SELECT * FROM product LIMIT 3 OFFSET 2;
```

id	name	description	price	quantity
3	Mac Mini M3	Apple desktop	999.99	100
4	Mac Studio	Apple desktop	1999.99	15
6	iPhone 17 Pro	Apple iPhone	999.99	30

3 rows in set (0.00 sec)

The same results can be achieved by omitting the `OFFSET` keyword and specifying the offset and limit values as a comma-separated list, as follows:

```
SELECT * FROM product LIMIT <offset>, <row_count>;
```

The following statement, for example, retrieves three rows starting after the first row:

```
SELECT * FROM product LIMIT 1, 3;
```

id	name	description	price	quantity
2	MacBook Air M3	Apple laptop	1499.99	55
3	Mac Mini M3	Apple desktop	999.99	100
4	Mac Studio	Apple desktop	1999.99	15

3 rows in set (0.00 sec)

12.5 Eliminating duplicate values

It is common for a combination of column values to be duplicated across multiple rows in a database. As a result, when retrieving data—especially when focusing on a single column in a table—you may encounter repeated values in the results. In our `product` table, some rows have matching description text, resulting in duplicate items in the result set:

```
SELECT description FROM product;
```

description

```
| Apple 14-in laptop |
| Apple laptop      |
| Apple desktop     |
| Apple desktop     |
| Apple iPhone      |
| Apple iPhone      |
| Apple desktop     |
+-----+
7 rows in set (0.00 sec)
```

The output above indicates that our fictional online store offers multiple types of Mac desktops and Apple iPhones. While the product IDs and names are unique, the descriptions contain duplications. To create a list of product descriptions without any duplicates, we can use the `SELECT DISTINCT` statement:

```
SELECT DISTINCT description FROM product;
+-----+
| description |
+-----+
| Apple 14-in laptop |
| Apple laptop      |
| Apple desktop     |
| Apple iPhone      |
+-----+
4 rows in set (0.00 sec)
```

12.6 Sorting result sets

A `SELECT` statement typically retrieves data from a table in the order in which it was added. However, this order can change if the database table has undergone updates or deletions, making it difficult to predict the exact order of the data retrieval. To address this issue, SQL offers the `ORDER BY` clause, which allows us to sort the data as the `SELECT` statement retrieves it.

Let's begin by performing a retrieval on the product table:

```
SELECT description FROM product;
+-----+
| description |
+-----+
| Apple 14-in laptop |
| Apple laptop      |
| Apple desktop     |
| Apple desktop     |
| Apple iPhone      |
| Apple iPhone      |
| Apple desktop     |
+-----+
7 rows in set (0.00 sec)
```

When this command is executed and the data is extracted, the rows may seem to have been retrieved in no particular order. When we retrieve all the columns from the table instead of just the description column, we observe that the results are listed in the order of the `id` column. This is not surprising, as the `id` is an auto-increment value that reflects the order in which the data was added.

While these results might be acceptable in certain situations, there may be cases where you want the results set to be sorted. This is where the `ORDER BY` clause comes into play. Using `ORDER BY`, you can control the order in which the query results are displayed, ensuring they are sorted according to your specific requirements. The

syntax for incorporating ORDER BY in a SELECT statement is as follows:

```
SELECT <column_1>, <column_2> ... FROM <table_name> ORDER By <column_name>;
```

To continue with our example, we could, therefore, sort the product descriptions as follows:

```
SELECT description FROM product ORDER BY description;
+-----+
| description |
+-----+
| Apple iPhone 17 Pro |
| Apple desktop |
| Apple desktop |
| Apple laptop |
| Apple 14-in laptop |
+-----+
5 rows in set (0.04 sec)
```

In this case, the result set has been ordered alphabetically according to the description column.

The column used to order the result set does not have to be included in the retrieved columns. It is possible to sort by any column in a table, even if it is not referenced in the SELECT statement. The statement below retrieves the name column based on the quantity field, sorted from lowest to highest:

```
SELECT name FROM product ORDER BY quantity;
+-----+
| name |
+-----+
| Mac Studio |
| iPhone 17 |
| iMac |
| iPhone 17 Pro |
| MacBook Air M3 |
| MacBook Pro M4 14-in |
| Mac Mini M3 |
+-----+
7 rows in set (0.00 sec)
```

12.7 Sorting on multiple columns

In the previous examples, we used a single column from a table as the criterion for sorting result sets. However, the ORDER BY clause allows us to sort based on multiple columns. The columns to be used for sorting should be specified after the ORDER BY clause and separated by commas. For instance, to sort by both the product description and price in our table, we would execute the following SQL statement:

```
SELECT * FROM product ORDER BY description, price;
```

12.8 Sorting data in descending order

In this chapter, we have primarily focused on sorting our MySQL database queries in ascending order. However, there may be instances when, for example, a visitor to your website wants to view prices for a list of items, starting with the most expensive. The standard ORDER BY clause won't fulfill this requirement. Fortunately, SQL provides the DESC keyword, which allows you to specify that the results of a SELECT statement should be sorted in descending order. For example, to sort the products by price in descending order, you would use the following statement:

```
SELECT name, price FROM product ORDER BY price DESC;
+-----+-----+
| name          | price |
+-----+-----+
| MacBook Pro M4 14-in | 2499.99 |
| Mac Studio      | 1999.99 |
| MacBook Air M3   | 1499.99 |
| iMac            | 1299.99 |
| Mac Mini M3     | 999.99  |
| iPhone 17 Pro   | 999.99  |
| iPhone 17       | 799.99  |
+-----+-----+
7 rows in set (0.00 sec)
```

12.9 Expressions and aggregate functions

Many SQL statements, such as SELECT and INSERT, support a variety of expressions. We can even use SELECT to perform arithmetic:

```
SELECT SUM(2 * 10 / 6);
+-----+
| 2 * 10 / 6 |
+-----+
| 3.3333 |
+-----+
1 row in set (0.00 sec)
```

The previous example used the SUM() aggregate function to return the calculation result. MySQL offers a wide range of aggregate functions for various operations on data sets. Some of the most commonly used functions are summarized in Figure 12-1:

Function	Description
AVG()	Returns the average of the values in the selected column
BIT_AND()	Return bitwise AND
BIT_OR()	Return bitwise OR
BIT_XOR()	Return bitwise XOR
COUNT()	Returns the number of rows returned for a selection
GROUP_CONCAT()	Return a concatenated string
MAX()	Returns the maximum value for a column
MIN()	Returns the minimum value of a column
SUM()	Returns the sum of the values in a specified column

Figure 12-1

The syntax for using aggregate functions in SELECT statements is as follows:

```
SELECT <column_name>, <aggregate_func>(<column_name>)
FROM <table_name> GROUP BY <column_name>;
```

Aggregate functions operate on rows that share matching column values. For example, consider the product table, which shows three Mac desktops, two Apple iPhones, and two MacBook models categorized by the description column. To calculate the average price for each category, we would use the AVG() function. When

applying aggregate functions, we must specify the column used to group the results using the GROUP BY clause. In this case, we group the results based on the description column. With this information, we can determine the average price for each product category:

```
SELECT description, AVG(price) FROM product GROUP BY description;
```

description	AVG(price)
Apple 14-in laptop	2499.990000
Apple laptop	1499.990000
Apple desktop	1433.323333
Apple iPhone	899.990000

4 rows in set (0.00 sec)

We can also use the COUNT() function to determine the number of device types in each category:

```
SELECT description, COUNT(*) FROM product GROUP BY description;
```

description	COUNT(*)
Apple 14-in laptop	1
Apple laptop	1
Apple desktop	3
Apple iPhone	2

4 rows in set (0.01 sec)

Similarly, we can restrict our criteria to count the number of products priced above a specific threshold:

```
SELECT COUNT(price) FROM product WHERE price > 1000;
```

COUNT(price)
4

1 row in set (0.00 sec)

When using aggregation, we create temporary columns that hold the results returned by the aggregate function. Often, you may need to reference these aggregate columns later in the statement. For example, consider the case where the SUM() function is used to multiply the price and quantity columns to calculate the total inventory value grouped by name:

```
SELECT name, SUM(price*quantity) FROM product GROUP BY name;
```

name	SUM(price*quantity)
MacBook Pro M4 14-in	149999.40
MacBook Air M3	82499.45
Mac Mini M3	99999.00
Mac Studio	29999.85
iPhone 17 Pro	29999.70
iPhone 17	23999.70
iMac	38999.70

7 rows in set (0.00 sec)

However, suppose you need to sort the results based on the total value ranked from the lowest to the highest. We know the ORDER BY option will do this for us, but we need a way to reference the unnamed total column. The solution is to assign an alias to the aggregate column and reference it when using ORDER BY. In the statement below, the aggregate column is assigned an alias (total), which is used to order the result set:

```
SELECT name, SUM(price*quantity) AS total FROM product GROUP BY name ORDER BY total;
```

name	total
iPhone 17	23999.70
iPhone 17 Pro	29999.70
Mac Studio	29999.85
iMac	38999.70
MacBook Air M3	82499.45
Mac Mini M3	99999.00
MacBook Pro M4 14-in	149999.40

7 rows in set (0.00 sec)

SELECT statements are not restricted to a single aggregate function. It is perfectly valid to include calls to multiple functions, for example:

```
SELECT MIN(price) AS min_price, MAX(price) AS max_price FROM product;
```

min_price	max_price
799.99	2499.99

1 row in set (0.00 sec)

12.10 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL queries:

<https://www.answertopia.com/m7r9>



12.11 Reference points

The main points covered in this chapter are as follows:

• Single-Column Retrieval

- Use `SELECT <column_name> FROM <table_name>` to extract data from a single column in a table.

• Multi-Column Retrieval

- List multiple column names after `SELECT` to retrieve data from several columns.
- Use `SELECT *` to retrieve all columns from a table.

• Limiting Results

- Use `LIMIT` to restrict the number of rows returned.
- Combine `LIMIT` with `OFFSET` to skip a specific number of rows before retrieving data.
- **Eliminating Duplicates**
 - Use `SELECT DISTINCT` to return unique values from a column, removing duplicates.
- **Sorting Results**
 - Use `ORDER BY` to sort data in ascending (default) or descending (`DESC`) order.
 - Sort by multiple columns by separating them with commas.
- **Descending Order Sorting**
 - Use the `DESC` keyword to sort results in descending order, such as prices from highest to lowest.
- **Combining Multiple Criteria for Sorting**
 - Use `ORDER BY` with multiple columns to create more complex sorting patterns (e.g., by description and then price).
- **Expressions in SQL**
 - SQL supports expressions for arithmetic operations in statements like `SELECT`.
 - Example functions include basic math and aggregate calculations.
- **Aggregate Functions**
 - Common functions include `SUM()`, `AVG()`, `MIN()`, `MAX()`, and `COUNT()`.
 - Aggregate functions process rows grouped by shared values, often paired with `GROUP BY`.
- **Using GROUP BY**
 - Groups result sets based on matching column values.
 - Example: Calculate average prices by product category or count items per category.
- **Column Aliases**
 - Aggregations generate temporary columns that can be referenced using aliases.

13. Filtering Result Sets with the WHERE Clause

In the previous chapters, we explored various methods for retrieving data using the SELECT statement and examined how to sort the results. A key requirement when retrieving data is the ability to filter it, ensuring that only rows meeting specific search criteria are returned. This chapter will focus on filtering query results using the WHERE clause and the SELECT statement.

13.1 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “Start Here” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampledb;"
mysql -u root -p sampledb < mysql_where_filtering.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampledb
```

13.2 The basics of the WHERE clause

The WHERE clause is used with SELECT, INSERT, UPDATE, and DELETE statements to filter results based on specified criteria. The syntax for a SELECT WHERE statement is as follows:

```
SELECT column(s) FROM table WHERE column = value
```

The first part of the statement resembles a typical SELECT query. The WHERE clause changes things, however. After the WHERE keyword, you need to specify the column on which the filtering will be based, followed by an operator that defines the type of comparison to perform (in this case, we are looking for equality), and finally, the value that the column must match.

Using our product table as an example, a standard, unfiltered SELECT statement will return the following result set:

```
SELECT * FROM product;
```

id	name	description	price	quantity
1	MacBook Pro M4 14-in	Apple 14-in laptop	2499.99	60
2	MacBook Air M3	Apple laptop	1499.99	55
3	Mac Mini M3	Apple desktop	999.99	100
4	Mac Studio	Apple desktop	1999.99	15
6	iPhone 17 Pro	Apple iPhone	999.99	30
7	iPhone 17	Apple iPhone	799.99	30
8	iMac	Apple desktop	1299.99	30

```
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.01 sec)
```

This result set is helpful if we want to view every row in our table. However, needing only a subset of the data is more common. For instance, if we want to select rows where the price column equals a specific value, we can construct an SQL statement like this:

```
SELECT * FROM product WHERE price = 999.99;
```

Such a statement will result in the following output when executed on our product table:

```
+-----+-----+-----+-----+-----+
| id | name          | description | price | quantity |
+-----+-----+-----+-----+-----+
| 3  | Mac Mini M3   | Apple desktop| 999.99 | 100      |
| 6  | iPhone 17 Pro | Apple iPhone | 999.99 | 30       |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Now that we have looked at the equality operator (=), we can explore other WHERE clause operators.

13.3 Comparison Operators

So far, we have only reviewed the WHERE clause equality operator (=). Equality is just one of several comparison operators that can be used to refine data retrieval results. Figure 13-1 lists the comparison operators that are supported by the WHERE clause:

Operator	Description
=	Equal to
!=	Not equal to
<>	Not equal to
>	Greater than
<	Less than
<=	Less than or equal to
>=	Greater than or equal to
BETWEEN x AND y	Falls between the two values x and y

Figure 13-1

We have already used the equality (=) operator to check for numerical equality. This operator can also evaluate other data types, including comparing string values. For instance, if we want to find the price and product description of an iPhone 17 Pro, we would use the following SQL statement:

```
SELECT description, price FROM product WHERE name = "iPhone 17 Pro";
+-----+-----+
| description | price |
+-----+-----+
| Apple iPhone | 999.99 |
+-----+-----+
1 row in set (0.00 sec)
```

Additionally, we can list all products priced at or below a specified threshold using the less than (<=) operator:

```
SELECT name, price FROM product WHERE price <= 1299.99;
```

```
+-----+-----+
| name          | price  |
+-----+-----+
| Mac Mini M3   | 999.99 |
| iPhone 17 Pro | 999.99 |
| iPhone 17     | 799.99 |
| iMac          | 1299.99|
+-----+-----+
4 rows in set (0.01 sec)
```

We can also retrieve data rows where a value does not match specific criteria. The following example retrieves all rows where the price is not equal to \$999.99:

```
SELECT * FROM product WHERE price != 999.99;
+---+-----+-----+-----+-----+
| id | name          | description          | price  | quantity |
+---+-----+-----+-----+-----+
| 1  | MacBook Pro M4 14-in | Apple 14-in laptop  | 2499.99 | 60       |
| 2  | MacBook Air M3       | Apple laptop        | 1499.99 | 55       |
| 4  | Mac Studio           | Apple desktop       | 1999.99 | 15       |
| 7  | iPhone 17            | Apple iPhone        | 799.99  | 30       |
| 8  | iMac                 | Apple desktop       | 1299.99 | 30       |
+---+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

13.4 Checking for NULL values

In addition to checking the values of a column, it's also possible to identify columns that contain no value. Columns with no value are referred to as containing NULL values. Consequently, we can check for these empty columns by looking for equality with NULL. However, it's important to note that when checking for NULL values, we must use the IS keyword instead of the equals sign (=):

```
SELECT * FROM product WHERE name IS NULL;
```

The above SELECT statement will retrieve any rows which do not contain a product name.

13.5 Searching within value ranges

Figure 13-1 above included the BETWEEN...AND operator. This operator selects rows that fall within specified upper and lower limits. For example, we can list all products in our table priced between \$500 and \$1,400:

```
SELECT * FROM product WHERE price BETWEEN 500 AND 1400;
+---+-----+-----+-----+-----+
| id | name          | description          | price  | quantity |
+---+-----+-----+-----+-----+
| 3  | Mac Mini M3   | Apple desktop       | 999.99 | 100      |
| 6  | iPhone 17 Pro | Apple iPhone        | 999.99 | 30       |
| 7  | iPhone 17     | Apple iPhone        | 799.99 | 30       |
| 8  | iMac          | Apple desktop       | 1299.99 | 30       |
+---+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

13.6 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of the MySQL WHERE clause:

<https://www.answerstopia.com/3p3m>



13.7 Reference points

The main points covered in this chapter are as follows:

- **Purpose of the WHERE Clause**

- Filters rows in a result set based on specific criteria.
- Ensures only relevant data is retrieved from a table.
- WHERE clauses can be used with SELECT, UPDATE, INSERT, and DELETE statements.

- **Basic Syntax**

- `SELECT <columns> FROM <table> WHERE <column> = value` filters rows matching the given condition.

- **Comparison Operators**

- `=` Equal to
- `!=` or `<>` Not equal to
- `>` and `<` Greater than or less than
- `>=` and `<=` Greater than or equal to, less than or equal to
- `BETWEEN x AND y` Within a specified range.

- **Filtering Examples**

- Filter by exact matches, value ranges, or specific criteria (e.g., products priced below a certain amount).
- Use `!=` to exclude rows with specific values.

- **Handling NULL Values**

Use `IS NULL` or `IS NOT NULL` to filter rows based on the presence or absence of values in a column.

- **Range Searches**

Use `BETWEEN . . . AND` to retrieve rows with values falling within an upper and lower limit.

- **Practical Use Cases**

- Combine multiple operators for complex filters.
- Validate criteria with SELECT queries before applying further actions.

14. Filtering Results with Logical Operators

The previous chapter covered the basics of filtering retrieved data using comparison operators and the WHERE clause. While those fundamentals addressed simple filtering techniques, more advanced methods are often required. This chapter will explain using logical operators with the WHERE clause to define multiple conditions.

Although most examples in this chapter use the SELECT statement to demonstrate logical operators, these same techniques can also be applied to other statements, including INSERT, DELETE, and UPDATE.

14.1 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “Start Here” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql_logical_operators.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

14.2 Filtering results with the OR operator

In the previous chapter, we examined how to filter data retrieved using WHERE based on a single criterion. For instance, we retrieved all products in a table with prices below a specified threshold. Imagine you want to retrieve all rows from a table where the product meets either of two conditions. This can be accomplished using the WHERE clause with the OR logical operator.

For example, if we want to list all products in our sample database that cost less than \$1000 or more than \$2000, the SQL statement would read as follows:

```
SELECT name, price FROM product WHERE price < 1000 OR price > 2000;  
+-----+-----+  
| name                | price |  
+-----+-----+  
| MacBook Pro M4 14-in | 2499.99 |  
| Mac Mini M3          | 999.99 |  
| iPhone 17 Pro        | 999.99 |  
| iPhone 17            | 799.99 |  
+-----+-----+  
4 rows in set (0.00 sec)
```

The result set from the above SQL statement contains all products except those priced between \$1000 and \$2000.

14.3 Filtering results with the AND operator

The AND operator combines multiple filtering criteria in a WHERE clause. Unlike the OR operator, which allows for either condition to be true, the AND operator requires both to be true. Suppose, for example, we need to find an Apple iPhone that costs less than \$500. To do so, we can run the following SELECT statement:

```
SELECT name, price FROM product
WHERE description = 'Apple iPhone' AND price < 500;
Empty set (0.00 sec)
```

In this case, no matches are returned because no products in the table satisfy both criteria. If the customer is willing to spend more, we can adjust our search to find a suitable item:

```
SELECT name, price FROM product
WHERE description = "Apple iPhone" AND price < 800;
+-----+-----+
| name      | price |
+-----+-----+
| iPhone 17 | 799.99 |
+-----+-----+
1 row in set (0.00 sec)
```

This time, the query returns a result that meets both the product description and price criteria.

14.4 Combining AND and OR operators

The WHERE clause can combine multiple AND and OR operators to create complex filtering requirements. For example, we can combine operators to find a Mac Desktop or iPhone that costs \$999.99 using the following statement:

```
SELECT name, price FROM product WHERE price = 999.99
AND (description = 'Apple desktop' OR description = 'Apple iPhone');
+-----+-----+
| name            | price |
+-----+-----+
| Mac Mini M3     | 999.99 |
| iPhone 17 Pro   | 999.99 |
+-----+-----+
2 rows in set (0.01 sec)
```

14.5 Understanding operator precedence

When combining operators, it is essential to understand *operator precedence*, which dictates the order in which operators within the same statement are evaluated. You might have noticed the parentheses surrounding the OR expression in the previous example. To grasp the significance of these parentheses, try running the statement without them:

```
SELECT name, price FROM product WHERE price = 999.99
AND description = 'Apple desktop' OR description = 'Apple iPhone';
+-----+-----+
| name            | price |
+-----+-----+
| Mac Mini M3     | 999.99 |
| iPhone 17 Pro   | 999.99 |
| iPhone 17       | 799.99 |
+-----+-----+
3 rows in set (0.05 sec)
```

You may notice that the results now include a product that doesn't match the \$999.99 price point. This inconsistency occurs because MySQL evaluates AND expressions before OR expressions, regardless of the order in which they appear when reading the statement from left to right. As a result, the query retrieves all Mac Desktop devices priced at \$999.99, along with all Apple iPhone products, regardless of their price.

To adjust the operator precedence and ensure the OR expression was evaluated before the AND expression in the original statement, we used parentheses around the OR condition:

```
SELECT name, price FROM product WHERE price = 999.99
AND (description = 'Apple desktop' OR description = 'Apple iPhone');
```

14.6 Specifying condition ranges using the IN operator

The IN operator allows us to specify multiple filter criteria within a WHERE clause by listing them in parentheses, separated by commas. The syntax for using IN with the WHERE clause is as follows:

```
SELECT <columns> FROM <table> WHERE <column>
IN (<criteria_1>, <criteria_2>, ...);
```

For instance, suppose we want to retrieve all products from our database priced at \$799.99, \$999.99, or \$1999.99. While it is possible to achieve this using a series of OR expressions, such an approach can be lengthy and cumbersome. A more concise option is to use the IN clause as shown below:

```
SELECT name, price FROM product
WHERE price IN (799.99, 999.99, 1999.99);
```

name	price
Mac Mini M3	999.99
Mac Studio	1999.99
iPhone 17 Pro	999.99
iPhone 17	799.99

```
4 rows in set (0.04 sec)
```

14.7 Using the NOT Operator

The next operator to look at in this chapter is the NOT operator. The NOT operator is used to negate the result of an expression and is of particular use when using the IN operator. For example, we could very easily change our previous IN example so that it lists all the products in our table that do NOT cost \$799.99, \$999.99, or \$1999.99 using the following NOT IN operator combination:

```
SELECT name, price FROM product
WHERE price NOT IN (799.99, 999.99, 1999.99);
```

name	price
MacBook Pro M4 14-in	2499.99
MacBook Air M3	1499.99
iMac	1299.99

```
3 rows in set (0.00 sec)
```

14.8 Using subqueries with WHERE IN clauses

In the example above, we manually entered a list of pricing criteria. A valuable feature of the IN keyword is that the criteria can be generated using a SELECT statement with the following syntax:

```
SELECT <columns> FROM <table> WHERE <column>
IN (SELECT <column> FROM <table> WHERE ....);
```

To illustrate this technique, we will create a new table to track discontinued products. Use the following statement to create this new table:

```
CREATE TABLE availability
(
  id INT NOT NULL AUTO_INCREMENT,
  name char(30) NOT NULL,
  discontinued BOOL NOT NULL,
  PRIMARY KEY (id)
);
```

Next, populate the table with several rows, including a mix of available and discontinued products:

```
INSERT INTO availability
VALUES
  (NULL, 'Mac Mini M3', true),
  (NULL, 'Mac Studio', false),
  (NULL, 'iMac', true),
  (NULL, 'iPhone 17', true),
  (NULL, 'MacBook Air M3', false),
  (NULL, 'iPhone 17 Pro', true);
```

Run the following SELECT statement to check that the rows were inserted correctly:

```
SELECT * FROM availability;
+----+-----+-----+
| id | name          | discontinued |
+----+-----+-----+
| 1  | Mac Mini M3   | 1           |
| 2  | Mac Studio    | 0           |
| 3  | iMac          | 1           |
| 4  | iPhone 17     | 1           |
| 5  | MacBook Air M3 | 0           |
| 6  | iPhone 17 Pro | 1           |
+----+-----+-----+
6 rows in set (0.00 sec)
```

Now that we have a table containing product availability information, we can use this data to retrieve discontinued items from the product table as follows:

```
SELECT name, description FROM product
WHERE name IN (SELECT name FROM availability WHERE discontinued = true);
+-----+-----+
| name          | description |
+-----+-----+
| Mac Mini M3   | Apple desktop|
| iPhone 17 Pro | Apple iPhone |
| iPhone 17     | Apple iPhone |
+-----+-----+
```

```
| iMac          | Apple desktop|
+-----+-----+
4 rows in set (0.01 sec)
```

As previously discussed, WHERE clauses can be used not only with the SELECT statement. For instance, we will use the WHERE IN construct to delete discontinued items from the product table. However, since we will need these rows in later chapters, we will perform the deletion within a transaction, allowing us to roll it back afterward:

```
BEGIN;
DELETE FROM product WHERE name IN
(SELECT name FROM availability WHERE discontinued);
```

Execute a SELECT statement to verify that discontinued items have been removed from the product table:

```
SELECT name, description FROM product;
+-----+-----+
| name          | description    |
+-----+-----+
| MacBook Pro M4 14-in | Apple 14-in laptop |
| MacBook Air M3      | Apple laptop    |
| Mac Studio         | Apple desktop   |
+-----+-----+
3 rows in set (0.01 sec)
```

Finally, roll back the deletions to restore the table to its original state.

```
ROLLBACK;
```

Before proceeding, execute the following statement to delete the availability table:

```
DROP TABLE availability;
```

14.9 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL result set filtering:

<https://www.answerstopia.com/eq61>



14.10 Reference points

The main points covered in this chapter are as follows:

- **Logical Operators for Filtering**
 - OR: Retrieves rows that match at least one of the specified conditions.
 - AND: Retrieves rows that meet all specified conditions.
 - Combine AND and OR to create complex filters, using parentheses to define precedence.
 - WHERE clauses can be used with SELECT, UPDATE, INSERT, and DELETE statements.
- **Operator Precedence**

Filtering Results with Logical Operators

- `AND` takes precedence over `OR` in queries.
- Use parentheses to explicitly control the evaluation order for desired results.
- **IN Operator**
 - Simplifies filtering with multiple criteria by listing them in parentheses.
 - Can use subqueries within `IN` to generate dynamic filtering criteria.
- **Purpose of the NOT Operator**
 - Negates the result of an expression, filtering rows that do not meet the specified condition.
 - Useful for exclusion-based filtering.
- **Combining NOT with IN**
 - `NOT IN` excludes rows with values matching the list in the `IN` clause.
 - Example: Retrieve products with prices not in a specified range.
- **Subqueries in WHERE Clauses**
 - Subqueries can be used with `IN` to filter results based on criteria from another table.
 - Example: Identifying discontinued products using availability data from a related table.
- **Deleting with Filters**
 - `WHERE` clauses can filter rows to delete, ensuring only the intended records are removed.
 - Perform deletions within transactions to allow rollbacks if needed.

15. Wildcard and Regular Expression Matching

In the previous chapters, we discussed how to retrieve data from MySQL database tables based on specific criteria, such as matching exact strings or comparing values. While this method is effective in many situations, there are times when more flexible criteria are needed. In this chapter, we will explore how to retrieve data using patterns by employing wildcards, regular expressions, and the LIKE and REGEXP operators. These operators enable more dynamic searches, making finding data that partially matches your criteria easier.

15.1 What are wildcards?

Wildcards are used when matching string values. A typical comparison requires using specific strings where each character in the two strings must exactly match before they are considered equal. Wildcards, however, provide more flexibility by allowing any character or group of characters in a string to be acceptable as a match for another string. As with most concepts, this is best demonstrated through examples.

15.2 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql_regex.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

Next, execute the following statement to add two rows to the product table:

```
INSERT INTO product  
VALUES (  
    NULL,  
    'NZXT Mid Tower',  
    'NZXT PC case (grey)',  
    129.99,  
    100  
) ,  
(  
    NULL,  
    'Corsair Full Tower',  
    'Corsair PC case (gray)',  
    179.99,  
    15  
) ,  
(
```

```

NULL,
'Dell XPS Model 4823',
'Dell Desktop PC',
299.99,
35
),
(
NULL,
'Dell XPS Model 7823',
'Dell Desktop PC',
749.99,
16
);

```

15.3 Single character wildcards

Suppose our product table is part of an online commerce site selling electronic items to customers worldwide. One problem, even between English-speaking nations, is a difference in spelling. For example, a particular color would be spelled “gray” in the United States, while the same color is spelled “grey” in the United Kingdom. If we have items in our product database that contain both spellings, we would, under normal circumstances, need to perform a search for each spelling to retrieve the required data. For example, we might choose to use the IN clause:

```

SELECT name, description FROM product WHERE description
IN ('NZXT PC case (grey)', 'Corsair PC case (gray)');
+-----+-----+
| name          | description          |
+-----+-----+
| NZXT Mid Tower | NZXT PC case (grey)  |
| Corsair Full Tower | Corsair PC case (gray) |
+-----+-----+
2 rows in set (0.01 sec)

```

A more straightforward and flexible approach is to use a *character wildcard* instead of the ‘e’ and ‘a’ characters in the comparison value. This can be done using the underscore wildcard (_) and the LIKE operator. To use this wildcard, insert an underscore in character positions where multiple matches are acceptable.

For example, in the following SELECT statement, any character that falls between the ‘Gr’ and ‘y’ will be considered a match. Try the following statement:

```

SELECT name, description FROM product
WHERE description LIKE 'gr_y';
Empty set (0.00 sec)

```

The empty result set occurs because, although the statement uses a character wildcard, the LIKE clause requires a complete match. While we can successfully search for “NZXT PC case (gr_y)” or “Corsair PC case (gr_y),” we need to employ multiple character wildcards to find descriptions that include “gray” or “grey” in addition to other text.

15.4 Multiple character wildcards

The single-character wildcard (_) is often useful, but the multiple-character percent sign (%) wildcard is even more powerful. The percent sign can represent any number of characters in a value match. When you use wildcards, not only do the characters not have to match, but the number of characters can also vary.

You can place wildcards in any number of positions within a string value. For example, you can place a wildcard

at the beginning of the comparison value to find all rows where the product name ends with “Tower,” as follows:

```
SELECT name, description FROM product WHERE name LIKE '%Tower';
```

name	description
NZXT Mid Tower	NZXT PC case (grey)
Corsair Full Tower	Corsair PC case (gray)

2 rows in set (0.00 sec)

Alternatively, the wildcard could be placed at the end of a value, for example, to retrieve all Mac devices:

```
SELECT name, description FROM product WHERE name LIKE 'Mac%';
```

name	description
MacBook Pro M4 14-in	Apple laptop
MacBook Air M3	Apple laptop
Mac Mini M3	Apple desktop
Mac Studio	Apple desktop

4 rows in set (0.00 sec)

Multiple wildcards can be used to retrieve all the rows in our product table where the name contains ‘Book’ and ‘Air’ (in that order):

```
SELECT name, description FROM product WHERE name LIKE '%Book%Air%';
```

name	description
MacBook Air M3	Apple laptop

1 row in set (0.00 sec)

Finally, the following statement combines single and multiple-character wildcards to retrieve product descriptions that contain either “gray” or “grey”:

```
SELECT name, description FROM product WHERE description LIKE '%gr_y%';
```

name	description
NZXT Mid Tower	NZXT PC case (grey)
Corsair Full Tower	Corsair PC case (gray)

2 rows in set (0.00 sec)

15.5 What are regular expressions?

The term “regular expressions” can be confusing, as it doesn’t clearly convey their purpose. Regular expressions, often referred to as *regex*, are a feature found in many programming languages, and entire books have been dedicated to the topic. Luckily, MySQL offers more limited support for regular expressions compared to other languages, which can simplify the learning process. However, this may also be frustrating for those familiar with the more complex regular expressions in other programming languages.

Regular expressions are instructions written in a predefined syntax, allowing flexible text matching. For instance,

Wildcard and Regular Expression Matching

they enable us to extract all occurrences of a specific word or phrase from a text block. Additionally, you can search for a particular piece of text and replace it with an alternate version.

15.6 Regular expression character matching

To introduce the REGEXP operator, we will begin by exploring regular expressions that are similar to those used with the LIKE operator. As mentioned in a previous example, we need to retrieve rows from a table while considering the different spellings of the color “gray” (and “grey”).

To perform this query, we will use the regex dot character (.) for matching. This character functions similarly to the LIKE underscore wildcard, indicating that any character in that position within the text will be considered a match. For example:

```
SELECT name, description FROM product WHERE description REGEXP 'gr.y';
```

name	description
NZXT Mid Tower	NZXT PC case (grey)
Corsair Full Tower	Corsair PC case (gray)

2 rows in set (0.00 sec)

So far, we haven’t done anything that couldn’t be accomplished using wildcards. However, regular expressions offer capabilities that extend beyond this basic functionality.

15.7 Matching with a group of characters

One limitation of the previous approach is that any letter between ‘gr’ and ‘y’ would be considered a match. In reality, we are only interested in words that contain either an ‘a’ or an ‘e’ in that position. Regular expressions provide a way to specify a group of acceptable characters for any given position in the match. This is done using square brackets around the permissible characters. For example:

```
SELECT name, description FROM product WHERE description REGEXP 'gr[ae]y';
```

This syntax ensures that only the words ‘grey’ and ‘gray’ match the search criteria. There is no limit to the number of characters included within the brackets. For instance:

```
SELECT * FROM product WHERE description REGEXP 'gr[aeiou]y';
```

15.8 Matching character ranges

The syntax for character group matching can be extended to include a range of characters. For instance, instead of specifying a regular expression that covers the letters A through F as [ABCDEF], we can use a hyphen (-) to indicate a range, writing it as [A-F]. Similarly, we can use the range syntax to list products based on model numbers that begin with digits from 1 to 6, for example:

```
SELECT name, description FROM product
WHERE name REGEXP 'Model [1-6]823';
```

name	description
Dell XPS Model 4823	Dell Desktop PC

1 row in set (0.00 sec)

15.9 Handling special characters

Regular expressions give special significance to certain characters. For instance, the dot (.) and square brackets ([]) each have specific meanings. You might be wondering how to handle character sequences that include one or more special characters in your search. Obviously, if you are looking for text that looks like a regular expression, the text you are searching for is, itself, going to be viewed as regular expression syntax.

To address this issue, a concept known as *escaping* is used. In SQL, escaping involves preceding any characters that would be misinterpreted as a regular expression special character with double backslashes (\\).

Let's start by adding a row to our product table and including square brackets in the name field:

```
INSERT INTO product
VALUES (
    NULL,
    'Dell Dock Model [4807]',
    'Dell docking station',
    299.99,
    35
);
```

If we search for this row without regard to the fact that the name value contains special characters, we don't get the results we expect:

```
SELECT name, description FROM product
WHERE name REGEXP 'Model [4807]';
```

name	description
Dell XPS Model 4823	Dell Desktop PC
Dell XPS Model 7823	Dell Desktop PC

```
2 rows in set (0.00 sec)
```

The issue arises because the regular expression is incorrectly interpreted as a search for entries containing "Model" followed by either the numbers 4, 8, 0, or 7. What we actually intended to find was the text "[4807]." To achieve the desired result, we need to escape the brackets using the \\ escape sequence, as follows:

```
SELECT name, description FROM product
WHERE name REGEXP 'Model \\[4807\\]';
```

name	description
Dell Dock Model [4807]	Dell docking station

```
1 row in set (0.00 sec)
```

15.10 Whitespace metacharacters

Regular expression syntax includes a way to reference whitespace characters such as tabs, carriage returns, and line feeds. These characters are represented in regular expressions using specific *metacharacters*. Figure 15-1 outlines the whitespace metacharacters supported by MySQL:

Metacharacter	Description
\\n	New line (line feed)
\\f	Form feed
\\t	Tab
\\r	Carriage return
\\v	Vertical tab

Figure 15-1

For instance, if we want to search for text that contains a tab character using a regular expression, we would do so as follows:

```
SELECT * from product where name REGEXP 'Microsoft Mouse\\tWireless';
```

15.11 Matching by character type

Another valuable technique in regular expressions is matching characters by their type or class. For instance, you may need to specify that a character must be a letter, a number, or an alphanumeric character. This can be achieved using the special class definitions outlined in the table below:

Class Keyword	Description of Matches
[:alnum:]	Alphanumeric - any number or letter. Equivalent to [a-z], [A-Z] and [0-9]
[:alpha:]	Alpha - any letter. Equivalent to [a-z] and [A-Z]
[:blank:]	Space or Tab. Equivalent to [\\t] and []
[:cntrl:]	ASCII Control Character
[:digit:]	Numeric. Equivalent to [0-9]
[:graph:]	Any character with the exception of space
[:lower:]	Lower case letters. Equivalent to [a-z]
[:print:]	Any printable character
[:punct:]	Characters that are neither control characters, nor alphanumeric (i.e., punctuation characters)
[:space:]	Any whitespace character (tab, new line, form feed, space etc)
[:upper:]	Upper case letters. Equivalent to [A-Z]
[:xdigit:]	Any hexadecimal digit. Equivalent to [A-F], [a-f] and [0-9]

Figure 15-2

Before we try some type-matching examples, we need to add a few more rows to our product table using the following INSERT statement:

```
INSERT INTO product
VALUES (
    NULL,
    'One2One USB-C Hub',
    '2-port USB hub',
    29.99,
    60
),
(
```

```

        NULL,
        'One4One USB-C Hub',
        'Two 2-port USB hubs',
        29.99,
        60
    ),
    (
        NULL,
        'One&One Touch 200',
        'Touch sensitive touchpad',
        29.99,
        60
    );

```

Suppose we want to retrieve a list of USB hub devices from the product table. At first glance, it looks like this might be possible using a single-character match expression:

```

SELECT name, description FROM product
WHERE name REGEXP 'One.One';
+-----+-----+
| name                | description          |
+-----+-----+
| One2One USB-C Hub  | 2-port USB hub      |
| One4One USB-C Hub  | Two 2-port USB hubs |
| One&One Touch 200  | Touch sensitive touchpad |
+-----+-----+
3 rows in set (0.00 sec)

```

As we can see from the results, the search also returned the One&One touchpad. To limit the search to numerical characters in the specified position, we can use the `[:digit:]` character class as follows:

```

SELECT name, description FROM product
WHERE name REGEXP 'One[:digit:]One';
+-----+-----+
| name                | description          |
+-----+-----+
| One2One USB-C Hub  | 2-port USB hub      |
| One4One USB-C Hub  | Two 2-port USB hubs |
+-----+-----+
2 rows in set (0.00 sec)

```

Similarly, we can use the `[:punct:]` class to restrict the results to punctuation characters (which includes the ampersand character):

```

SELECT name, description FROM product
WHERE name REGEXP 'One[:punct:]One';
+-----+-----+
| name                | description          |
+-----+-----+
| One&One Touch 200  | Touch sensitive touchpad |
+-----+-----+
1 row in set (0.00 sec)

```

15.12 Repetition metacharacters

In addition to enabling searches on individual instances, regular expressions can also be written to identify repetition in text using the repetition metacharacters listed in Figure 15-3:

Metacharacter	Description
*	Any number of matches
+	One or more matches
{n}	n number of matches
{n,}	Not less than n number of matches
{n1,n2}	A range of matches between n1 and n2
?	Optional single character match (character may be present or not to qualify as match)

Figure 15-3

Examples illustrate concepts more effectively than tabular lists. Let’s begin by searching for all 3-digit sequences in the name column:

```
SELECT name, description FROM product
WHERE name REGEXP '[:digit:]{3}';
```

name	description
Dell XPS Model 4823	Dell Desktop PC
Dell XPS Model 7823	Dell Desktop PC
Dell Dock Model [4807]	Dell docking station
One&One Touch 200	Touch sensitive touch pad

4 rows in set (0.00 sec)

In the above example, we have indicated that we are looking for any sequence of 3 digits by using the `[:digit:]{3}` regular expression. In this case, we picked up entries with three and four-digit sequences. If we want to limit the result set to four digits, we can use `{4,}` to exclude sequences with less than four digits:

```
SELECT name, description FROM product
WHERE name REGEXP '[:digit:]{4,}';
```

name	description
Dell XPS Model 4823	Dell Desktop PC
Dell XPS Model 7823	Dell Desktop PC
Dell Dock Model [4807]	Dell docking station

3 rows in set (0.00 sec)

Our One&One Touch 200 is no longer listed because it only contains a three-digit sequence.

The ‘?’ metacharacter is particularly useful when we need to allow for plural words. For example, we may want to list product descriptions where the word “hub” or “hubs” is used. To achieve this, we follow the ‘s’ with a ‘?’, thereby making the trailing ‘s’ optional for a match:

```
SELECT name, description FROM product WHERE name REGEXP 'hubs?';
```

name	description
------	-------------

```
+-----+-----+
| One2One USB-C Hub | Two port USB hub |
| One4One USB-C Hub | Two 2-port USB hubs |
+-----+-----+
2 rows in set (0.01 sec)
```

15.13 Matching by text position

The next area of regular expressions to cover in this chapter involves matching based on the location of text in a string. For example, we may want to find a match that requires a word to appear at the beginning or end of a text string. Once again, this requires the use of some special metacharacters:

Metacharacter	Description
<code>^</code>	Beginning of text
<code>\$</code>	End of text
<code>\\b</code>	Word boundary

Figure 15-4

For instance, we can look for strings that start with a digit:

```
SELECT name, description FROM product
WHERE description REGEXP '^[[:digit:]]';
+-----+-----+
| name          | description |
+-----+-----+
| One2One USB-C Hub | 2-port USB hub |
+-----+-----+
1 row in set (0.00 sec)
```

We can also stipulate that character sequences are separate words within a string. For example, if we search for product names containing “air” we get the following result set:

```
SELECT name FROM product WHERE name REGEXP 'air';
+-----+
| name          |
+-----+
| MacBook Air M3 |
| Corsair Full Tower |
+-----+
2 rows in set (0.01 sec)
```

As we can see from the above example, because the word “Corsair” contains “air,” we have retrieved more rows than we anticipated. To focus specifically on instances where “air” appears as a standalone word, we can use the word boundary metacharacter (`\\b`):

```
SELECT name FROM product WHERE name REGEXP '\\bair\\b';
+-----+
| name          |
+-----+
| MacBook Air M3 |
+-----+
1 row in set (0.01 sec)
```

We can also use the end of text character to select names ending with a specific sequence of characters:

```
SELECT name FROM product WHERE name REGEXP 'hub$';
+-----+
| name                |
+-----+
| One2One USB-C Hub   |
| One4One USB-C Hub   |
+-----+
2 rows in set (0.00 sec)
```

15.14 Replacing text with regular expressions

So far, in this chapter, we have focused on using regular expressions to refine search criteria using the REGEXP operator. Regular expressions can also replace occurrences of text patterns within string values. Replacements are performed using the REGEXP_REPLACE operator, the basic syntax for which is as follows:

```
REGEXP_REPLACE(string, pattern, repl, position, occurrence, match_type);
```

The parameters accepted by REGEXP_REPLACE are as follows:

- **string** - The input string where the replacement will occur. This can be a string, or a table column that contains a string value.
- **pattern**: The regular expression pattern to search for.
- **replacement**: The replacement string.
- **position**: The position in the string to start the search. Default is 1 (the first character).
- **occurrence** (optional): Specifies which occurrence of the pattern to replace. Default is 0, which means all occurrences.
- **match_type** (optional): Modifiers that change the behavior of the regular expression, such as case sensitivity or character class handling.

The following statement, for example, replaces all occurrences of “dog” with “cat” in the specified string:

```
SELECT REGEXP_REPLACE('The quick brown dog','dog', 'cat');
+-----+
| REGEXP_REPLACE('The quick brown dog','dog', 'cat') |
+-----+
| The quick brown cat                                |
+-----+
1 row in set (0.00 sec)
```

Similarly, we can provide an occurrence parameter to change the only the second instance of “dog” in a string:

```
SELECT REGEXP_REPLACE(
  'The quick dog chased the slow dog','dog', 'cat', 1, 2);
+-----+
| REGEXP_REPLACE('The quick dog chased the slow dog','dog', 'cat', 1, 2) |
+-----+
| The quick dog chased the slow cat                                         |
+-----+
1 row in set (0.01 sec)
```

As another example, we can provide a match type parameter to make the search and replacement case-insensitive:

```
SELECT REGEXP_REPLACE(
```

```
'The quick dog chased the slow dog','DOG', 'cat', 1, 0, 'i');
+-----+
| REGEXP_REPLACE('The quick dog chased the slow dog','DOG', 'cat', 1, 0, 'i') |
+-----+
| The quick cat chased the slow cat |
+-----+
1 row in set (0.01 sec)
```

In addition to directly manipulating strings, we can use regular expression replacement to update values in table columns. The following statement, for example, changes all occurrences of “USB-C” to “USB-A” in the product name column:

```
UPDATE product SET name = REGEXP_REPLACE(name, 'USB-C', 'USB-A');
Query OK, 2 rows affected (0.00 sec)
Rows matched: 15  Changed: 2  Warnings: 0
```

After executing the statement, check that “USB-C” has been changed to “USB-A”:

```
SELECT name FROM product WHERE name LIKE '%USB%';
+-----+
| name |
+-----+
| One2One USB-A Hub |
| One4One USB-A Hub |
+-----+
2 rows in set (0.00 sec)
```

15.15 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL regular expressions:

<https://www.answerstopia.com/dd9v>



15.16 Reference points

The main points covered in this chapter are as follows:

- **Introduction to Wildcards**
 - Wildcards enable flexible string matching in SQL queries.
 - Useful for finding patterns rather than exact matches.
- **Single-Character Wildcard (_)**
 - Matches any single character at a specific position.
- **Multiple-Character Wildcard (%)**
 - Matches any sequence of characters, including none.
- **Regular Expressions (REGEXP)**
 - REGEXP allows for advanced pattern matching beyond wildcards.

Wildcard and Regular Expression Matching

- Offers greater flexibility for text searches and transformations.
- **Character Matching in REGEXP**
 - Regex dot (.) character similar to underscore (_) in LIKE.
 - Use [aefg...] to match specific characters, e.g., “gray” or “grey.”
- **Character Ranges in REGEXP**
 - Use [a-z] or [1-6] to define ranges for matches.
- **Escaping Special Characters**
 - Use \ to escape special characters like '[' and ']' in text searches.
- **Whitespace Matching with Metacharacters**
 - Match whitespace characters like tabs (\t) or newlines (\n).
- **Class-Based Matching in REGEXP**
 - Use character classes like [[:digit:]] or [[:alpha:]] to match types of characters.
- **Repetition Matching**
 - Use repetition metacharacters like *, +, or {n} for matches.
- **Position-Based Matching**
 - Use ^ to match text at the beginning, \$ for the end, and \b for word boundaries.
- **Combining Wildcards and REGEXP**
 - Wildcards are simpler for basic matches; REGEXP is better for complex patterns.
- **String Replacement (REGEXP_REPLACE)**
 - Enables dynamic text replacement in strings or table columns based on regular expression pattern.
 - Provides flexibility in pattern matching and replacements.
- **Basic Syntax of REGEXP_REPLACE**
 - `REGEXP_REPLACE(string, pattern, repl, position, occurrence, type);`
 - Defaults: Position starts at 1, occurrence is 0 (all matches), and match type is case-sensitive unless modified.

16. Understanding Joins and Unions

Up to this point, we have focused on creating a table that contains information about a single category—specifically, the products sold by an online electronics store. However, we would also need to store information about each product’s supplier in a real-world scenario. Supplier data might include the supplier’s name, address, and telephone number. We would also need to link these suppliers to the products they provide.

In this scenario, we have two options for storing contact information about the supplier. One option is to include the supplier’s contact details in each row of the product table for every product sourced from that supplier. While this approach would be functional, it is highly inefficient, as it requires duplicating the supplier’s contact information for every product they sell to us. Additionally, if the supplier relocates, we would have to update every row in the product table related to that supplier.

A more effective approach would be to create a separate supplier table that includes the contact information for each supplier. We can then reference this table when we want to retrieve supplier information for a specific product in the product table or list products by supplier. These connections between tables are known as a *joins* and are the foundation of relational databases.

16.1 How joins work

A join operates by leveraging the relationships between keys in different tables. For instance, in the above example, the supplier table includes a column called `id`, configured as the primary key, and the company name and address, as illustrated in Figure 16-1. (For more details on primary keys, see the chapter on “*The Basics of Databases*”).

Suppliers	
PK	<u>id int NOT NULL</u>
	company varchar(25) NOT NULL
	address varchar(40) NOT NULL

Figure 16-1

The product table, however, stores information about our products, such as product ID, name, and description. Additionally, it contains a `supplier_id` column, which specifies the supplier from which each product is sourced. Since this `supplier_id` corresponds to a key in another table (the `id` key in the supplier table), it is known as a *foreign key*.

Products	
PK	<u>id int NOT NULL</u>
	name varchar(25) description varchar(50) price decimal(6,2) quantity int
FK	supplier_id int

Figure 16-2

When writing a SELECT statement to retrieve data from the product table, we can use this foreign key to link it to the supplier table, extracting supplier information for each product. This approach allows us to combine data from both tables seamlessly, making it easier to analyze the relationships between products and their suppliers:

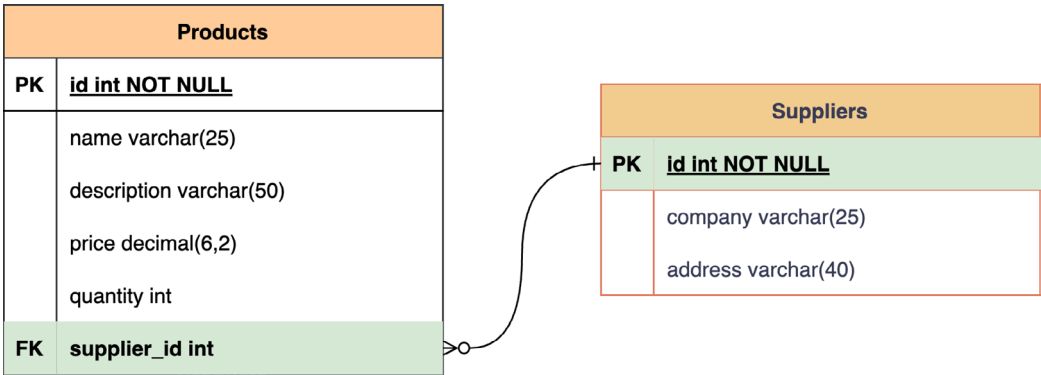


Figure 16-3

16.2 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “Start Here” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampledb;"
mysql -u root -p sampledb < mysql_joins.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampledb
```

16.3 Creating the supplier table

Before exploring table joins in MySQL, we need to add a table to the *sampledb* database to contain supplier information. Using the MySQL client, execute the following statement to create the supplier table:

```
CREATE TABLE supplier
(
  id int NOT NULL AUTO_INCREMENT,
  company Varchar(25) NOT NULL,
  address VARCHAR(40) NULL,
  PRIMARY KEY (id)
);
```

After the supplier table has been created, use the following INSERT statement to add some supplier records:

```
INSERT INTO supplier VALUES (
  NULL, 'Apple', 'Cupertino, CA'
),
(
  NULL, 'Dell', 'Round Rock, TX'
),
(
  NULL, 'NZXT', 'City of Industry, CA'
),
(
  NULL, 'Corsair', 'Milpitas, CA'
);
```

16.4 Adding the foreign key to the product table

Our product table currently lacks a column to store the supplier ID reference, so use the following ALTER TABLE statement to add a column named `supplier_id`:

```
ALTER TABLE product ADD supplier_id INT;
```

Though we have added the `supplier_id` column, we still need to let MySQL know that it contains foreign keys. We do this using the `FOREIGN KEY` and `REFERENCES` keywords. These keywords enable us to define the new column as a foreign key and specify the corresponding table and column it refers to, thereby establishing a relationship between the product and supplier tables:

```
ALTER TABLE product
ADD FOREIGN KEY(supplier_id)
REFERENCES supplier(id);
```

16.5 Adding key values to supplier_id

The next step is to populate the `supplier_id` field of each product table row with the corresponding ID from the supplier table. Although the product row count is low enough that we could do this manually, the process would need to be automated for larger datasets. In this case, we use an `UPDATE SET` statement with the `WHERE` and `LIKE` operators to compare the first word of the `product.description` field (which consistently contains the manufacturer name) with the `supplier.company` column as follows:

```
UPDATE product
SET supplier_id = (SELECT id FROM supplier
WHERE product.description LIKE CONCAT(supplier.company, '%'));
```

In the statement above, we used the `CONCAT()` function to combine multiple strings into one string. In this case, the current company name is followed by the `'%'` wildcard. After the update, check that products with matching suppliers have been assigned the correct supplier id:

```
SELECT description, supplier_id FROM product;
```

description	supplier_id
Apple laptop	1
Apple laptop	1
Apple desktop	1
Apple desktop	1
Apple iPhone	1
Apple iPhone	1
Apple desktop	1
NZXT PC case (gray)	3
Corsair PC case (grey)	4
Dell desktop	2
Dell desktop	2
Dell docking station	2
One2One touchpad	NULL
One2One 2-port USB hub	NULL
One2One 4 port USB hub	NULL

15 rows in set (0.00 sec)

With the table updates completed, we can begin exploring table joins.

16.6 Performing a cross join

When you join two tables using the CROSS JOIN clause, the result set will consist of all possible combinations of rows from both tables. Each row from the first table is paired with every row from the second table. The diagram in Figure 16-4 illustrates this concept:

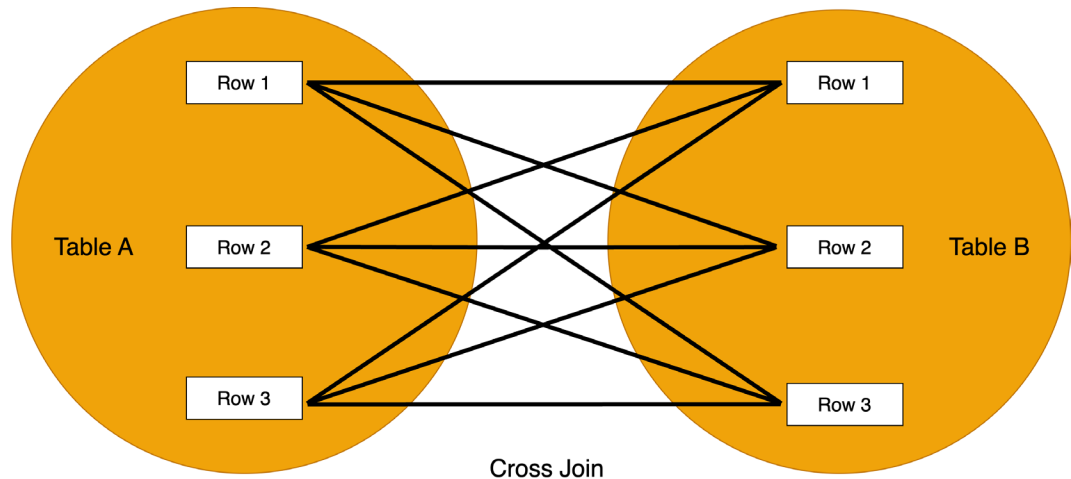


Figure 16-4

The SQL syntax for cross joins is as follows:

```
SELECT <column_names> FROM <table_1> CROSS JOIN <table_2>;
```

A cross join between our product and supplier tables will generate the following output (abbreviated for conciseness):

```
SELECT name, supplier.id from product CROSS JOIN supplier;
```

```

| name | id |
+-----+----+
| MacBook Pro M4 14-in | 4 |
| MacBook Pro M4 14-in | 3 |
| MacBook Pro M4 14-in | 2 |
| MacBook Pro M4 14-in | 1 |
| MacBook Air M3 | 4 |
| MacBook Air M3 | 3 |
| MacBook Air M3 | 2 |
| MacBook Air M3 | 1 |
| Mac Mini M3 | 4 |
.
.
| One2One USB-A Hub | 2 |
| One2One USB-A Hub | 1 |
| One4One USB-A Hub | 4 |
| One4One USB-A Hub | 3 |
| One4One USB-A Hub | 2 |
| One4One USB-A Hub | 1 |
+-----+----+
60 rows in set (0.01 sec)

```

Cross joins include all row combinations, including those for which tables are not matched. For example, the USB hub rows are listed in the above result set, even though no corresponding supplier exists for those products.

Note that we have to use what is known as the *fully qualified name* for the supplier id column since both tables contain an id column. A fully qualified column name is defined by specifying the table name followed by a dot (.) and then the column name (i.e., supplier.id).

16.7 Performing an inner join

The inner join combines rows from two or more tables based on comparisons between a specific column in each table, as illustrated in Figure 16-5:

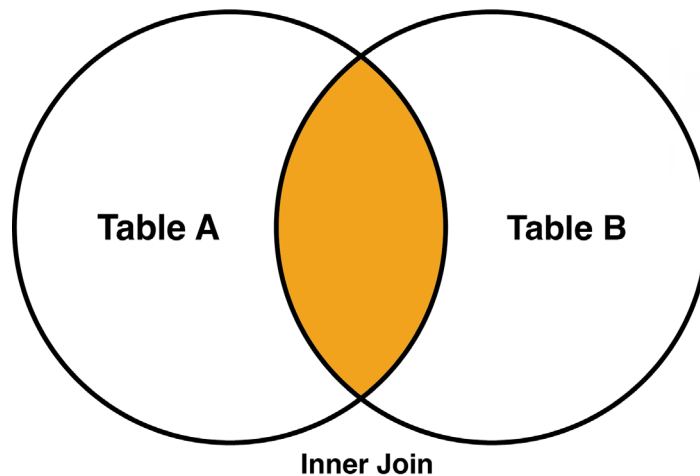


Figure 16-5

We could, for example, use an inner join to select rows from one or more tables that share a common customer phone number. Alternatively, a join could be established to retrieve rows where one table's price column value is less than another's. Inner joins use the following syntax:

Understanding Joins and Unions

```
SELECT <column_names>
FROM <table_1>
  INNER JOIN <table_2>
    ON <column_1> <comparison_operator> <column_2>
  INNER JOIN <table_3>
    ON <column_3> <comparison_operator> <column_4> .. ;
```

The following statement, for example, retrieves the product name and supplier company from rows where the product.supplier_id and supplier.id columns match:

```
SELECT product.name, supplier.company
FROM product
INNER JOIN supplier
ON product.supplier_id = supplier.id;
```

```
+-----+-----+
| name                | company |
+-----+-----+
| MacBook Pro M4 14-in | Apple   |
| MacBook Air M3       | Apple   |
| Mac Mini M3          | Apple   |
| Mac Studio           | Apple   |
| iPhone 17 Pro        | Apple   |
| iPhone 17            | Apple   |
| iMac                 | Apple   |
| Dell XPS Model 4823   | Dell    |
| Dell XPS Model 7823   | Dell    |
| Dell Dock Model [4807] | Dell    |
| NZXT Mid Tower        | NZXT    |
| Corsair Full Tower    | Corsair |
+-----+-----+
12 rows in set (0.00 sec)
```

16.8 Performing left joins

A left join retrieves all records from the left table and the matching records from the right table. If no match is found in the right table, the corresponding columns in the result set will contain NULL values, guaranteeing that all rows from the left table are included, regardless of whether a match exists in the right table:

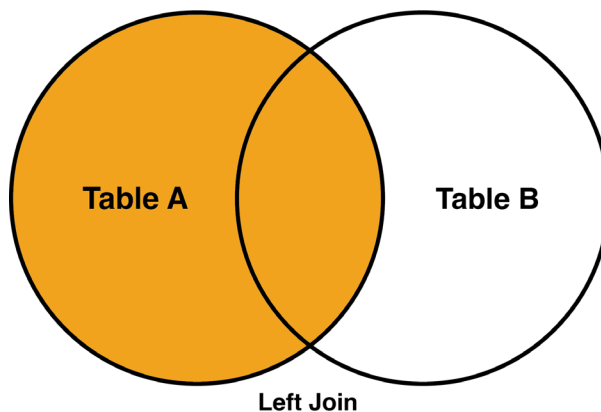


Figure 16-6

Left joins are created using the following syntax:

```
SELECT <column_names>
FROM <table_1>
LEFT JOIN <table_2>
ON <column_1> <comparison_operator> <column_2>;
```

One key difference with LEFT JOIN is that it will include rows from the left table for which there is no match in the right table. For example, suppose we have rows in our product table for which there is no matching supplier in the supplier table. For example, when we run the following SELECT statement, the rows will still be displayed, but with NULL values for the supplier columns where no supplier exists:

```
SELECT product.name, supplier.company
FROM product
LEFT JOIN supplier
ON product.supplier_id = supplier.id;
```

name	company
MacBook Pro M4 14-in	Apple
MacBook Air M3	Apple
Mac Mini M3	Apple
Mac Studio	Apple
.	.
Dell Dock Model [4807]	Dell
One&One Touch 200	NULL
One2One USB-A Hub	NULL
One4One USB-A Hub	NULL

15 rows in set (0.00 sec)

16.9 Performing right joins

As the name suggests, a right join achieves the opposite result of a left join. With a right join, all the rows from the second table (in this case, the supplier table) are included in the result set, regardless of any matching entries in the product table. If a supplier does not have any associated products in the product table, the corresponding columns from the product table will contain NULL values:

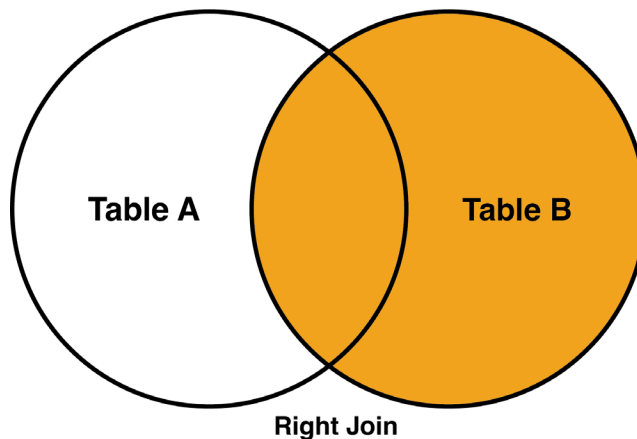


Figure 16-7

Right joins are formed using the following syntax:

Understanding Joins and Unions

```
SELECT <column_names>
FROM <table_1>
RIGHT JOIN <table_2>
ON <column_1> <comparison_operator> <column_2>;
```

16.10 Understanding WHERE in join statements

The WHERE keyword allows us to narrow the results of table joins based on specific filtering criteria. Say, for example, that we want to list only products supplied by Dell. To do so, we would create a right join using the following statement:

```
SELECT product.name, supplier.company
FROM product
RIGHT JOIN supplier
ON product.supplier_id = supplier.id
WHERE company = 'Dell';
```

name	company
Dell XPS Model 4823	Dell
Dell XPS Model 7823	Dell
Dell Dock Model [4807]	Dell

3 rows in set (0.00 sec)

16.11 Working with unions

Although it is not considered a join, the UNION operator is a valuable tool for combining the results of multiple tables. As we will see later, it can also merge join statements. A UNION combines the result sets from two or more SELECT queries and follows this syntax:

```
SELECT <column_names> FROM <table_1>
UNION
SELECT <column_names> FROM <table_2>;
UNION
SELECT ... ;
```

When using the UNION operator, there are several rules to follow:

- Each SELECT statement must return the same number of columns.
- The data types of the columns in the corresponding positions of each SELECT statement must be compatible. For example, you cannot mix INT and VARCHAR types in the same column position.
- The column names in each SELECT statement must be presented in the same order.

By default, the UNION operator will eliminate duplicate rows from the combined result set. If you want to keep duplicate rows, you can use the UNION ALL operator instead:

```
SELECT <column_names> FROM <table_1>
UNION ALL
SELECT <column_names> FROM <table_2> ... ;
```

To demonstrate unions, we will add a second supplier table to our sample database containing information about European suppliers. We will then create a union to retrieve the supplier addresses from both tables. This scenario is illustrated in Figure 16-8:

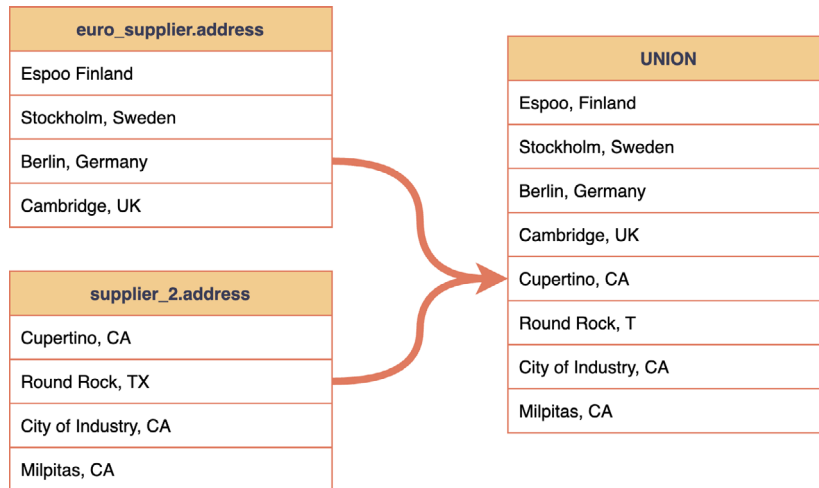


Figure 16-8

Start by adding the European supplier table to the sampled database:

```
CREATE TABLE euro_supplier
(
  id int NOT NULL AUTO_INCREMENT,
  company Varchar(25) NOT NULL,
  address VARCHAR(40) NULL,
  PRIMARY KEY (id)
);
```

Next, insert four supplier rows into the euro_supplier table:

```
INSERT INTO euro_supplier VALUES (
  NULL, 'Nokia', 'Espoo, Finland'
),
(
  NULL, 'Ericsson', 'Stockholm, Sweden'
),
(
  NULL, 'Siemens', 'Berlin, Germany'
),
(
  NULL, 'ARM', 'Cambridge, UK'
);
```

Now that we have two supplier tables, we can use the SELECT statement and the UNION operator to retrieve supplier addresses from both tables:

```
SELECT address FROM euro_supplier
UNION SELECT address FROM supplier;
+-----+
| address |
+-----+
| Espoo, Finland |
| Stockholm, Sweden |
| Berlin, Germany |
```

```
| Cambridge, UK      |
| Cupertino, CA      |
| Round Rock, TX     |
| City of Industry, CA |
| Milpitas, CA       |
+-----+
8 rows in set (0.08 sec)
```

16.12 Full outer joins in MySQL

A full outer join, also known as a full join, combines the results of both left and right joins. It returns all rows from both tables being joined, even if there are no matching rows. The diagram below illustrates this concept:

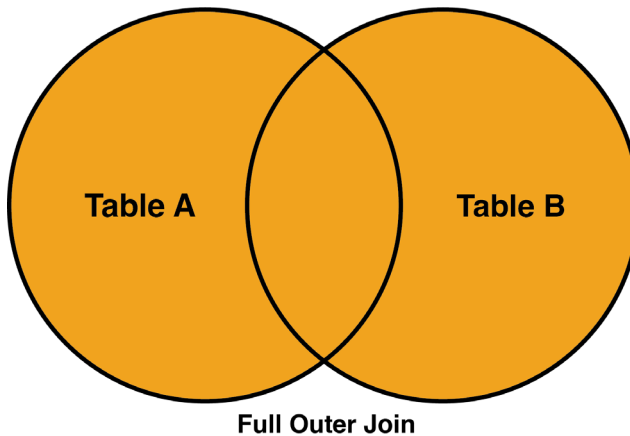


Figure 16-9

Unlike some other SQL implementations, MySQL does not support FULL OUTER JOIN or FULL JOIN clauses. However, we can achieve the same results by creating a union of a left join and a right join using the following syntax:

```
SELECT <column_names> FROM <table_1>
LEFT JOIN <table_2> ON <column_1> <comparison_operator> <column_2>
UNION
SELECT <column_names> FROM <table_1>
RIGHT JOIN <table_2> ON <column_1> <comparison_operator> <column_2>;
```

In previous examples, we have observed that some items in the product table do not have corresponding suppliers. To illustrate the impact of missing matches in both tables, we will also add a company to the supplier table for which there are no entries in the product table:

```
INSERT INTO supplier VALUES (NULL, 'Nvidia', 'Santa Clara, CA');
Query OK, 1 row affected (0.03 sec)
```

We can now use a full outer join to list products and corresponding supplier names using the following statement:

```
SELECT product.name, supplier.company FROM product
LEFT JOIN supplier
ON product.supplier_id = supplier.id
UNION
SELECT product.name, supplier.company FROM product
RIGHT JOIN supplier
ON product.supplier_id = supplier.id;
```

```

+-----+-----+
| name           | company |
+-----+-----+
| MacBook Pro M4 14-in | Apple   |
| MacBook Air M3       | Apple   |
| Mac Mini M3          | Apple   |
| Mac Studio           | Apple   |
| iPhone 17 Pro         | Apple   |
| iPhone 17             | Apple   |
| iMac                 | Apple   |
| NZXT Mid Tower        | NZXT    |
| Corsair Full Tower    | Corsair |
| Dell Model 4823       | Dell    |
| Dell XPS Model 7823   | Dell    |
| Dell Dock Model [4807] | Dell    |
| One&One Touch 200     | NULL    |
| One2One USB-A Hub     | NULL    |
| One4One USB-A Hub     | NULL    |
| NULL                 | Nvidia  |
+-----+-----+
16 rows in set (0.04 sec)

```

The result set above displays all products and suppliers, substituting a NULL value when there is no match between the two tables. To replace NULL with a more descriptive value, we can use the COALESCE() function. This function returns the first non-NULL value from a provided list of values. For example, the following call to the function will return the supplier's company name if it is not NULL; if it is NULL, it will return the word 'Unknown':

```
COALESCE(supplier.company, 'Unknown')
```

Modify the original statement to handle NULL values for product and company names and provide an alias for the product name column:

```

SELECT product.name, COALESCE(supplier.company, 'Unknown')
AS company
FROM product
LEFT JOIN supplier ON product.supplier_id = supplier.id
UNION
SELECT COALESCE(product.name, 'No product'), supplier.company
FROM product
RIGHT JOIN supplier ON product.supplier_id = supplier.id;
+-----+-----+
| name           | company |
+-----+-----+
| MacBook Pro M4 14-in | Apple   |
| MacBook Air M3       | Apple   |
| Mac Mini M3          | Apple   |
| Mac Studio           | Apple   |
| iPhone 17 Pro         | Apple   |
| iPhone 17             | Apple   |
| iMac                 | Apple   |
| NZXT Mid Tower        | NZXT    |
| Corsair Full Tower    | Corsair |
| Dell Model 4823       | Dell    |
| Dell XPS Model 7823   | Dell    |

```

Understanding Joins and Unions

```
| Dell Dock Model [4807] | Dell |
| One&One Touch 200 | Unknown |
| One2One USB-A Hub | Unknown |
| One4One USB-A Hub | Unknown |
| No product | Nvidia |
+-----+-----+
16 rows in set (0.01 sec)
```

16.13 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL joins and unions:

<https://www.answerptopia.com/fqpw>



16.14 Reference points

The main points covered in this chapter are as follows:

- **Introduction to Joins**

- Joins enable relationships between tables.
- Foreign keys connect rows in different tables, linking a primary key in one table to a column in another.
- The `FOREIGN KEY` constraint establishes relationships between tables.

- **Cross Join**

- Produces all possible combinations of rows from two tables.
- Useful for exploratory analysis but inefficient for practical queries.

- **Inner Join**

- Combines rows from two tables based on matching values in specified columns.
- Returns only rows with matches in both tables.

- **Left Join**

- Returns all rows from the left table and matching rows from the right table.
- Unmatched rows in the right table are filled with `NULL`.

- **Right Join**

- Returns all rows from the right table and matching rows from the left table.
- Unmatched rows in the left table are filled with `NULL`.

- **Union Operator**

- Combines result sets from multiple `SELECT` queries into a single set.
- The number and order of columns must match in all queries.
- Data types must be compatible.

- `UNION ALL` retains duplicate rows, whereas `UNION` eliminates them.
- **Full Outer Join Simulation**
 - MySQL does not natively support `FULL OUTER JOIN`.
 - Achieved by combining `LEFT JOIN` and `RIGHT JOIN` with `UNION`.
 - Includes all rows from both tables, with unmatched rows filled with `NULL`.
 - Use the `COALESCE ()` function to replace `NULL` values with default or descriptive placeholders.
- **Populating Foreign Keys**
 - Use `UPDATE` with subqueries and string functions like `CONCAT ()` to link rows across tables.

17. An Introduction to MySQL Views

When we execute a `SELECT` query, the MySQL server returns a result set that meets the specified search criteria. These results can be used in various ways, such as displaying them to users within an application or as input for further operations like updates or insertions.

In this chapter, we will explore views in MySQL, which provide another way to interact with the results of `SELECT` queries.

17.1 An introduction to MySQL views

MySQL views allow us to create reusable virtual tables from `SELECT` queries. Once created, we can search, update, and modify result sets by referencing the view, just as we would with any other table, rather than repeating the associated `SELECT` statement.

For example, consider two tables named *customer* and *order*, as illustrated in Figure 17-1 below:

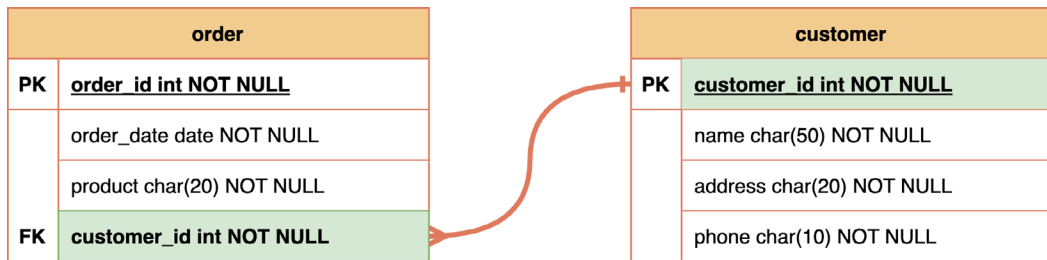


Figure 17-1

To retrieve a result set matching customers with their orders, we can use the following `SELECT` statement:

```
SELECT order.date, order.product, customer.name
FROM order
INNER JOIN customer
ON order.customer_id = customer.id;
```

The result set from the above query will include order dates, products, and the corresponding customer names. Typically, we would need to execute this statement each time we want to obtain the same results. Instead, we can create a view that includes these three columns, allowing us to reference the result set as if it were a database table.

To create a view in MySQL, we use the `CREATE VIEW` statement. The syntax for this is as follows:

```
CREATE VIEW <table_name> AS SELECT <conditions>;
```

To create a view called *customer_orders* from the previous customer order query, we would use the following `CREATE VIEW` statement:

```
CREATE VIEW customer_orders
AS SELECT order.date, order.product, customer.name
FROM order
INNER JOIN customer
```

```
ON order.customer_id = customer.id;
```

The diagram in Figure 17-2 illustrates the relationship between the two tables and the view:

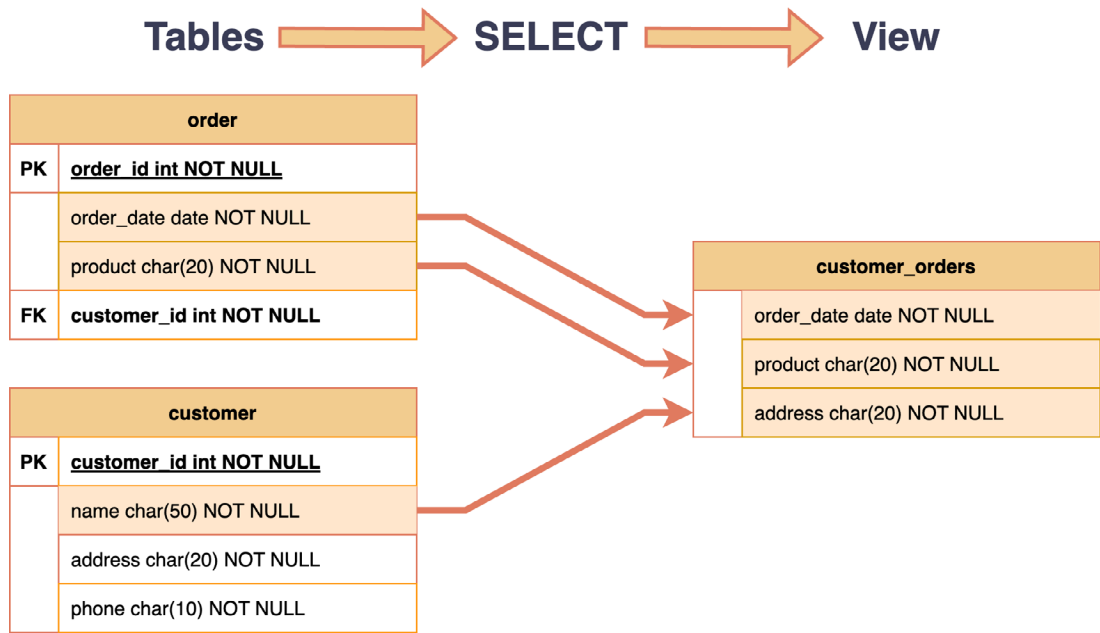


Figure 17-2

Once created, the view’s result set updates automatically whenever accessed and can, with a few exceptions, be treated like an ordinary database table.

17.2 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven’t, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampledb;"
mysql -u root -p sampledb < mysql_views.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampledb
```

17.3 Creating a basic view

The simplest type of view pulls data from a single table. For instance, you may have a standard SELECT statement that formats data from a table in a specific way. The following SELECT statement, for example, retrieves supplier data with some additional formatting:

```
SELECT CONCAT (UPPER(company), ' - ', address) FROM supplier;
+-----+
| CONCAT (UPPER(company), ' - ', address) |
+-----+
```

```
| APPLE - Cupertino, CA |
| DELL - Round Rock, TX |
| NZXT - City of Industry, CA |
| CORSAIR - Milpitas, CA |
| NVIDIA - Santa Clara, CA |
+-----+
5 rows in set (0.00 sec)
```

If we want to repeat this selection, we could re-enter the same `SELECT` statement each time. However, a more efficient approach would be to create a virtual table based on this `SELECT` statement. This would allow us to reference it by name, just like any other table, without needing to retype the lengthy query. We can accomplish this by using views.

For instance, we can create a view called *supplier_summary* using the `SELECT` statement from before:

```
CREATE VIEW supplier_summary
AS SELECT CONCAT(UPPER(company), ' - ', address) FROM supplier;
```

We have now created a virtual table from our `SELECT` statement, which we can reference in other `SELECT` statements:

```
SELECT * FROM supplier_summary;
+-----+
| CONCAT(UPPER(company), ' - ', address) |
+-----+
| APPLE - Cupertino, CA |
| DELL - Round Rock, TX |
| NZXT - City of Industry, CA |
| CORSAIR - Milpitas, CA |
| NVIDIA - Santa Clara, CA |
+-----+
5 rows in set (0.01 sec)
```

Similarly, we could create a view comprising all the desktop computers in the *product* table listed alphabetically:

```
CREATE VIEW desktop_computers
AS SELECT name FROM product
WHERE description LIKE '%desktop%' ORDER BY name;
```

A query of the *desktop_computers* view will generate the following result set:

```
SELECT * FROM desktop_computers;
+-----+
| name |
+-----+
| Dell Model 4823 |
| Dell XPS Model 7823 |
| iMac |
| Mac Mini M3 |
| Mac Studio |
+-----+
5 rows in set (0.00 sec)
```

Views offer similar querying capabilities to those of tables. For instance, we can use `SELECT` to display only Mac-based desktop computers, as demonstrated below:

```
SELECT * FROM desktop_computers WHERE name LIKE '%Mac%';
```

name
iMac
Mac Mini M3
Mac Studio

```
3 rows in set (0.01 sec)
```

17.4 Joins and views

In the “*Understanding Joins and Unions*” chapter, we learned how to create a join between two tables to extract data from both. By crafting a SELECT statement that joined the tables, we matched the supplier ID column in each table to identify corresponding rows. This join allowed us to display product details from the product table alongside the related supplier information from the supplier table. The statement below utilizes an inner join to list the product names and supplier companies for products manufactured by Apple:

```
SELECT product.name, supplier.company
FROM product
INNER JOIN supplier
ON product.supplier_id = supplier.id
WHERE supplier.company = 'Apple';
```

name	company
MacBook Pro M4 14-in	Apple
MacBook Air M3	Apple
Mac Mini M3	Apple
Mac Studio	Apple
iPhone 17 Pro	Apple
iPhone 17	Apple
iMac	Apple

```
7 rows in set (0.00 sec)
```

We can create a virtual table using the CREATE VIEW statement based on a join operation, similar to how we created a view from a single SELECT statement:

```
CREATE VIEW join_demo
AS SELECT product.name, supplier.company
FROM product
INNER JOIN supplier
ON product.supplier_id = supplier.id
WHERE supplier.company = 'Apple';
```

```
SELECT * FROM join_demo;
```

name	company
MacBook Pro M4 14-in	Apple
MacBook Air M3	Apple
Mac Mini M3	Apple
Mac Studio	Apple
iPhone 17 Pro	Apple
iPhone 17	Apple